

MATH70134: Mathematical Foundations of Machine Learning

Nicolas Boullé

Last updated: November 30, 2025

Table of contents

Chapter 1 – Introduction into learning	4
1.1 Preliminaries	4
1.2 Introduction to learning setup	4
1.3 Statistical learning formulation	6
1.3.1 Formulation	6
1.3.2 Estimating f	7
1.3.2.1 Parametric approaches	7
1.3.2.2 Non-parametric approaches	8
1.4 The generalization error	10
1.4.1 Intuition and heuristic approach	10
1.4.2 Background: concentration inequalities	13
1.4.2.1 Markov’s inequality	13
1.4.2.2 Chernoff’s bounding	14
1.4.2.3 Hoeffding’s inequality	15
1.4.2.4 McDiarmid’s inequality	17
1.4.3 Applying concentration inequalities to the estimators	20
1.5 Rademacher complexity	23
1.6 Maximum likelihood estimation	27
1.6.1 Conditional log-likelihood	28
1.6.2 Linear regression and maximum likelihood	28
1.7 When training the model	29

1.7.1 Regularisation	29
1.7.1.1 A Bayesian perspective on regularization	30
1.7.2 Hyperparameters and validation sets	30
1.7.3 Challenges in machine learning	32
Chapter 2 – Optimisation overview	35
2.1 Definitions	35
2.1.1 Optimality conditions	36
2.1.2 Convexity	37
2.1.3 L -smoothness	38
2.1.4 Strong convexity	40
2.2 Iterative methods: the intuition	41
2.3 Gradient descent	42
2.3.1 Gradient descent - convergence proofs	42
2.3.2 Gradient descent with exact line search	45
2.4 Accelerating gradient descent using momentum	45
2.4.1 The issue with gradient descent	46
2.4.2 Improving convergence with momentum	48
2.4.2.1 The analysis through eigenvectors	49
2.4.2.2 Variations on momentum	55
2.4.3 Second-order methods - preconditioning	55
Chapter 3 – Introduction to deep learning	58
3.1 Introduction	58
3.2 Fully connected neural networks	59
3.2.1 Learning the XOR operation	60
3.2.2 A classification example	64
3.3 Optimizing neural networks	66
3.3.1 Cost functions	66
3.3.1.1 Conditional statistics and mean absolute error (MAE)	67
3.3.1.2 Output layers	68
3.3.2 Backpropagation algorithm	70
3.3.3 Stochastic gradient descent	72
3.3.4 Adam	74
3.3.4.1 Momentum	75

3.3.4.2 Nesterov momentum	77
3.3.4.3 Adaptive learning rates algorithms	77
3.3.4.4 Adam algorithm	77
3.4 Overview of architectures and operations	77
3.4.1 Activation functions	81
3.4.2 Weights initialization	82
3.4.3 Dropout	83
3.4.4 Convolution neural networks (CNNs)	84
3.4.4.1 Convolution	84
3.4.4.2 Pooling	88
 Chapter 4 – Approximation theory of neural networks	 89
4.1 Universal approximation theorem	89
4.1.1 Univariate case	89
4.1.2 Multivariate case	90
4.1.3 Single-layer neural networks are universal approximators	93
4.2 Approximation power of neural networks	96

Chapter 1: Introduction into learning

1.1 Preliminaries

Goals. We will introduce the general setting we consider that of supervised learning. We will formalize the concept of learning. We will discuss generalization bounds and try to understand the capacity of the model and the relations between it and overfitting and underfitting.

Literature

- We recommend [James et al., 2013, Chapt. 2].
- We also recommend [Goodfellow et al., 2016, Chapt. 5].
- Additional lecture notes that may be useful for information on the generalization error and VC dimension (which we don't discuss here): <https://people.cs.umass.edu/~domke/courses/sm12010/10theory.pdf>

1.2 Introduction to learning setup

Some notation. We denote by $\mathcal{X}$ the set of all possible examples or instances. $\mathcal{X}$ is sometimes also referred to as the input space. The set of all possible labels or target values is denoted by $\mathcal{Y}$. A function $f : \mathcal{X} \rightarrow \mathcal{Y}$ is a mapping from $\mathcal{X}$ to $\mathcal{Y}$. It can be referred to as the concept, predictor, hypothesis, or classifier. A hypothesis class is a set of functions we may wish to learn and is denoted by $\mathcal{H}$.

Classification and regression. In its most basic form, the goal of machine learning is to learn an underlying function:

$$f : \mathcal{X} \rightarrow \mathcal{Y},$$

where $\mathcal{X}$ is a space of inputs or features and $\mathcal{Y}$ is the space of outputs. The input space $\mathcal{X}$ is usually modelled as a metric space (such as $\mathbb{R}^d$), and the inputs can represent anything from images, time series data, customer data and more. The output space $\mathcal{Y}$ could then correspond to quantitative values, such as temperatures, stock prices or qualitative or categorical values, such as $\{0, 1, \dots, 9\}$ or $\{\text{dog}, \text{cat}\}$. In the first case we refer to it as regression, while in the latter we call it classification.

A classifier with only two possible output classes (e.g., $\mathcal{Y} = \{0, 1\}$) is called a binary classifier; a classifier with more than two output classes is called a multiclass or multinomial classifier. Typically, the function h approximating f is chosen from some predetermined set of functions, the so-called hypothesis class $\mathcal{H} \subset \{g : \mathcal{X} \rightarrow \mathcal{Y}\}$. The learning algorithms will “navigate” this space to find, as well as is possible, the best approximation h^* to $f \in \mathcal{F}$. This set of candidate functions $\mathcal{H}$ can be seen as our best estimate of the true function class $\mathcal{F}$.

Supervised vs. unsupervised learning. Most learning problems fall into one of two categories: supervised or unsupervised. For supervised learning, for each observation $x_i \in \mathcal{X}$, there is an associated output $y_i \in \mathcal{Y}$, for $1 \leq i \leq n$. We wish to fit a model that relates the response to the predictors, with the aim of accurately predicting the response for future observations. Unsupervised learning describes the somewhat more challenging situation in which for every observation $1 \leq i \leq n$, we observe a vector of measurements $x_i \in \mathcal{X}$, but no associated response y_i . It is not possible to fit a linear regression model, since there is no response variable to predict. In this setting, we are in some sense working blind; the situation is referred to as unsupervised because we lack a response variable that can supervise our analysis. In this setting we typically seek to understand the relationships between the variables. One example of unsupervised learning is clustering where the goal is to ascertain, on the basis of $\{x_1, \dots, x_n\}$, whether the observations fall into relatively distinct groups. For example, given a dataset of customer information you may want to group “similar” customers. Most of the course will be focused on *supervised learning*.

Loss functions. The notion of best fit is formalized by using a loss function. A loss function $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$ measures the mismatch between a prediction $h(x)$ on a given input $x \in \mathcal{X}$ and an element $y \in \mathcal{Y}$.

Definition 1.1 (Loss function). A loss function $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$ is a function that satisfies some or all properties of a metric.

Definition 1.2 (Metric). A function $d : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ is a metric if for any $y_1, y_2, y_3 \in \mathcal{Y}$,

1. $d(y_1, y_2) \geq 0$ for all y_1, y_2 (nonnegativity),

2. $d(y_1, y_2) = 0$ if and only if $y_1 = y_2$ (definiteness),
3. $d(y_1, y_2) = d(y_2, y_1)$ (symmetry),
4. $d(y_1, y_2) \leq d(y_1, y_3) + d(y_3, y_2)$ (triangle inequality).

1.3 Statistical learning formulation

1.3.1 Formulation

A statistical learning model is formulated as follows:

- Consider an hypothesis class $\mathcal{H}$ of functions from $\mathcal{X} \rightarrow \mathcal{Y}$.
- Assume that the data is distributed as $(x, y) \sim \mathcal{D}$, i.e. it follows some unknown joint data-generating distribution $D(x, y)$.
- Consider a loss function $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ measuring the goodness of the fit.

Definition 1.3 (Risk). Let $\mathcal{D}$ be a data-generating distribution over $\mathcal{X} \times \mathcal{Y}$, and $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$ be a loss function. The *generalization error* or *risk* associated with the hypothesis $h \in \mathcal{H}$ is defined as

$$R(h) := \mathbb{E}_{(x,y) \sim \mathcal{D}}[L(h(x), y)]. \quad (1.1)$$

Remark 1.4. Statistical learning seeks to find $h \in \mathcal{H}$, that minimizes the population risk, i.e., it solves,

$$h^* = \arg \min_{h \in \mathcal{H}} R(h). \quad (1.2)$$

Now the issue with the above is that $\mathcal{D}$ is unknown: we do not know the true data distribution. This makes the above problem intractable. We therefore approximate h^* by minimizing the *empirical risk* instead of the risk.

Definition 1.5 (Empirical risk). Consider a sample $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$ distributed i.i.d. from $\mathcal{D}$, and a loss function $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$. The *empirical risk* associated with the hypothesis $h \in \mathcal{H}$ is defined as

$$\hat{R}_S(h) := \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i). \quad (1.3)$$

One would thus aim to find a function h among a set of candidate functions $\mathcal{H}$ that minimizes the loss when applying the function to the training data S :

$$\hat{h} = \arg \min_{h \in \mathcal{H}} \hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i). \quad (1.4)$$

By the law of large numbers, we can expect that for each $h \in \mathcal{H}$

$$R(h) := \mathbb{E}_{(x,y) \sim \mathcal{D}}[L(h(x), y)] \approx \hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i), \quad (1.5)$$

when n is large enough, with high probability. The general search space $\mathcal{H}$ is possibly infinite dimensional (we are optimizing over functions). Remember, one wants to select the $\hat{h}$ according to Eq. (1.4). However, for this $\hat{h}$ we want the full error in Eq. (1.2) to be small. The difference between the two, $R(\hat{h}) - \hat{R}_S(\hat{h})$, is called the generalization error. In practice, since we do not know $\mathcal{D}$, we can approximate the generalization error by considering a second sample of data $(\hat{x}_i, \hat{y}_i)_{i=1}^m$ (we can call this the test sample or validation sample) and then the generalization error is given by,

$$\frac{1}{m} \sum_{i=1}^m L(h(\hat{x}_i), \hat{y}_i) - \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i). \quad (1.6)$$

1.3.2 Estimating f

More generally, how do we estimate some function $f \in \mathcal{F}$?

1.3.2.1 Parametric approaches

Learning processes do not take place in a vacuum. Some structure needs to be present at the beginning of the process. Parametric methods involve a two-step model-based approach:

1. First, we make an assumption about the functional form, or shape, of f . In our formal development, we assume that this structure takes the form of a class of functions (e.g., a space of polynomials, of splines, etc.). Before we referred to this as the hypothesis class $\mathcal{H}$.
2. After the model has been selected we need a procedure that uses the training data to fit or train the model. The goal of the learning process is thus to find the best approximation of h^* within this class.

The model-based approach just described is referred to as parametric; it reduces the problem of estimating f down to one of estimating a set of parameters: suppose the function f has some parametric form $h_\theta(\cdot)$ so that instead of Eq. (1.4) we solve,

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(h_\theta(x_i), y_i). \quad (1.7)$$

Here, $\mathcal{H} = \{h_\theta \mid \theta \in \Theta\}$ is the hypothesis class.

Learning results in optimization problems. More generally, in machine learning we typically have the following problem.

$$\hat{\theta} = \arg \min_{\theta \in \Theta} (F(\theta)), \quad (1.8)$$

for some function F depending on the sample space $\mathcal{X}$, the probability distribution $\mathcal{D}$, and the samples (x_i, y_i) . In other words, learning amounts to some kind of optimization of an objective.

Remark 1.6 (Some remarks on $\mathcal{D}$).

- In the most general scenario of supervised learning, the distribution $\mathcal{D}$ is defined over $\mathcal{X} \times \mathcal{Y}$ and the training data $\{x_i, y_i\}_{i=1}^N$ is a sample drawn i.i.d. according to $\mathcal{D}$. Within this setting, the output label is a probabilistic function of the input. The stochastic scenario captures many real-world problems where the label of an input point is not unique. For example, if we seek to predict gender based on input pairs formed by the height and weight of a person, then the label will typically not be unique. For most pairs, both male and female are possible genders. For each fixed pair, there would be a probability distribution of the label being male.
- When the label of a point can be uniquely determined by some measurable function $c : \mathcal{X} \rightarrow \mathcal{Y}$ (with probability one), then the scenario is said to be deterministic. In that case, it suffices to consider a distribution $\mathcal{D}$ over the input space with a sample $\{x_i\}_{i=1}^n$ drawn from $\mathcal{D}$ and $y_i = c(x_i)$, a deterministic function of the input. Many learning problems can be formulated within this deterministic scenario.

1.3.2.2 Non-parametric approaches

Non-parametric methods do not make explicit assumptions about the functional form of f . Instead, they seek an estimate of f that gets as close to the data points

as possible without being too rough or wiggly. Such approaches can have a major advantage over parametric approaches: by avoiding the assumption of a particular functional form for f , they have the potential to accurately fit a wider range of possible shapes. Any parametric approach brings with it the possibility that the functional form used to estimate f is very different from the true f , in which case the resulting model will not fit the data well. In contrast, non-parametric approaches completely avoid this danger, since essentially no assumption about the form of f is made. But non-parametric approaches do suffer from a major disadvantage: since they do not reduce the problem of estimating f to a small number of parameters, a very large number of observations (far more than is typically needed for a parametric approach) is required in order to obtain an accurate estimate for f . An example of nonparametric approach is Gaussian process (see the dedicated tutorial).

Example 1.7 (Regression). Among the most used models is linear regression. This amounts to finding a straight line that best approximates a functional relationship presumed to be implicit in a set of data points in $\mathbb{R}^2$:

$$S = \{(x_1, y_1), \dots, (x_n, y_n)\}.$$

Note that in this case we thus consider each of the x_i, y_i to be one-dimensional. The yardstick that can be used to measure how good an approximation the function $y = h(x)$ with $h(x) = ax + b$ is the least squares error or the *mean squared error* (MSE). The best line is the one that minimizes,

$$\hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n (y_i - ax_i - b)^2. \quad (1.9)$$

Define the following

$$Y = [y_1, \dots, y_n]^\top, X = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \theta = [b, a]^\top. \quad (1.10)$$

Then the loss function is given by

$$\begin{aligned} \hat{R}_S(h) &= \frac{1}{n} \|Y - X\theta\|_2^2 \\ &= \frac{1}{n} (Y - X\theta)^\top (Y - X\theta) \\ &= \frac{1}{n} (Y^\top Y - Y^\top X\theta - \theta^\top X^\top Y + \theta^\top X^\top X\theta). \end{aligned} \quad (1.11)$$

The gradient of the loss is,

$$\nabla_{\theta} \hat{R}_S(f) = \frac{1}{n}(-2X^{\top}Y + 2X^{\top}X\theta). \quad (1.12)$$

Setting the gradient of the loss to zero and solving for θ we get,

$$-2X^{\top}Y + 2X^{\top}X\theta = 0 \implies X^{\top}Y = X^{\top}X\theta \quad (1.13)$$

so that the least-squares estimator is given by (assuming $(X^{\top}X)$ is invertible),

$$\hat{\theta}_{LS} = (X^{\top}X)^{-1}X^{\top}Y. \quad (1.14)$$

Note that in this case,

- The sample is given by $(x_i, y_i) \in \mathbb{R} \times \mathbb{R}$, $i = 1, \dots, N$.
- A function class is given by $\mathcal{H} := \{h_{\theta}(\cdot) = \langle [1, \cdot], \theta \rangle \mid \theta \in \mathbb{R}^2\}$.
- A loss function given by $L(h_{\theta}(x), y) = (y - f_{\theta}(x))^2$.

1.4 The generalization error

What we care about when solving these kinds of learning problems is the question of: how well does the function we learned over our data samples generalize to unseen data? In other words, how close is the empirical risk to the true risk?

1.4.1 Intuition and heuristic approach

Remember that we already discussed that we want the empirical risk as approximated over the sampled data to be close to the true empirical risk. We typically estimate the generalization error of a machine learning model by measuring the performance on a test set: a set of examples collected separately from the train data and which the model has not seen during training time. We typically do assume that the training and test data come from the same distribution (although there are recent lines of work that allow for distribution shift: a different distribution during train and test time; but this is tricky to make work and requires a lot of assumptions).

Our goals are thus as follows: i) make the training error small; ii) make the gap between training and test error small. These two factors correspond to two central challenges in machine learning: underfitting and overfitting. Underfitting happens

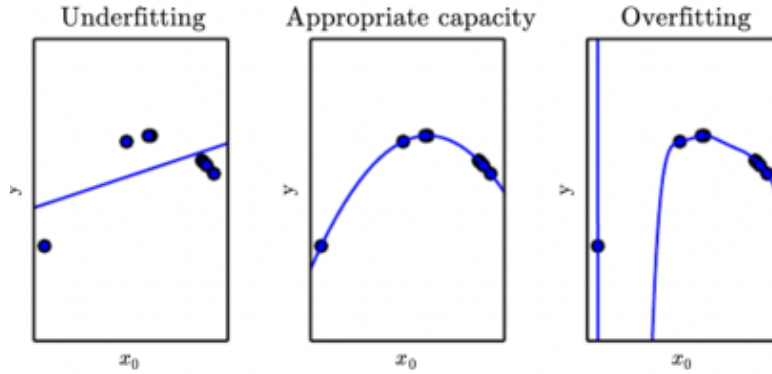


Figure 1.1: We fit three models to this example training set. The training data was generated synthetically, by randomly sampling 2 values and choosing y deterministically by evaluating a quadratic function. (Left) A linear function fit to the data suffers from underfitting—it cannot capture the curvature that is present in the data. (Center) A quadratic function fit to the data generalizes well to unseen points. It does not suffer from a significant amount of overfitting or underfitting. (Right) A polynomial of degree 9 fit to the data suffers from overfitting. Here we used the Moore–Penrose pseudoinverse to solve the underdetermined normal equations. The solution passes through all the training points exactly, but we have not been lucky enough for it to extract the correct structure. It now has a deep valley between two training points that does not appear in the true underlying function. It also increases sharply on the left side of the data, while the true function decreases in this area. Picture taken from <https://www.deeplearningbook.org/contents/ml.html>.

when the model does not obtain a sufficiently low error on the train dataset. Overfitting on the other hand corresponds to a very low error on the train dataset, but a large test data error (and hence large generalization error). We can control how much the model overfits or underfits by controlling its capacity. Informally, the capacity is the model’s ability to fit a wide variety of functions. It depends on both the choice of hypothesis class (certain model classes can only represent certain functions), however it also depends on the optimisation process (i.e. how we find the parameters of the model). While the former’s relationship to capacity is clear, the latter can also be used to control the capacity of the final model.

Fig. 1.1 is a standard image you see in machine learning. It compares a linear, quadratic and degree-9 predictor function fit to some data. The linear function underfits as it is unable to capture the curvature in the model, the degree-9 model on the other hand may not be able to generalize well as it overfits on the train data. Another classic image we often see in the study of ML is in Fig. 1.2.

Example 1.8. Consider the following function

$$f(x) = \sin(x), \quad (1.15)$$

and a dataset (x_i, y_i) where $x_i \in [-1, 1]$ uniformly spaced with $\Delta x = 0.1$ and

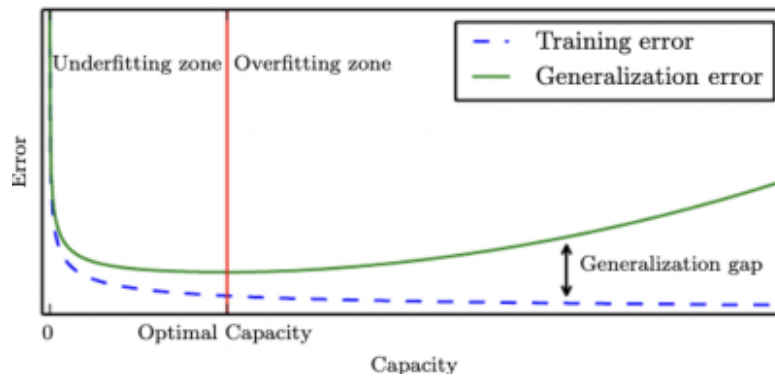


Figure 1.2: Typical relationship between capacity and error. Training and test error behave differently. At the left end of the graph, training error and generalization error are both high. This is the underfitting regime. As we increase capacity, training error decreases, but the gap between training and generalization error increases. Eventually, the size of this gap outweighs the decrease in training error, and we enter the overfitting regime, where capacity is too large, above the optimal capacity. Picture taken from <https://www.deeplearningbook.org/contents/ml.html>.

$y_i = f(x_i) + \epsilon$ where $\epsilon \sim \mathcal{N}(0, 0.1)$ The code to replicate the example is given below:

```

1 # generate data
2 x_data = np.arange(-1,1,0.1)
3 y_data = np.sin(np.pi*x_data) + np.random.normal(0,0.1,len(x_data))
4 plt.plot(x_data, y_data, 'o', label = 'Data')
5 plt.plot(x_data, np.sin(np.pi*x_data), label = 'True function')
6 plt.legend()
7 plt.show()
8
9 # fit a polynomial
10 deg = 3
11 # p(x) = p[0] * x**deg + ... + p[deg]
12 z = np.polyfit(x_data, y_data, deg)
13 plt.plot(x_data, y_data, 'o', label = 'Data')
14 plt.plot(x_data, np.sin(np.pi*x_data), label = 'True function')
15 pol_data = np.zeros(x_data.shape)
16 for r in range(deg):
17     pol_data += z[r] * x_data**(deg-r)
18 plt.plot(x_data, pol_data, label = 'Fitted function')
19 plt.legend()
20 plt.show()

```

You can change the degree of the polynomial yourself. We observe the results in Fig. 1.3.

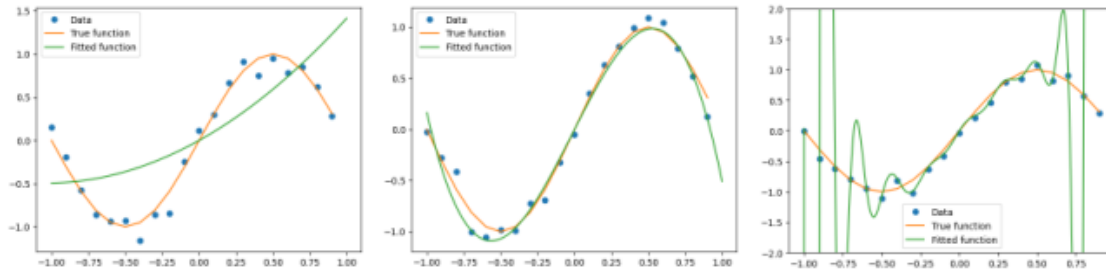


Figure 1.3: A polynomial of degree 2 (left), 3 (center) and 19 (right). We first underfit and then overfit; the best fit is achieved with a degree of 3.

Occam's razor. Improving the generalization of machine learning models relate to early ideas in philosophy: specifically the idea of Occam's razor (1287-1347) which states that among competing hypotheses that explain known observations equally well, we should choose the simplest one. It also relates to the idea that the real world cannot be too complex.

1.4.2 Background: concentration inequalities

Concentration inequalities give probability bounds for a random variable to be concentrated around its mean, or for it to deviate from its mean or some other value.

1.4.2.1 Markov's inequality

Example 1.9. The number of calories in drinks sold on campus is a random variable X with range $[0, 110]$. Suppose that the average of the number of calories $\mathbb{E}[X] = 50$. At most what fraction of drinks has more than 100 calories? The answer is $1/2$.

Why? If more than $1/2$ of drinks have more than 100 calories, the average would exceed 50 (consider the extreme example of the other $1/2$ having zero calories). Suppose now the average is $\mathbb{E}[X] = 10$. What fraction of drinks has more than 100 calories? The answer is $1/10$ th. If more than $1/10$ th have over 100 calories, the average would exceed 10. Note, if X could be negative, we could not provide similar bounds.

Markov's inequality formalizes the above for a nonnegative random variable.

Proposition 1.10 (Markov's inequality). Let X be a nonnegative random variable. Then for any $\epsilon > 0$,

$$\mathbb{P}(X \geq \epsilon) \leq \frac{\mathbb{E}[X]}{\epsilon}. \quad (1.16)$$

This allows us to bound the tail of a distribution using Chebyshev's inequality.

Proposition 1.11 (Chebyshev's inequality). Let X be a random variable with mean μ and variance σ^2 . Then for any $\epsilon > 0$,

$$\mathbb{P}(|X - \mu| \geq \epsilon) \leq \frac{\sigma^2}{\epsilon^2}. \quad (1.17)$$

Proof. Consider the random variable $(X - \mu)^2$ which is nonnegative. Apply Markov's inequality to it:

$$\mathbb{P}((X - \mu)^2 \geq \epsilon^2) \leq \frac{\mathbb{E}[(X - \mu)^2]}{\epsilon^2} = \frac{\sigma^2}{\epsilon^2}. \quad (1.18)$$

□

1.4.2.2 Chernoff's bounding

Given a random variable X and $\epsilon \geq 0$, Chernoff's bounding technique consists of proceeding as follows to bound $\mathbb{P}(X \geq \epsilon)$. For any $t > 0$, first Markov's inequality is used to bound $\mathbb{P}(X \geq \epsilon)$ as follows:

$$\mathbb{P}(X \geq \epsilon) = \mathbb{P}(e^{tX} \geq e^{t\epsilon}) \leq e^{-t\epsilon} \mathbb{E}[e^{tX}]. \quad (1.19)$$

We then aim to find an explicit upper bound $\mathbb{E}[e^{tX}] \leq g(t)$ for some function $g : \mathbb{R}^+ \rightarrow \mathbb{R}_+$, using the moment generating function of X . Finally, we select t to minimize $e^{-t\epsilon}g(t)$.

Example 1.12 (Chernoff bound for normal variable). Let $X \sim \mathcal{N}(0, 1)$ and $\epsilon > 0$. We can use Chernoff's bounding technique to bound $\mathbb{P}(X \geq \epsilon)$. We have

$$\mathbb{P}(X \geq \epsilon) \leq e^{-t\epsilon} \mathbb{E}[e^{tX}], \quad (1.20)$$

and we can find an upper bound for $\mathbb{E}[e^{tX}]$ by using the fact that $\mathbb{E}[e^{tX}]$ is the moment generating function of X . We have

$$\mathbb{E}[e^{tX}] = e^{t\mu + \frac{1}{2}t^2\sigma^2} = e^{\frac{1}{2}t^2}, \quad (1.21)$$

where we have used $\mu = 0$ and $\sigma^2 = 1$. Therefore,

$$\mathbb{P}(X \geq \epsilon) \leq e^{-t\epsilon} e^{\frac{1}{2}t^2}. \quad (1.22)$$

Then, we can find the optimal t by minimizing $e^{-t\epsilon} e^{\frac{1}{2}t^2}$. We have

$$\frac{d}{dt} \left(e^{-t\epsilon} e^{\frac{1}{2}t^2} \right) = e^{-t\epsilon} e^{\frac{1}{2}t^2} (-\epsilon + t) = 0 \implies t = \epsilon. \quad (1.23)$$

Then, we have

$$\mathbb{P}(X \geq \epsilon) \leq e^{-\epsilon^2} e^{\frac{1}{2}\epsilon^2} = e^{-\epsilon^2/2}. \quad (1.24)$$

For large ϵ , the probability that X takes values greater than ϵ is exponentially small.

1.4.2.3 Hoeffding's inequality

We first present Hoeffding's inequality, whose proof makes use of the general Chernoff bounding technique. For Hoeffding's inequality $\mathbb{E}[e^{tX}]$ is bounded as given below.

Lemma 1.13 (Hoeffding's lemma). Let X be a real random variable such that $X \in [a, b]$ almost surely. Then, for any $t > 0$, the following inequality holds:

$$\mathbb{E}[e^{t(X - \mathbb{E}[X])}] \leq e^{\frac{t^2(b-a)^2}{8}}. \quad (1.25)$$

Proof. Note that we can prove the statement when $\mathbb{E}[X] = 0$ since we can always consider $X - \mathbb{E}[X]$ instead of X . Thus, we assume that $\mathbb{E}[X] = 0$. Note that in this case, $X \in [a, b]$ with $a \leq 0 \leq b$.

By the convexity of $x \mapsto e^x$, for all $x \in [a, b]$, the following holds:

$$e^{tx} \leq \frac{b-x}{b-a} e^{ta} + \frac{x-a}{b-a} e^{tb}. \quad (1.26)$$

Then, using $\mathbb{E}[X] = 0$,

$$\mathbb{E}[e^{tX}] \leq \mathbb{E} \left[\frac{b-X}{b-a} e^{ta} + \frac{X-a}{b-a} e^{tb} \right] = \frac{b}{b-a} e^{ta} + \frac{-a}{b-a} e^{tb} = e^{\phi(t)}, \quad (1.27)$$

where

$$\phi(t) = \log \left(\frac{b}{b-a} e^{ta} + \frac{-a}{b-a} e^{tb} \right) = ta + \log \left(\frac{b}{b-a} + \frac{-a}{b-a} e^{t(b-a)} \right). \quad (1.28)$$

For any $t > 0$, the first and second derivative of ϕ are given below:

$$\phi'(t) = a - \frac{ae^{t(b-a)}}{\frac{b}{b-a} - \frac{a}{b-a} e^{t(b-a)}} = a - \frac{a}{\frac{b}{b-a} e^{-t(b-a)} - \frac{a}{b-a}}, \quad (1.29)$$

$$\phi''(t) = \frac{-abe^{-t(b-a)}}{\left(\frac{b}{b-a}e^{-t(b-a)} - \frac{a}{b-a}\right)^2} = \frac{\alpha(1-\alpha)e^{-t(b-a)}(b-a)^2}{((1-\alpha)e^{-t(b-a)} + \alpha)^2} \quad (1.30)$$

$$= \frac{\alpha}{(1-\alpha)e^{-t(b-a)} + \alpha} \frac{(1-\alpha)e^{-t(b-a)}}{(1-\alpha)e^{-t(b-a)} + \alpha} (b-a)^2, \quad (1.31)$$

where $\alpha = -a/(b-a)$. Note that $\phi(0) = \phi'(0) = 0$ and that $\phi''(t) = u(1-u)(b-a)^2$ where $u = \frac{\alpha}{|(1-\alpha)e^{-t(b-a)} + \alpha|}$ so that $1-u = \frac{(1-\alpha)e^{-t(b-a)}}{|(1-\alpha)e^{-t(b-a)} + \alpha|}$. We will use the inequality of arithmetic and geometric means which states that for non-negative numbers x and y :

$$\sqrt{xy} \leq \frac{x+y}{2}, \quad (1.32)$$

and thus $xy \leq (x+y)^2/4$. Since u is upper-bounded by 1 and is non-negative (since $a \leq 0$), using the inequality we have

$$u(1-u) \leq \frac{1}{4}. \quad (1.33)$$

Then,

$$\phi''(t) \leq \frac{(b-a)^2}{4}. \quad (1.34)$$

Thus, by the second order expansion of function ϕ (using Taylor's theorem), there exists $\theta \in [0, t]$ such that:

$$\phi(t) = \phi(0) + t\phi'(0) + \frac{t^2}{2}\phi''(\theta) \leq t^2 \frac{(b-a)^2}{8}. \quad (1.35)$$

□

Then Hoeffding's inequality is given below.

Theorem 1.14 (Hoeffding's Inequality). Let $X_1, \dots, X_m$ be independent real random variables with $X_i \in [a_i, b_i]$ for all $i \in \{1, \dots, m\}$. Then, for any $\epsilon > 0$, the following inequalities hold for $S_m := \sum_{i=1}^m X_i$:

$$\mathbb{P}(S_m - \mathbb{E}[S_m] \geq \epsilon) \leq e^{-2\epsilon^2 / \sum_{i=1}^m (b_i - a_i)^2}, \quad (1.36)$$

$$\mathbb{P}(S_m - \mathbb{E}[S_m] \leq -\epsilon) \leq e^{-2\epsilon^2 / \sum_{i=1}^m (b_i - a_i)^2}. \quad (1.37)$$

Proof. Let us write

$$\mathbb{P}[S_m - \mathbb{E}[S_m] \geq \epsilon] \leq e^{-t\epsilon} \mathbb{E}[e^{t(S_m - \mathbb{E}[S_m])}] \quad (1.38)$$

$$= e^{-t\epsilon} \prod_{i=1}^m \mathbb{E}[e^{t(X_i - \mathbb{E}[X_i])}] \quad (1.39)$$

$$\leq e^{-t\epsilon} \prod_{i=1}^m e^{\frac{t^2(b_i-a_i)^2}{8}} \quad (1.40)$$

$$= e^{-t\epsilon} e^{t^2 \sum_{i=1}^m \frac{(b_i-a_i)^2}{8}} \quad (1.41)$$

$$\leq e^{-2\epsilon^2 \frac{1}{\sum_{i=1}^m (b_i-a_i)^2}}, \quad (1.42)$$

where in the first line we have used Markov's inequality, in the second line we used the independence of each of the X_i 's, in the third line we use Lemma 1.13, and in the fifth line we chose $t = 4\epsilon / \sum_{i=1}^m (b_i - a_i)^2$ to minimize the upper bound. The second statement follows similarly. $\square$

1.4.2.4 McDiarmid's inequality

Finally, we present a concentration inequality that is more general than Hoeffding's inequality. The proof makes use of a Hoeffding's inequality for martingale differences.

Definition 1.15 (Martingale difference). A sequence of random variables $V_1, V_2, \dots$ is a martingale difference sequence with respect to $X_1, X_2, \dots$ if for all $i > 0$, V_i is a function of $X_1, \dots, X_i$ and

$$\mathbb{E}[V_{i+1} | X_1, \dots, X_i] = 0. \quad (1.43)$$

Example 1.16. If $X_1, X_2, \dots$ is a martingale such that $\mathbb{E}[X_{i+1} | X_1, \dots, X_i] = X_i$, then the sequence defined by $V_i = X_i - X_{i-1}$ is a martingale difference sequence with respect to $X_1, X_2, \dots$. Hence, $\mathbb{E}[V_{i+1} | X_1, \dots, X_i] = \mathbb{E}[X_{i+1} - X_i | X_1, \dots, X_i] = \mathbb{E}[X_{i+1} | X_1, \dots, X_i] - X_i = 0$.

The following result is similar to Hoeffding's lemma.

Lemma 1.17. Let V and Z be random variables that satisfy i) $\mathbb{E}[V | Z] = 0$ and ii) for some function f and constant $c \geq 0$, the inequalities:

$$f(Z) \leq V \leq f(Z) + c. \quad (1.44)$$

Then, for all $t > 0$, the following upper bound holds:

$$\mathbb{E} \left[e^{tV} | Z \right] \leq e^{t^2 c^2 / 8}. \quad (1.45)$$

Proof. The proof follows exactly the same proof as Lemma 1.13 except it uses

conditional expectations. Note that conditioned on Z , V takes values in $[a, b]$ with $a = f(Z)$ and $b = f(Z) + c$ and its expectation vanishes. $\square$

Lemma 1.17 can be used to prove the following statement.

Theorem 1.18 (Azuma's inequality). Let $V_1, V_2, \dots$ be a martingale difference sequence with respect to the random variables $X_1, X_2, \dots$ and assume that for all $i > 0$ there is a constant $c_i \geq 0$ and a random variable Z_i , which is a function of $X_1, \dots, X_{i-1}$, satisfying the following inequalities:

$$Z_i \leq V_i \leq Z_i + c_i.$$

Then, for all $\epsilon > 0$ and $m \geq 1$, the following inequalities hold:

$$\mathbb{P} \left[\sum_{i=1}^m V_i \geq \epsilon \right] \leq \exp \left(\frac{-2\epsilon^2}{\sum_{i=1}^m c_i^2} \right), \quad (1.46)$$

$$\mathbb{P} \left[\sum_{i=1}^m V_i \leq -\epsilon \right] \leq \exp \left(\frac{-2\epsilon^2}{\sum_{i=1}^m c_i^2} \right). \quad (1.47)$$

Proof. Remember Chernoff's bound (which followed from applying Markov's inequality to the variable e^{tX}):

$$\mathbb{P}(X \geq a) = P(e^{tX} \geq e^{ta}) \leq \frac{\mathbb{E}[e^{tX}]}{e^{ta}}. \quad (1.48)$$

For $1 \leq k \leq m$, let $S_k = \sum_{i=1}^k V_i$. Then, using Chernoff's bounding technique, for any $t > 0$, we can write:

$$\mathbb{P}[S_m \geq \epsilon] \leq e^{-t\epsilon} \mathbb{E}[e^{tS_m}] = e^{-t\epsilon} \mathbb{E}[e^{tS_{m-1}} e^{tV_m}] \quad (1.49)$$

$$= e^{-t\epsilon} \mathbb{E} [\mathbb{E}[e^{tS_{m-1}} e^{tV_m} | X_1, \dots, X_{m-1}]] \quad (1.50)$$

$$= e^{-t\epsilon} \mathbb{E} [e^{tS_{m-1}} \mathbb{E}[e^{tV_m} | X_1, \dots, X_{m-1}]] \quad (1.51)$$

$$\leq e^{-t\epsilon} \mathbb{E}[e^{tS_{m-1}}] e^{t^2 c_m^2 / 8} \quad (1.52)$$

$$\leq e^{-t\epsilon} e^{t^2 \sum_{i=1}^m c_i^2 / 8} \quad (1.53)$$

$$= e^{-2\epsilon^2 / \sum_{i=1}^m c_i^2}, \quad (1.54)$$

where in the second line we have used the tower property of conditional expectations, in the third line we have used that $S_{m-1} = \sum_{i=1}^{m-1} V_i$ and V_{m-1} is a function of $X_1, \dots, X_{m-1}$ by definition of the martingale difference sequence, in the fourth line we have used Lemma 1.17, in the fifth line we have re-iterated the same argument as we used in the first line, and in the final line we have chosen $t = 4\epsilon / \sum_{i=1}^m c_i^2$ to

minimize the upper bound. This proves the first statement of the theorem. One can prove the second statement in the same way. $\square$

We can then derive McDiarmid's inequality (it generalizes Hoeffding in the sense that Hoeffding is for a sum of random variables, but McDiarmid's is for a function of random variables).

Theorem 1.19 (McDiarmid's inequality). Let $(X_1, \dots, X_m) \in \mathcal{X}^m$ be a set of $m \geq 1$ independent random variables and assume that there exist $c_1, \dots, c_m > 0$ such that $f : \mathcal{X}^m \rightarrow \mathbb{R}$ satisfies the following conditions:

$$|f(x_1, \dots, x_i, \dots, x_m) - f(x_1, \dots, x'_i, \dots, x_m)| \leq c_i, \quad (1.55)$$

for all $1 \leq i \leq m$ and any points $x_1, \dots, x_m, x'_i \in \mathcal{X}$. Let $f(S)$ denote $f(X_1, \dots, X_m)$. Then, for all $\epsilon > 0$, the following inequalities hold:

$$\mathbb{P}[f(S) - \mathbb{E}[f(S)] \geq \epsilon] \leq \exp\left(\frac{-2\epsilon^2}{\sum_{i=1}^m c_i^2}\right), \quad (1.56)$$

$$\mathbb{P}[f(S) - \mathbb{E}[f(S)] \leq -\epsilon] \leq \exp\left(\frac{-2\epsilon^2}{\sum_{i=1}^m c_i^2}\right). \quad (1.57)$$

Proof. Let $V = f(S) - \mathbb{E}[f(S)]$, and define a sequence of random variables V_k , for $1 \leq k \leq m$, as follows:

$$\begin{aligned} V_1 &= \mathbb{E}[V|X_1] - \mathbb{E}[V], \\ V_2 &= \mathbb{E}[V|X_1, X_2] - \mathbb{E}[V|X_1], \\ &\vdots \\ V_k &= \mathbb{E}[V|X_1, \dots, X_k] - \mathbb{E}[V|X_1, \dots, X_{k-1}]. \end{aligned}$$

Note that $V = \sum_{k=1}^m V_k$. Furthermore, the random variable $\mathbb{E}[V|X_1, \dots, X_k]$ is a function of $X_1, \dots, X_k$. Conditioning on $X_1, \dots, X_{k-1}$ and taking its expectation is therefore:

$$\mathbb{E}[\mathbb{E}[V|X_1, \dots, X_k]|X_1, \dots, X_{k-1}] = \mathbb{E}[V|X_1, \dots, X_{k-1}],$$

which implies that $\mathbb{E}[V_k|X_1, \dots, X_{k-1}] = 0$. Thus, the sequence $(V_k)_{1 \leq k \leq m}$ is a martingale difference sequence. Next, we observe that, since $\mathbb{E}[f(S)]$ is a scalar, V_k can be expressed as follows:

$$V_k = \mathbb{E}[f(S)|X_1, \dots, X_k] - \mathbb{E}[f(S)|X_1, \dots, X_{k-1}].$$

Thus, we can define an upper bound W_k and lower bound U_k for V_k by

$$W_k = \sup_x \{ \mathbb{E}[f(S)|X_1, \dots, X_{k-1}, x] - \mathbb{E}[f(S)|X_1, \dots, X_{k-1}] \},$$

$$U_k = \inf_x \{ \mathbb{E}[f(S)|X_1, \dots, X_{k-1}, x] - \mathbb{E}[f(S)|X_1, \dots, X_{k-1}] \}.$$

Now, using the property in Eq. (1.55), for any $1 \leq k \leq m$, the following holds:

$$W_k - U_k = \sup_{x, x'} \{ \mathbb{E}[f(S)|X_1, \dots, X_{k-1}, x] - \mathbb{E}[f(S)|X_1, \dots, X_{k-1}, x'] \} \leq c_k, \quad (1.58)$$

thus $U_k \leq V_k \leq U_k + c_k$. In view of these inequalities, we can apply Azuma's inequality (Theorem 1.18) to $V = \sum_{k=1}^m V_k$ which yields the statements. $\square$

1.4.3 Applying concentration inequalities to the estimators

We are interested in bounding somehow the distance between $R(h)$ and $\hat{R}_S(h)$, the true and empirical risks:

$$R(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[L(h(x), y)], \quad \hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i). \quad (1.59)$$

Note that $S = \{(x_i, y_i)\}_{i=1}^n$ is the training sample of size n with $(x_i, y_i) \sim \mathcal{D}$ i.i.d. and $h : \mathcal{X} \rightarrow \mathcal{Y}$ is a hypothesis. We denote by D^n the distribution over n points. We will also focus on the deterministic setting with $\{x_i\}_{i=1}^n \sim D^n$ and $y_i = c(x_i)$ (see Remark 1.6). Observe that,

$$\mathbb{E}_{S \sim \mathcal{D}^n}[\hat{R}_S(h)] = R(h). \quad (1.60)$$

Furthermore, let us consider a hypothesis $h : \mathcal{X} \rightarrow \{0, 1\}$ and

$$L(h(x), y) = 1_{h(x) \neq y}. \quad (1.61)$$

We need this since, in order to apply Hoeffding's lemma, we need a bounded variable (note: any loss between 0 and 1 would work, but this is a nice way to define a binary loss). Given that $(x_1, y_1), \dots, (x_n, y_n)$ are i.i.d. we can view $L(h(x_i), y_i)$ as i.i.d. variables. Then we can directly obtain the below result from Hoeffding's inequality.

Corollary 1.20. Fix $\epsilon > 0$ and let S denote an i.i.d. sample of size n from $\mathcal{D}^n$. Then, for any hypothesis $h : \mathcal{X} \rightarrow \{0, 1\}$, the following inequalities hold,

$$\mathbb{P}_{S \sim \mathcal{D}^n}[\hat{R}_S(h) - R(h) \geq \epsilon] \leq \exp(-2n\epsilon^2), \quad (1.62)$$

$$\mathbb{P}_{S \sim \mathcal{D}^n}[\hat{R}_S(h) - R(h) \leq -\epsilon] \leq \exp(-2n\epsilon^2). \quad (1.63)$$

By the union bound this implies,

$$\mathbb{P}_{S \sim \mathcal{D}^n}[|\hat{R}_S(h) - R(h)| \geq \epsilon] \leq 2 \exp(-2n\epsilon^2). \quad (1.64)$$

We have the following result for a fixed hypothesis. Note that this result holds regardless of whether the hypothesis is consistent, that is, regardless of whether it is able to achieve zero error on the training sample.

Theorem 1.21 (generalization bound for a fixed hypothesis). Fix a hypothesis $h : \mathcal{X} \rightarrow \{0, 1\}$. Then, for any $\delta > 0$, the following inequality holds with probability at least $1 - \delta$:

$$R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log \frac{2}{\delta}}{2n}}. \quad (1.65)$$

Proof. Choose $\epsilon \geq \sqrt{\frac{\log \frac{2}{\delta}}{2n}}$. Consider,

$$\mathbb{P}_{S \sim \mathcal{D}^n}[|\hat{R}_S(h) - R(h)| \geq \epsilon] \leq 2 \exp(-2n\epsilon^2) \leq \delta.$$

□

We can generalize the above for all hypotheses $h \in \mathcal{H}$, assuming the hypothesis class is *finite*.

Theorem 1.22 (generalization bound for a finite hypothesis class). Let $\mathcal{H}$ be a finite hypothesis set. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds for any $h \in \mathcal{H}$,

$$R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log |\mathcal{H}| + \log \frac{2}{\delta}}{2n}}. \quad (1.66)$$

Proof. Let $h_1, \dots, h_{|\mathcal{H}|}$ be the elements of $\mathcal{H}$. We are interested in the probability that at least one of the events $|\hat{R}_S(h) - R(h)| \geq \epsilon$ happens. Using the union bound and applying the result in Eq. (1.64) to each hypothesis yields,

$$\mathbb{P}(\exists h \in \mathcal{H}, |\hat{R}_S(h) - R(h)| \geq \epsilon) \quad (1.67)$$

$$= \mathbb{P}\left(|\hat{R}_S(h_1) - R(h_1)| \geq \epsilon \cup \dots \cup |\hat{R}_S(h_{|\mathcal{H}|}) - R(h_{|\mathcal{H}|})| \geq \epsilon\right) \quad (1.68)$$

$$\leq \sum_{h \in \mathcal{H}} \mathbb{P}\left(|\hat{R}_S(h) - R(h)| \geq \epsilon\right) \quad (1.69)$$

$$\leq 2|\mathcal{H}| \exp(-2n\epsilon^2). \quad (1.70)$$

□

From these results we can remark the following:

- A larger sample size n guarantees better generalization.
- The bound increases with $|\mathcal{H}|$, but only logarithmically.
- The bound suggests seeking a trade-off between reducing the empirical error versus controlling the size of the hypothesis set: a larger hypothesis set is penalised by the second term but could help reduce the empirical error $\hat{R}_S(h)$ that is the first term. But, for a similar empirical error, it suggests using a smaller hypothesis set. This can be viewed as an instance of the so-called Occam's Razor principle named after the theologian William of Occam: Plurality should not be posited without necessity, also rephrased as, the simplest explanation is best. In this context, it could be expressed as follows: All other things being equal, a simpler (smaller) hypothesis set is better.

Estimation error. Denote with R^* the infimum of the errors achieved by measurable functions $f : \mathcal{X} \rightarrow \mathcal{Y}$,

$$R^* = \inf_f R(f) = \inf_f \mathbb{E}_{(x,y) \sim \mathcal{D}}[L(f(x), y)].$$

This is called the Bayes error. In the deterministic case it is possible to have $R^* = 0$ but in the stochastic case $R^* \neq 0$. For a given hypothesis $h \in \mathcal{H}$, the error can be decomposed as follows,

$$R(h) - R^* = \underbrace{(R(h) - R(h^*))}_{\text{estimation}} + \underbrace{(R(h^*) - R^*)}_{\text{approximation}},$$

where $h^* = \arg \min_{h \in \mathcal{H}} R(h)$ denotes the best-in-class hypothesis (of our chosen hypothesis class). The second term is referred to as the approximation error, since it measures how well the Bayes error can be approximated using $\mathcal{H}$. It is a property of the hypothesis set $\mathcal{H}$, a measure of its richness. The approximation error is not accessible, since in general the underlying distribution $\mathcal{D}$ is not known. Even with various noise assumptions, estimating the approximation error is difficult. The first term is the estimation error, and it depends on the hypothesis h selected. It measures the quality of the hypothesis h with respect to the best-in-class hypothesis.

Let $\hat{h} = \arg \min_{h \in \mathcal{H}} \hat{R}_S(h)$ denote the hypothesis returned by the *empirical risk minimization algorithm*, that is the algorithm that returns a hypothesis with the smallest empirical error. The estimation error of the algorithm can then be bounded in terms of the generalization error:

$$\begin{aligned} R(\hat{h}) - R(h^*) &= (R(\hat{h}) - \hat{R}_S(\hat{h})) + (\hat{R}_S(\hat{h}) - R(h^*)) \\ &\leq (R(\hat{h}) - \hat{R}_S(\hat{h})) + (\hat{R}_S(h^*) - R(h^*)) \\ &\leq 2 \sup_{h \in \mathcal{H}} |R(h) - \hat{R}_S(h)|, \end{aligned}$$

where we used $\hat{R}_S(\hat{h}) \leq \hat{R}_S(h^*)$. One can then use the bound in Theorem 1.22.

1.5 Rademacher complexity

We will continue to use $\mathcal{H}$ to denote a hypothesis set as in the previous sections and h an element of $\mathcal{H}$. We will let G denote the family of loss functions associated to $\mathcal{H}$, i.e., functions that map $Z = \mathcal{X} \times \mathcal{Y}$ to $\mathbb{R}_+$, defined by:

$$G = \{g : (x, y) \mapsto L(h(x), y) \mid h \in \mathcal{H}\}, \quad (1.71)$$

The Rademacher complexity captures the richness of a family of functions by measuring the degree to which a hypothesis set can fit random noise. The following states the formal definitions of the empirical and average Rademacher complexity.

Definition 1.23 (Empirical Rademacher complexity). Let G be a family of functions mapping from Z to $[a, b]$ and $S = (z_1, \dots, z_m)$ a fixed sample of size m of elements in Z . Then, the empirical Rademacher complexity of G with respect to the sample S is defined as,

$$\hat{\mathcal{R}}_S(G) = \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m \sigma_i g(z_i) \right], \quad (1.72)$$

where $\boldsymbol{\sigma} = (\sigma_1, \dots, \sigma_m)^\top$, with σ_i 's independent uniform random variables taking values in $\{+1, -1\}$. The random variables σ_i are called Rademacher variables.

Let $\mathbf{g}_S$ denote the vector of values taken by the function $g \in G$ over the sample S : $\mathbf{g}_S = (g(z_1), \dots, g(z_m))^\top$. Then, the empirical Rademacher complexity can be rewritten as,

$$\hat{\mathcal{R}}_S(G) = \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{g \in G} \frac{\boldsymbol{\sigma} \cdot \mathbf{g}_S}{m} \right]. \quad (1.73)$$

The inner product $\sigma \cdot \mathbf{g}_S$ measures the correlation of $\mathbf{g}_S$ with the vector of random noise σ . The supremum is a measure of how well the function class G correlates with σ over the sample S . Thus, the empirical Rademacher complexity measures on average how well the function class G correlates with random noise on S . This describes the richness of the family G : richer or more complex families G can generate more vectors $\mathbf{g}_S$ and thus better correlate with random noise, on average.

Intuition. Assume that G is a family of functions mapping Z to $[-1, 1]$. Sample σ_i , $i = 1, \dots, m$. Find the $g \in G$ that “best fits” the randomly sampled noise, i.e. the $g \in G$ for which $\frac{1}{m} \sum_{i=1}^m \sigma_i g(z_i)$ is highest. If some g fits the noise perfectly, $\frac{1}{m} \sum_{i=1}^m \sigma_i g(z_i) \approx 1$. If you repeat this for many samples and take the average and it is close to 1, then the function class has a high complexity. If it is close to zero, then G does not fit the data too well.

Definition 1.24 (Rademacher complexity). Let $\mathcal{D}$ denote the distribution according to which samples are drawn. For any integer $m \geq 1$, the Rademacher complexity of G is the expectation of the empirical Rademacher complexity over all samples of size m drawn according to $\mathcal{D}$:

$$\mathcal{R}_m(G) = \mathbb{E}_{S \sim \mathcal{D}^m} [\hat{\mathcal{R}}_S(G)]. \quad (1.74)$$

The Rademacher complexity can be used to obtain generalization bounds.

Theorem 1.25. Let G be a family of functions mapping Z to $[0, 1]$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, each of the following holds for all $g \in G$:

$$\mathbb{E}[g(z)] \leq \frac{1}{m} \sum_{i=1}^m g(z_i) + 2\mathcal{R}_m(G) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}, \quad (1.75)$$

and

$$\mathbb{E}[g(z)] \leq \frac{1}{m} \sum_{i=1}^m g(z_i) + 2\hat{\mathcal{R}}_S(G) + 3\sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \quad (1.76)$$

Proof. For any sample $S = (z_1, \dots, z_m)$ and for any $g \in G$, we denote with $\hat{\mathbb{E}}_S[g]$ the empirical average of y over S : $\hat{\mathbb{E}}_S[g] = \frac{1}{m} \sum_{i=1}^m g(z_i)$. The proof consists of applying McDiarmid’s inequality to the function Φ defined over sample S by

$$\Phi(S) = \sup_{g \in G} \mathbb{E}[g] - \hat{\mathbb{E}}_S[g].$$

Let S and S' be two samples differing in exactly one point, say z_m in S and z'_m in S' . Then, since the difference of suprema does not exceed the supremum of the difference we have,

$$\Phi(S') - \Phi(S) \leq \sup_{g \in G} \left(\hat{\mathbb{E}}_S[g] - \hat{\mathbb{E}}_{S'}[g] \right) = \sup_{g \in G} \frac{g(z_m) - g(z'_m)}{m} \leq \frac{1}{m}. \quad (1.77)$$

Similarly, $\Phi(S) - \Phi(S') \leq \frac{1}{m}$, thus $|\Phi(S') - \Phi(S)| \leq \frac{1}{m}$. Then, by McDiarmid's inequality, for any $\delta > 0$, with probability at least $1 - \delta/2$, the following holds.

$$\Phi(S) \leq \mathbb{E}_S[\Phi(S)] + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \quad (1.78)$$

Further,

$$\mathbb{E}_S[\Phi(S)] = \mathbb{E}_S \left[\sup_{g \in G} \mathbb{E}[g] - \hat{\mathbb{E}}_S[g] \right] \quad (1.79)$$

$$= \mathbb{E}_S \left[\sup_{g \in G} \mathbb{E}_{S'} \left[\hat{\mathbb{E}}_{S'}[g] - \hat{\mathbb{E}}_S[g] \right] \right] \quad (1.80)$$

$$\leq \mathbb{E}_{S, S'} \left[\sup_{g \in G} \hat{\mathbb{E}}_{S'}[g] - \hat{\mathbb{E}}_S[g] \right] \quad (1.81)$$

$$= \mathbb{E}_{S, S'} \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m (g(z'_i) - g(z_i)) \right] \quad (1.82)$$

$$= \mathbb{E}_{\sigma, S, S'} \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m \sigma_i (g(z'_i) - g(z_i)) \right] \quad (1.83)$$

$$\leq \mathbb{E}_{\sigma, S'} \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m \sigma_i g(z'_i) \right] + \mathbb{E}_{\sigma, S} \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m -\sigma_i g(z_i) \right] \quad (1.84)$$

$$= 2\mathbb{E}_{\sigma, S} \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m \sigma_i g(z_i) \right] = 2\mathcal{R}_m(G), \quad (1.85)$$

where in the second line we use that S' are sampled in an i.i.d. fashion and thus $\mathbb{E}[g] = \mathbb{E}_{S'}[\hat{\mathbb{E}}_{S'}[g]]$, in the third line we use the Jensen's inequality and the convexity of supremum functions. In the fifth line we introduce the Rademacher variables σ_i 's, that is uniformly distributed independent random variables taking values in $\{+1, -1\}$; this does not change the expectation: when $\sigma_i = 1$, the associated summand remains unchanged; when $\sigma_i = -1$, the associated summand flips signs, which is equivalent to swapping z_i and z'_i between S and S' . Since we are taking the expectation over all possible S, S' this swap does not change the overall expectation. In the sixth line we use the sub-additivity of the supremum function, $\sup(U+V) \leq \sup(U) + \sup(V)$. Finally, in the last line we use the definition of Rademacher

complexity and the fact that σ_i and $-\sigma_i$ are distributed in the same way. The first statement follows. To derive the second statement, changing one point in S changes $\hat{\mathcal{R}}_S(G)$ by at most $1/m$, so $|\hat{\mathcal{R}}_S(G) - \hat{\mathcal{R}}_{S'}(G)| \leq \frac{1}{m}$. Then, using again McDiarmid's inequality, with probability $1 - \delta/2$ the following holds:

$$\mathcal{R}_m(G) \leq \hat{\mathcal{R}}_S(G) + \sqrt{\frac{\log \frac{2}{\delta}}{2m}}. \quad (1.86)$$

The second statement then follows. $\square$

In certain cases, it is possible to compute or bound the Rademacher complexity.

Example 1.26 (Linear predictor with ℓ_2 regularisation). Let $S = \{x_1, \dots, x_m\}$ be a set of m points in $\mathbb{R}^d$. Let $G = \{x \mapsto w^\top x \mid w \in \mathbb{R}^d, \|w\|_2 \leq \Lambda\}$ be the set of linear predictors with bounded norm. Then, the empirical Rademacher complexity of G with respect to S can be bounded as follows:

$$\hat{\mathcal{R}}_S(G) = \mathbb{E}_\sigma \left[\sup_{g \in G} \frac{1}{m} \sum_{i=1}^m \sigma_i g(x_i) \right] \quad (1.87)$$

$$= \mathbb{E}_\sigma \left[\sup_{\|w\|_2 \leq \Lambda} \frac{1}{m} \sum_{i=1}^m \sigma_i w^\top x_i \right] \quad (1.88)$$

$$= \frac{1}{m} \mathbb{E}_\sigma \left[\sup_{\|w\|_2 \leq \Lambda} w^\top \sum_{i=1}^m \sigma_i x_i \right]. \quad (1.89)$$

By Cauchy-Schwarz inequality, we have

$$\hat{\mathcal{R}}_S(G) \leq \frac{1}{m} \mathbb{E}_\sigma \left[\sup_{\|w\|_2 \leq \Lambda} \|w\|_2 \left\| \sum_{i=1}^m \sigma_i x_i \right\|_2 \right] \quad (1.90)$$

$$\leq \frac{\Lambda}{m} \mathbb{E}_\sigma \left[\left\langle \sum_{i=1}^m \sigma_i x_i, \sum_{i=1}^m \sigma_i x_i \right\rangle^{1/2} \right] \quad (1.91)$$

$$\leq \frac{\Lambda}{m} \mathbb{E}_\sigma \left[\left(\sum_{i,i'=1}^m \sigma_i \sigma_{i'} x_i x_{i'} \right)^{1/2} \right] \quad (1.92)$$

$$\leq \frac{\Lambda}{m} \mathbb{E}_\sigma \left[\sum_{i,i'=1}^m \sigma_i \sigma_{i'} x_i x_{i'} \right]^{1/2}, \quad (1.93)$$

as $\mathbb{E}[\Psi(X)] \leq \mathbb{E}[X]$ for any concave function Ψ by Jensen's inequality. Moreover, we note that

$$\mathbb{E}_\sigma[\sigma_i \sigma_{i'}] = \begin{cases} \mathbb{E}_\sigma[\sigma_i^2] = 1, & \text{if } i = i', \\ 0, & \text{otherwise.} \end{cases} \quad (1.94)$$

by independence of the Rademacher variables and since $\sigma_i \in \{-1, 1\}$. Thus,

$$\hat{\mathcal{R}}_S(G) \leq \frac{\Lambda}{m} \left(\sum_{i=1}^m \|x_i\|_2^2 \right)^{1/2} \leq \frac{\Lambda}{m} \sqrt{m} \max_i \|x_i\|_2 = \frac{\Lambda}{\sqrt{m}} \max_i \|x_i\|_2. \quad (1.95)$$

1.6 Maximum likelihood estimation

In general, how do we choose good estimators? A common principle is to choose the estimator that maximises the likelihood of the observed data. This is the maximum likelihood principle. Consider a set of n examples $X = \{x_1, \dots, x_n\}$ drawn i.i.d. from some unknown distribution $p_{\text{data}}(x)$. Let $p_{\text{model}}(x; \theta)$ be a parameterized family of probability distributions over the same space indexed by θ . It maps any point x to a number approximating the probability $p_{\text{data}}(x)$ of generating x with the underlying data distribution. The maximum likelihood estimator for θ is defined as the value of θ that maximises the likelihood of the observed data X :

$$\theta_{\text{ML}} = \arg \max_{\theta} p_{\text{model}}(X; \theta) = \arg \max_{\theta} \prod_{i=1}^n p_{\text{model}}(x_i; \theta). \quad (1.96)$$

This product can be difficult to optimize (especially if some terms are very small or very large), so we use the fact that the logarithm is a monotonic function to convert the product into a sum:

$$\theta_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^n \log p_{\text{model}}(x_i; \theta). \quad (1.97)$$

We can then divide by n to obtain the average log-likelihood and express this as an expectation over the data distribution:

$$\theta_{\text{ML}} = \arg \max_{\theta} \mathbb{E}_{x \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(x; \theta), \quad (1.98)$$

where $\hat{p}_{\text{data}}$ denotes the empirical (i.e., observed) data distribution. One could also interpret this as minimising the KL divergence between the empirical data distribution and the model distribution:

$$\theta_{\text{ML}} = \arg \min_{\theta} D_{\text{KL}}(\hat{p}_{\text{data}} \| p_{\text{model}}(\cdot; \theta)) \quad (1.99)$$

$$= \arg \min_{\theta} \mathbb{E}_{x \sim \hat{p}_{\text{data}}} [\log \hat{p}_{\text{data}}(x) - \log p_{\text{model}}(x; \theta)]. \quad (1.100)$$

The first term does not depend on θ and can be ignored. The second term is the negative log-likelihood.

1.6.1 Conditional log-likelihood

One can generalize the log-likelihood to estimate conditional probability distribution of the form $p(y|x; \theta)$ to predict the output y given the observed input x . This is the basis of most *supervised learning* techniques. Let x and y represent the inputs and observed outputs, the conditional log-likelihood estimator is defined as:

$$\theta_{\text{ML}} = \arg \max_{\theta} p(y|x; \theta). \quad (1.101)$$

If the examples are i.i.d., we have

$$\theta_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^n \log p(y_i|x_i; \theta). \quad (1.102)$$

1.6.2 Linear regression and maximum likelihood

Let's recall the regression example from Example 1.7. We have a dataset of the form $\{(x_1, y_1), \dots, (x_n, y_n)\}$ where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. We assume that the output y is generated by a linear model $y = \mathbf{w}^\top x + c + \epsilon$, for some coefficients $\mathbf{w} \in \mathbb{R}^d$, $c \in \mathbb{R}$. Here $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is a Gaussian noise term with mean 0 and variance σ^2 and is independent on x and across the observations. Thus implies that the response variable Y is independent across observations given the input x , and we can decompose the likelihood (probability density of seeing the data) as:

$$\mathcal{L}(Y|x; \theta) = p(y_1, \dots, y_n|x_1, \dots, x_n; \mathbf{w}, c, \sigma^2) = \prod_{i=1}^n p(y_i|x_i; \mathbf{w}, c, \sigma^2). \quad (1.103)$$

Following Eq. (1.102), we have

$$\theta_{\text{ML}} = \arg \max_{\mathbf{w}, c, \sigma^2} \sum_{i=1}^n \log p(y_i|x_i; \mathbf{w}, c, \sigma^2), \quad (1.104)$$

where $\theta_{\text{ML}} = (\hat{\mathbf{w}}, \hat{b}, \hat{\sigma})$ is the list of parameters maximising the likelihood. Since the noise ϵ is Gaussian distributed, our output model is also Gaussian distributed with mean $\mu = \mathbf{w}^\top x + c$ and variance σ^2 . Recall that the probability density function of the Gaussian variable with mean μ and variance σ^2 is given by

$$f(x; \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Thus, the log-likelihood of observing y_i given x_i is:

$$\log \mathcal{L}(Y|x; \theta) = \sum_{i=1}^n \log f(y_i; \mathbf{w}^\top x_i + c, \sigma^2) \quad (1.105)$$

$$= -\frac{n}{2} \log(2\pi) - n \log(\sigma) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mathbf{w}^\top x_i - c)^2 \quad (1.106)$$

$$= -\frac{n}{2} \log(2\pi) - n \log(\sigma) - \sum_{i=1}^n \frac{\|y_i - \hat{y}_i\|^2}{2\sigma^2}, \quad (1.107)$$

where $\hat{y}_i = \mathbf{w}^\top x_i + c$ is the output of the linear model. If we assume that the variance is fixed to some given constant (i.e., we do not optimize σ), then the maximum likelihood estimator is equivalent to the least squares (or MSE) estimator:

$$\operatorname{argmax}_{\mathbf{w}, c} \log \mathcal{L}(Y|x; \mathbf{w}, c) = \operatorname{argmin}_{\mathbf{w}, c} \frac{1}{n} \sum_{i=1}^n \|y_i - \mathbf{w}^\top x_i - c\|^2. \quad (1.108)$$

1.7 When training the model

1.7.1 Regularisation

Remember that we informally discussed about the capacity of a model and mentioned that the capacity was directly related to the choice of the hypothesis class, i.e. the class of functions the learning algorithm can choose from. However, we also briefly mentioned that the learning algorithm itself can impact the capacity. Relatedly, the loss function can be used to give the algorithm a preference for functions with certain properties. When adding a certain term to the loss function, we refer to its impact as regularisation. For example, consider a setting where we fit some model with parameters θ with a mean squared error. In addition we add a criterion that expresses a preference for weights to have smaller squared L^2 norm (in this notation I suppress the dependence of the loss function on the train data and only make explicit the dependence on the parameters of the model):

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \|h_\theta(x_i) - y_i\|_2^2 + \lambda \theta^\top \theta,$$

where λ is a value chosen ahead of time that controls the strength of our preference for smaller parameter values. Minimising this loss function thus results in a choice of weights that make a tradeoff between fitting the training data and being small. More generally, any kind of regulariser can be added to the model with a general form of $g(\theta)$. We will discuss more regularisation methods used in deep learning when we discuss neural networks later on. An alternative choice of the regularisation is to use the L^1 norm of the weights: $g(\theta) = \|\theta\|_1$. This will result in a more sparse solution.

1.7.1.1 A Bayesian perspective on regularization

Let's first recap the Bayesian perspective. What we have done before is take a frequentist approach: we estimated a single value of the parameters θ based on minimising the empirical risk (some loss function over samples of our data). The Bayesian perspective represents the parameters θ as a random variable and computes a distribution over it. Before observing the data, we represent the knowledge about θ using some prior probability distribution $p(\theta)$. One example of a prior when we don't really know anything about the parameters is to use a uniform distribution over some range of possible parameter values. Now consider a set of datapoints $\{x_i\}_{i=1}^n$. We can recover the effect of data on our belief about θ by conditioning the data likelihood $p(x_1, \dots, x_n|\theta)$ with the prior via Bayes' rule.

$$p(\theta|x_1, \dots, x_n) = \frac{p(x_1, \dots, x_n|\theta)p(\theta)}{p(x_1, \dots, x_n)}. \quad (1.109)$$

To regularise we thus added something to the loss function to 'reduce' complexity. The Bayesian interpretation of this is that we impose a prior on the parameters and don't want to deviate too much from this prior. Let's denote our data with $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$. We typically assume our data samples are i.i.d. so that $p(S|\theta) = \prod_{i=1}^n p(x_i)p_\theta(y_i|x_i)$, where $p_\theta(y|x)$ can be for example Gaussian $\mathcal{N}(h_\theta(x), \Sigma_\theta(x))$. Observe that $\log p_\theta(y|x) = -\frac{1}{2}\|f_\theta(x) - y\|_2^2 + C$ if $\Sigma_\theta(x) = I$. Thus, maximising the log-posterior distribution is the same as minimising the mean squared error loss. Then,

$$\log p(\theta|S) \propto \log p(S|\theta) + \log p(\theta) \quad (1.110)$$

$$= -\frac{1}{2}\|h_\theta(x) - y\|_2^2 + C + \log p(\theta). \quad (1.111)$$

Suppose we choose a standard Gaussian prior: $p(\theta) = \mathcal{N}(0, \sigma^2 I)$. Then,

$$\log p(\theta) = \frac{1}{2\sigma^2} \sum_{i=1}^d \theta_i^2 - \log \sigma - \frac{1}{2} \log 2\pi \quad (1.112)$$

$$= -\lambda \|\theta\|_2^2 + C, \quad (1.113)$$

with $\lambda = \frac{1}{2\sigma^2}$. Thus, as our loss function we get back:

$$L = -\log p(\theta|S) \propto \frac{1}{2}\|h_\theta(x) - y\|_2^2 + \lambda \|\theta\|_2^2. \quad (1.114)$$

1.7.2 Hyperparameters and validation sets

Most machine learning algorithms have hyperparameters, settings that we can use to control the algorithm's behavior. The values of hyperparameters are not

adapted by the learning algorithm itself (though we can design a nested learning procedure in which one learning algorithm learns the best hyperparameters for another learning algorithm). In the case of fitting a polynomial to the data, the hyperparameter can be the degree of the polynomial. In neural networks the hyperparameter choices get even more involved: we have to choose the number of layers, the width of the layers, the structure of the layers, and so on. Sometimes a setting is chosen to be a hyperparameter that the learning algorithm does not learn because the setting is difficult to optimize. More frequently, the setting must be a hyperparameter because it is not appropriate to learn that hyperparameter on the training set. This applies to all hyperparameters that control model capacity. If learned on the training set, such hyperparameters would always choose the maximum possible model capacity resulting in overfitting. To solve this problem it is also typical to save part of your data as a validation set. In particular, given a bunch of data you would split it into a train set, a validation set and a test set. The train dataset can be used to learn the parameters of the model and the validation set is used to set the hyperparameters. The test set should not be shown to the learning algorithm or be used to make any model choices. In practice, when the same test set has been used repeatedly to evaluate performance of different algorithms over many years, and especially if we consider all the attempts from the scientific community at beating the reported state-of-the-art performance on that test set, we end up having optimistic evaluations with the test set as well. Benchmarks can thus become stale and then do not reflect the true field performance of a trained system. Thankfully, the community tends to move on to new (and usually more ambitious and larger) benchmark datasets.

Cross validation. Dividing the dataset into a fixed training and fixed test set can be challenging if the dataset itself is quite small. In this case one can use cross validation. We would repeat the training and testing computations on different randomly chosen subsets or splits of the original dataset. k -fold cross validation is performed as follows:

- Split the dataset S into k mutually exclusive subsets S_j , whose union is $\cup_{j=1}^k S_j = S$
- For each j from 1 to k
 - Train a model h_j on $S \setminus S_j$
 - Test the model on S_j by computing the loss function: $e_j = L(h_j(x), y)$ for $x, y \in S_j$

- Return $e_1, \dots, e_k$.

One can then produce an estimator by returning the average of $e_1, \dots, e_k$.

1.7.3 Challenges in machine learning

Curse of dimensionality. You may have heard before about the curse of dimensionality (for example, it is also a problem often mentioned when discussing methods for solving partial differential equations). What is it exactly? Let's consider a simple example. You are going on a trip and you want to find a suitable bag to take with you and you go to the store to look for options. After speaking with the sales manager, you realise that the best choice of bag will depend on whether you go on an active holiday or on a holiday where you will be relaxing by the beach most of the time. There thus seem to be two factors that influence your choice. After talking to the sales manager a bit more you realise that other factors play a role too: within both the categories are subcategories of light/heavy. We thus went from 2 options, to $2 \times 2 = 4$; if in each of the subcategories there are another 5 options we get $2 \times 2 \times 5 = 20$. Quickly, the number of options increases as we add more factors (or dimensions) into the decision, you start adding lots of possible combinations and the number of possible configurations quickly increases. In general, the dimension is a coordinate/factor/variable you need to describe the state of your system. Initially in our example we just had one coordinate: active or relaxing. Each time we added a new category, this became an extra dimension to consider. The dimensions then increase the number of possible configurations. See Fig. 1.4.

Statistically, one challenge posed by the curse of dimensionality is that we may have very many more possible configurations than training samples. Suppose our input space is split into grids. Suppose we have a model that has assigned to some of the gridpoints that were part of the training dataset a label. When we receive a new datapoint we would inspect the training dataset and see what the label was in the grid close to the new datapoint's grid values. In high-dimensional settings however, the number of configurations is huge and the probability that our new datapoint is in any way 'close' to the training dataset becomes smaller. In machine learning, and specifically deep learning, a debate is going on about whether the models extrapolate or interpolate. The recent work of Yann LeCun and colleagues [Balestriero et al., 2021] explores this in more detail. They use the following definitions. Interpolation occurs for a sample x whenever this sample falls inside or on the boundary of the given dataset (the train set) its convex hull.



Figure 1.4: As the number of relevant dimensions of the data increases (from left to right), the number of configurations of interest may grow exponentially. (Left) In this one-dimensional example, we have one variable for which we only care to distinguish 10 regions of interest. With enough examples falling within each of these regions (each region corresponds to a cell in the illustration), learning algorithms can easily generalize correctly. A straightforward way to generalize is to estimate the value of the target function within each region (and possibly interpolate between neighboring regions). (Center) With two dimensions, it is more difficult to distinguish 10 different values of each variable. We need to keep track of up to $10 \times 10 = 100$ regions, and we need at least that many examples to cover all those regions. (Right) With three dimensions, this grows to $10^3 = 1,000$ regions and at least that many examples. For d dimensions and U values to be distinguished along each axis, we seem to need $O(U^d)$ regions and examples. This is an instance of the curse of dimensionality. Figure provided by Nicolas Chapados and corresponding to [Goodfellow et al., 2016, Fig. 5.9].

Extrapolation occurs when x falls outside of the convex hull. This work argues that on any high-dimensional dataset (with over 100 dimensions), interpolation almost surely never happens. In other words, given the realistic amount of data that can be stored and used by current computational capacities, it is extremely unlikely that a newly observed sample lies in the convex hull of that dataset. The work relies on the following result from [Bárány and Füredi, 1988].

Theorem 1.27 (Bárány and Füredi, 1988). Given a d -dimensional dataset $X = \{x_1, \dots, x_n\}$ with i.i.d. samples uniformly drawn from an hyperball, the probability that a new sample x is in interpolation regime has the following asymptotic behavior:

$$\lim_{d \rightarrow \infty} p(x \in \text{Hull}(X)) = \begin{cases} 1, & \text{if } n > d^{-1}2^{d/2}, \\ 0, & \text{if } n < d^{-1}2^{d/2}. \end{cases} \quad (1.115)$$

In other words: as the dimension d grows, to be in the interpolation regime, i.e. $x \in \text{Hull}(X)$, requires more and more data samples.

Robustness. We already discussed the idea of generalization, i.e. that the train and test errors should be close to each other. One way of achieving generalization was regularisation. The idea of regularisation is in essence guided by our belief of what kind of function should be learned, e.g. one with small parameter values. Among the most widely used priors is the smoothness prior. This prior states that the function we learn should not change too much within a small area:

$$h^*(x) \approx h^*(x + \epsilon), \quad (1.116)$$

for an input x and a perturbation ϵ . We will see later on that deep neural networks are vulnerable to so-called adversarial examples: these are perturbations of datapoints on which the model completely changes its prediction.

The underlying data manifold. A manifold is a connected region. More formally, it is a set of points associated with a neighborhood around each point. From any given point, the manifold locally appears to be a Euclidean space. The concept of the neighborhood allows for transformations that move you from one position to a neighboring one.

Why is this related to machine learning? Remember we mentioned that learning in very high dimensions would become very challenging (due to never being able to find the required amount of data). Therefore, a common hypothesis in machine learning is that real-world high dimensional data (such as images) lie on low-dimensional manifolds embedded in the high-dimensional space. Thus, while the number of possible configurations may be very large, the real data lies in a subset of those configurations.

We will not go deep into manifolds here, but for extra reading this blog post was very nice: <https://bjlkeng.github.io/posts/manifolds/>.

Chapter 2: Optimisation overview

Overview. Here we will recap optimisation focusing gradient descent, and some extensions of gradient descent.

Further reading and general resources.

- If you'd like a formal introduction please refer to [Boyd and Vandenberghe, 2004, Chapt. 9] for the optimisation methods. The book is available online: https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.
- The lecture notes in <https://www.cs.cornell.edu/courses/cs4787/2020sp/lectures/Lecture2.pdf> for gradient descent and <https://www.cs.cornell.edu/courses/cs4787/2020sp/lectures/Lecture7.pdf> for momentum gradient descent.
- The notes in: http://courses.csail.mit.edu/6.854/16/Notes/n16-gradient_descent.html
- The notes in <https://mitliagkas.github.io/ift6085-2020/ift-6085-lecture-3.pdf>.
- The intuitive explanation of momentum gradient descent is based on <https://distill.pub/2017/momentum/>.
- For an overview of different accelerations of gradient descent see <https://runder.io/optimizing-gradient-descent/>
- On strong convexity: <https://xingyuzhou.org/blog/notes/strong-convexity>
- A video I like on momentum: <https://ocw.mit.edu/courses/18-065-matrix-meth-resources/lecture-23-accelerating-gradient-descent-use-momentum/>.

2.1 Definitions

A general mathematical optimisation problem is a problem of the form,

$$\min f(\mathbf{x}), \quad \text{subject to } \mathbf{x} \in \Omega, \quad (2.1)$$

where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is a real-valued objective function and $\Omega \subset \mathbb{R}^d$ is a set defining the constraints. If $\Omega = \mathbb{R}^d$, then the problem is an unconstrained optimisation

problem. Among all $\mathbf{x} \in \Omega$, we seek the one with the smallest f -value. Typically the constraint set Ω will consist of such $\mathbf{x} \in \mathbb{R}^d$ that satisfy certain equations and inequalities:

$$g_1(\mathbf{x}) \leq 0, \dots, g_m(\mathbf{x}) \leq 0, \quad f_1(\mathbf{x}) = 0, \dots, f_p(\mathbf{x}) = 0. \quad (2.2)$$

A vector $\mathbf{x}^*$ satisfying the constraints is called an optimum or minimizer of the problem if $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all other $\mathbf{x} \in \Omega$. Note that replacing f by $-f$, we could state the problem as a maximization problem.

2.1.1 Optimality conditions

Let's focus on the unconstrained problem for now, i.e. $\Omega = \mathbb{R}^d$. We would like to define some optimality conditions that identify properties that potential minimizers need to satisfy in relation to $f(\mathbf{x})$. Let's first define a local and global optimum.

Definition 2.1 (Local and global optimum). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a function. A point $\mathbf{x}^* \in \mathbb{R}^d$ is a local minimum of f if there exists an $\epsilon > 0$ such that for all $\mathbf{x} \in \mathbb{R}^d$ satisfying $\|\mathbf{x} - \mathbf{x}^*\| < \epsilon$,

$$f(\mathbf{x}^*) \leq f(\mathbf{x}). \quad (2.3)$$

A point $\mathbf{x}^* \in \mathbb{R}^d$ is a strict minimizer of f if there exists an $\epsilon > 0$ such that for all $\mathbf{x} \neq \mathbf{x}^* \in \mathbb{R}^d$ satisfying $\|\mathbf{x} - \mathbf{x}^*\| < \epsilon$,

$$f(\mathbf{x}^*) < f(\mathbf{x}). \quad (2.4)$$

A point $\mathbf{x}^* \in \mathbb{R}^d$ is a global minimum of f if,

$$f(\mathbf{x}^*) \leq f(\mathbf{x}), \quad \forall \mathbf{x} \in \mathbb{R}^d. \quad (2.5)$$

Denote with $C^k(U)$ the set of functions that are k times continuously differentiable on some set U . Denote with $\nabla f(\mathbf{x})$ the gradient of f at $\mathbf{x}$, i.e.

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{x}), \dots, \frac{\partial f}{\partial x_d}(\mathbf{x}) \right)^\top. \quad (2.6)$$

We have the following first-order necessary condition for optimality.

Theorem 2.2. Let $\mathbf{x}^*$ be a local minimizer of f and assume that $f \in C^1(U)$ for a neighborhood U of $\mathbf{x}^*$. Then, $\nabla f(\mathbf{x}^*) = 0$.

This however is not a sufficient condition: maxima and saddle points will also have a vanishing gradient. If we have access to second-order information, in the form of the Hessian, then we can say more. The Hessian of f at $\mathbf{x}$ is given by

$$\nabla^2 f(\mathbf{x}) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j} \right)_{1 \leq i, j \leq d}. \quad (2.7)$$

Recall that a matrix is positive semidefinite, written $A \succeq 0$ if for every $\mathbf{x} \in \mathbb{R}^d$ $\mathbf{x}^T A \mathbf{x} \geq 0$ and positive definite, written $A \succ 0$ if $\mathbf{x}^T A \mathbf{x} > 0$. Then we have the following condition.

Theorem 2.3. Let $f \in C^2(U)$ for some open set U and $\mathbf{x}^* \in U$. If $\mathbf{x}^*$ is a local minimizer, then $\nabla f(\mathbf{x}^*) = 0$ and $\nabla^2 f(\mathbf{x}^*)$ is positive semidefinite. Conversely, if $\nabla f(\mathbf{x}^*) = 0$ and $\nabla^2 f(\mathbf{x}^*)$ is positive definite, then $\mathbf{x}^*$ is a strict local minimizer.

Unfortunately, the above criteria can only be used to identify local minimisers; differentiability is a local property so it only gives us local information.

2.1.2 Convexity

A particular type of function we like to work with is a convex one. For convex functions, local optimality implies global optimality.

Definition 2.4 (Convexity). Let $S \subset \mathbb{R}^d$. A function $f : S \rightarrow \mathbb{R}$ is called convex if S is convex and for all $\mathbf{x}, \mathbf{y} \in S$ and for $\lambda \in [0, 1]$,

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \quad (2.8)$$

Intuition: take two points on the function; the graph of the function connecting the two points lies below the line connecting those points. Note, $S = \mathbb{R}$ is a convex set.

The nice property of convex functions is the following.

Theorem 2.5 (Global minimizer of convex functions). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a convex function. Then any local minimizer of f is a global minimizer.

Proof. Let $\mathbf{x}^*$ be a local minimizer and assume that it is not a global minimizer. Then there exists a vector $\mathbf{y} \in \mathbb{R}^d$ such that $f(\mathbf{y}) < f(\mathbf{x}^*)$. Since f is convex, for any $\lambda \in [0, 1]$ and $\mathbf{x} = \lambda\mathbf{y} + (1 - \lambda)\mathbf{x}^*$ we have,

$$f(\mathbf{x}) \leq \lambda f(\mathbf{y}) + (1 - \lambda)f(\mathbf{x}^*) < \lambda f(\mathbf{x}^*) + (1 - \lambda)f(\mathbf{x}^*) = f(\mathbf{x}^*). \quad (2.9)$$

This holds for all $\mathbf{x}$ on the line segment connecting $\mathbf{y}$ and $\mathbf{x}^*$. Since every open neighborhood U of $\mathbf{x}^*$ contains a bit of this line segment, this means that every open neighborhood U of $\mathbf{x}^*$ contains an $\mathbf{x} \neq \mathbf{x}^*$ such that $f(\mathbf{x}) < f(\mathbf{x}^*)$. But this contradicts $\mathbf{x}^*$ being a local minimizer. Thus, $\mathbf{x}^*$ must be a global minimizer. $\square$

There are a variety of equivalent conditions that work if the function f is differentiable everywhere. For example, convexity is equivalent to the following property:

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}), \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^d. \quad (2.10)$$

If the function is twice differentiable, then convexity on $\mathbb{R}^d$ is equivalent to (positive) semidefiniteness, see https://en.wikipedia.org/wiki/Definite_matrix:

$$\nabla^2 f(\mathbf{x}) \succeq 0, \quad \forall \mathbf{x} \in \mathbb{R}^d, \quad (2.11)$$

which means $u^\top \nabla^2 f(\mathbf{x}) u \geq 0$ for all $u \in \mathbb{R}^d$. One more equivalent condition is:

$$(\mathbf{x} - \mathbf{y})^\top (\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})) \geq 0, \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^d. \quad (2.12)$$

Example 2.6. The easiest example of a convex function is

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x} - a^\top \mathbf{x},$$

with S positive semi-definite. For this function $\nabla^2 f(\mathbf{x}) = S$. To compute the minimum we set $\nabla f(\mathbf{x}) = S\mathbf{x} - a = 0$, so that $\mathbf{x}^* = \arg \min_{\mathbf{x}} f(\mathbf{x}) = S^{-1}a$

2.1.3 L -smoothness

Definition 2.7 (Lipschitz continuity). A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is called Lipschitz continuous with Lipschitz constant $L > 0$ if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$,

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L \|\mathbf{x} - \mathbf{y}\|. \quad (2.13)$$

Definition 2.8 (L -smoothness). A differentiable function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is called L -smooth if its gradient is Lipschitz continuous:

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|. \quad (2.14)$$

In other words, L -smoothness is putting an upper bound on the curvature of the function. The benefit of such functions is that their gradients tend to decay when $\mathbf{x}$ gets closer to the minimum; unlike nonsmooth functions who may have abrupt bends around the minimum causing oscillations in gradient descent.

Proposition 2.9. Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a twice-differentiable function. Then f is L -smooth if and only if for all $\mathbf{x} \in \mathbb{R}^d$,

$$\forall \mathbf{u}, \mathbf{v} \in \mathbb{R}^d, \quad \langle \nabla^2 f(\mathbf{x}) \mathbf{u}, \mathbf{v} \rangle \leq L\|\mathbf{u}\|\|\mathbf{v}\|.$$

Proof. Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$. We apply the fundamental theorem of calculus to the function $t \mapsto \nabla f(\mathbf{x} + t\mathbf{h})$ with $\mathbf{h} = \mathbf{y} - \mathbf{x}$ to obtain

$$\nabla f(\mathbf{y}) - \nabla f(\mathbf{x}) = \int_0^1 \nabla^2 f(\mathbf{x} + t\mathbf{h}) \mathbf{h} \, dt.$$

Therefore,

$$\|\nabla f(\mathbf{y}) - \nabla f(\mathbf{x})\| \leq \int_0^1 \|\nabla^2 f(\mathbf{x} + t\mathbf{h}) \mathbf{h}\| \, dt \leq \int_0^1 L\|\mathbf{h}\| \, dt = L\|\mathbf{h}\|.$$

For the other side, we assume that f is L -smooth. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, and define $\psi(t) = \langle \nabla f(\mathbf{x} + t\mathbf{u}) - \nabla f(\mathbf{x}), \mathbf{v} \rangle$. By L -smoothness, we have $\psi(t) \leq Lt\|\mathbf{u}\|\|\mathbf{v}\|$, and so $\psi'(t) = \lim_{t \rightarrow 0} (\psi(t) - \psi(0))/t \leq L\|\mathbf{u}\|\|\mathbf{v}\|$. However, $\psi'(0) = \langle \nabla^2 f(\mathbf{x}) \mathbf{u}, \mathbf{v} \rangle$, which concludes the proof. □

Remark 2.10. If f is twice differentiable, one can equivalently state that L -smoothness holds if

$$\|\nabla^2 f(\mathbf{x})\| \leq L,$$

or alternatively that all eigenvalues of the Hessian are less than L :

$$\mathbf{u}^\top \nabla^2 f(\mathbf{x}) \mathbf{u} \leq L\|\mathbf{u}\|^2.$$

Proposition 2.11. Let f be an L -smooth function on $\mathbb{R}^d$, then

$$|f(\mathbf{x}) - f(\mathbf{y}) - \nabla f(\mathbf{y})^\top (\mathbf{x} - \mathbf{y})| \leq \frac{L}{2} \|\mathbf{x} - \mathbf{y}\|^2.$$

Hence, L -smoothness holds if,

$$f(\mathbf{x}) \leq f(\mathbf{y}) + \nabla f(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{L}{2} \|\mathbf{x} - \mathbf{y}\|_2^2.$$

Intuitively, at every point $\mathbf{y}$, there is a corresponding quadratic function that upper bounds f everywhere and agrees with f at the point $\mathbf{x} = \mathbf{y}$.

2.1.4 Strong convexity

Another condition that is useful is that of strong convexity; it will allow one to obtain faster convergence results.

Definition 2.12 (Strong convexity). A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is μ -strongly convex with $\mu > 0$ if for $t \in [0, 1]$,

$$f(t\mathbf{x} + (1-t)\mathbf{y}) \leq tf(\mathbf{x}) + (1-t)f(\mathbf{y}) - \frac{\mu}{2}t(1-t)\|\mathbf{x} - \mathbf{y}\|_2^2. \quad (2.15)$$

Proposition 2.13. If f is μ -strongly convex and differentiable on $\mathbb{R}^d$, then for any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$,

$$f(\mathbf{x}) \geq f(\mathbf{y}) + \nabla f(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{\mu}{2} \|\mathbf{x} - \mathbf{y}\|_2^2. \quad (2.16)$$

The opposite is true if f is differentiable everywhere. Similarly, if f is twice-differentiable, then μ -strong convexity is equivalent to

$$u^\top \nabla^2 f(\mathbf{x}) u \geq \mu \|u\|^2, \quad \forall \mathbf{x}, u \in \mathbb{R}^d. \quad (2.17)$$

Note that a function f is μ -strongly convex if and only if the function $\mathbf{y} \mapsto f(\mathbf{y}) - \mu \|\mathbf{y}\|^2/2$ is convex. Intuitively, strong convexity means that there exists a quadratic lower bound on the growth of the function.

Let f be a μ -strongly convex and f^* denotes the minimum value of f , the following condition holds.

$$\|\nabla f(\mathbf{x})\|^2 \geq 2\mu(f(\mathbf{x}) - f^*). \quad (2.18)$$

This is known as the Polyak-Lojasiewicz condition and it shows that the gradient grows as a quadratic function of sub-optimality. It holds for the μ -strongly convex functions, and also certain other function classes can satisfy this condition.

Example 2.14. The easiest example of a μ -strongly convex function is $f(x) = \mu x^2/2$ this function is however not $(\mu + \epsilon)$ -strongly convex for $\epsilon > 0$. Another example of a μ -strongly convex function is.

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x} - a^\top \mathbf{x},$$

with S positive definite (not semidefinite this time!). For this function $\nabla^2 f(\mathbf{x}) = S$ and to have $\nabla^2 f(\mathbf{x}) \succ 0$ we need S positive definite.

2.2 Iterative methods: the intuition

We would like to find an iterative method that moves from one point to another, ideally improving the objective function at each iteration. In other words, we would like to move from $\mathbf{x}_0$, to $\mathbf{x}_1$, ... in a way that:

$$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \dots$$

Suppose we want to generate such a direction using the following iterative algorithm:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k p_k,$$

where P_k is some descent direction and α_k is the stepsize. Assume that f is a differentiable function; by Taylor's theorem we have:

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) + \alpha_k \nabla f(\mathbf{x}_k)^\top p_k + \mathcal{O}(\alpha_k^2 \|p_k\|_2^2). \quad (2.19)$$

For α_k small enough, the term $\alpha_k \nabla f(\mathbf{x}_k)^\top p_k$ dominates $\mathcal{O}(\alpha_k^2)$ for a fixed p_k . Therefore, in order to have $f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k)$, we require,

$$\nabla f(\mathbf{x}_k)^\top p_k < 0.$$

Since, (see for example the first page in <https://www.math.usm.edu/lambers/mat169/fall109/lecture21.pdf>)

$$\nabla f(\mathbf{x}_k)^\top p_k = \|\nabla f(\mathbf{x}_k)\| \|p_k\| \cos \theta,$$

where θ is the angle between $\nabla f(\mathbf{x}_k)$ and p_k , we have

$$p_k := -\nabla f(\mathbf{x}_k), \quad (2.20)$$

as the local steepest descent direction ($\cos \pi = -1$).

2.3 Gradient descent

The gradient descent method for a function f is given as follows. Consider a starting point $\mathbf{x}_0$ in the domain of f . Update your parameters as follows:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k). \quad (2.21)$$

We will require some kind of optimality criterion. A standard choice is saying: if $\|\nabla f(\mathbf{x}_k)\|_2 \leq \epsilon$, then stop. We also need a learning rate schedule α_k , $k = 1, \dots, K$. A common schedule is to use a diminishing step size (for some a and b):

$$\alpha_k = \frac{1}{a + bk}, \quad (2.22)$$

or $\alpha_k = a \exp(-bk)$. Decreasing step size algorithms are popular in applied machine learning and typically, because they perform well and theoretical convergence results are fairly easily obtained for them. We will later also discuss how to set the learning rate with exact line search.

2.3.1 Gradient descent - convergence proofs

There exist many different convergence proofs which depend on the assumptions we make on the objective function.

Theorem 2.15 (Gradient descent for L -smooth function). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a twice-differentiable function, L -smooth, and $\epsilon > 0$. Then, gradient descent with a step size of $\alpha \leq 1/L$ achieves a point $\mathbf{x}^*$ such that $\|\nabla f(\mathbf{x}^*)\|_2 \leq \epsilon$ in T iterations, where T satisfies:

$$T \geq \frac{2(f(\mathbf{x}_0) - f^*)}{\alpha \epsilon^2}. \quad (2.23)$$

Proof. What will we assume for this proof? We will use:

$$|u^\top \nabla^2 f(\mathbf{x}) u| \leq L \|u\|_2^2, \quad (2.24)$$

for any $u \in \mathbb{R}^d$ and for $L > 0$. Here, $\nabla^2 f(\mathbf{x})$ denotes the matrix of partial derivatives of the function f . This is equivalent to the condition that $\|\nabla^2 f(\mathbf{x})\|_2 \leq L$ or that f is L -smooth.

Let us start to derive how the objective function changes over time as we run gradient descent with a fixed step size. From Taylor's theorem, there exists a ζ_k such that,

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)) \quad (2.25)$$

$$= f(\mathbf{x}_k) - \alpha \nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_k) + \frac{1}{2} (\alpha \nabla f(\mathbf{x}_k))^\top \nabla^2 f(\zeta_k) (\alpha \nabla f(\mathbf{x}_k)) \quad (2.26)$$

$$\leq f(\mathbf{x}_k) - \alpha \|\nabla f(\mathbf{x}_k)\|^2 + \frac{\alpha^2 L}{2} \|\nabla f(\mathbf{x}_k)\|^2 \quad (2.27)$$

$$= f(\mathbf{x}_k) - \alpha \left(1 - \frac{\alpha L}{2}\right) \|\nabla f(\mathbf{x}_k)\|^2, \quad (2.28)$$

where in the second-to-last line we have used the condition given by Eq. (2.24). Remember that we wanted to have $f(\mathbf{x}_k)$ decrease with every iteration k . How can we achieve this using the above? We need $1 - \alpha L/2 \geq 0$, or: $2 \geq \alpha L$. Suppose we choose something stronger: $\alpha L \leq 1$. Then, it holds:

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \frac{\alpha}{2} \|\nabla f(\mathbf{x}_k)\|^2. \quad (2.29)$$

That is, the objective decreases at each iteration. This implies

$$\frac{\alpha}{2} \|\nabla f(\mathbf{x}_k)\|^2 \leq f(\mathbf{x}_k) - f(\mathbf{x}_{k+1}).$$

Sum this up across T iterations of gradient descent to get:

$$\frac{\alpha}{2} \sum_{k=0}^{T-1} \|\nabla f(\mathbf{x}_k)\|^2 \leq \sum_{k=0}^{T-1} f(\mathbf{x}_k) - f(\mathbf{x}_{k+1}) = f(\mathbf{x}_0) - f(\mathbf{x}_T) \leq f(\mathbf{x}_0) - f^*$$

where f^* is the global minimum value of the loss function f (the last inequality thus holds trivially as the value at the global minimum is the lowest of all). From here, we can get

$$\min_{k \in \{0, \dots, T-1\}} \|\nabla f(\mathbf{x}_k)\|^2 \leq \frac{1}{T} \sum_{k=0}^{T-1} \|\nabla f(\mathbf{x}_k)\|^2 \leq \frac{2(f(\mathbf{x}_0) - f^*)}{\alpha T}.$$

Why does the first inequality hold? The average over some values will always be larger than the smallest of those values. What does this result tell us? The smallest gradient we observe after T iterations is getting smaller proportional to $1/T$. So long as we look at the smallest gradient, gradient descent converges (converges in the sense that we reach close to zero gradient). $\square$

In the convex case, one can show the following result.

Theorem 2.16 (Convergence for a convex objective). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be an L -smooth and convex function. Then, if we set the learning rate to $\alpha = 1/L$ and denote $(\mathbf{x}_k)_k$ the gradient descent sequence, we have

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \frac{2L \|\mathbf{x}_0 - \mathbf{x}^*\|_2^2}{k}, \quad (2.30)$$

where $\mathbf{x}^* \in \arg \min_{\mathbf{x}} f(\mathbf{x})$ (we assume that the minimum is attained but not

necessarily unique).

We can further go on to derive convergence for a μ -strongly convex objective.

Theorem 2.17 (Convergence for a μ -strongly convex objective). Let f be a twice-differentiable function and suppose it is both L -smooth and μ -strongly convex. Then, if we set the learning rate to $\alpha = 1/L$ and denote $(\mathbf{x}_k)_k$ the gradient descent sequence,

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \exp\left(-\frac{\mu}{L}k\right) (f(\mathbf{x}_0) - f(\mathbf{x}^*)). \quad (2.31)$$

Proof. Remember the expression from Eq. (2.29) (which holds for our choice of α):

$$f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k) \leq -\frac{1}{2}\alpha \|\nabla f(\mathbf{x}_k)\|_2^2. \quad (2.32)$$

Now, remember the condition in Eq. (2.18):

$$\|\nabla f(\mathbf{x})\|^2 \geq 2\mu(f(\mathbf{x}) - f(\mathbf{x}^*)). \quad (2.33)$$

Let us apply this to the above (and setting $\alpha = 1/L$):

$$f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k) \leq -\frac{\mu}{L}(f(\mathbf{x}_k) - f(\mathbf{x}^*)). \quad (2.34)$$

Subtract $f(\mathbf{x}^*)$ from both sides,

$$f(\mathbf{x}_{k+1}) - f(\mathbf{x}^*) \leq f(\mathbf{x}_k) - f(\mathbf{x}^*) - \frac{\mu}{L}(f(\mathbf{x}_k) - f(\mathbf{x}^*)) \quad (2.35)$$

$$= \left(1 - \frac{\mu}{L}\right) (f(\mathbf{x}_k) - f(\mathbf{x}^*)). \quad (2.36)$$

Repeat the above recursively,

$$f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \left(1 - \frac{\mu}{L}\right)^k (f(\mathbf{x}_0) - f(\mathbf{x}^*)) \quad (2.37)$$

$$\leq \exp\left(-\frac{\mu}{L}k\right) (f(\mathbf{x}_0) - f(\mathbf{x}^*)). \quad (2.38)$$

□

The above shows that with strong convexity one can achieve this exponential decrease. Sometimes, this is referred to as a linear rate. Furthermore, it shows the dependence on the quantity

$$\frac{L}{\mu}. \quad (2.39)$$

This quantity is known as the *condition number*. The condition number encodes how hard a strongly convex problem is to solve. When μ/L is very small (i.e. L is very large), then GD converges very slowly as $\exp(-\mu/L)$ is close to 1.

2.3.2 Gradient descent with exact line search

In the above, we set an α_k a priori. An alternative is to choose α_k to make $f(\mathbf{x}_{k+1})$ a minimum in the search direction. The search direction is given by the gradient $\nabla f(\mathbf{x})$. The α_k represents how far in the search direction we move. An exact line search wants to avoid ‘overshooting’ the descent direction, thus avoiding that we go up in function value again. In the notation of [Boyd and Vandenberghe, 2004], α is chosen to minimise f along the ray $\{\mathbf{x} + t\Delta\mathbf{x} | t \geq 0\}$:

$$t = \arg \min_{s \geq 0} f(\mathbf{x} + s\Delta\mathbf{x}). \quad (2.40)$$

Example 2.18. Consider the following function.

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x},$$

where $S \in \mathbb{R}^{2 \times 2}$ (we will later specify its values). Let’s assume we use gradient descent with exact line search for the step size. Let’s derive the exact line search step size. We have,

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)) \quad (2.41)$$

$$= \frac{1}{2} (\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k))^\top S (\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)) \quad (2.42)$$

$$= \frac{1}{2} \mathbf{x}_k^\top S \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)^\top S \mathbf{x}_k + \frac{1}{2} \alpha_k^2 \nabla f(\mathbf{x}_k)^\top S \nabla f(\mathbf{x}_k) \quad (2.43)$$

$$= f(\mathbf{x}_k) + \frac{1}{2} \alpha_k^2 \nabla f(\mathbf{x}_k)^\top S \nabla f(\mathbf{x}_k) - \alpha_k \nabla f(\mathbf{x}_k)^\top S \mathbf{x}_k =: g(\alpha_k) \quad (2.44)$$

We thus look at the above as a function of α_k . In exact line search we want to find the α_k such that $g(\alpha_k)$ is minimised, i.e. the step size that leads us to the lowest value of the objective function at the next iteration. How to minimise? Let’s take the gradient:

$$g'(\alpha_k) = \alpha_k \nabla f(\mathbf{x}_k)^\top S \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_k)^\top S \mathbf{x}_k, \quad (2.45)$$

and set it to zero. Solving for α_k we get,

$$\alpha_k = \frac{\nabla f(\mathbf{x}_k)^\top S \mathbf{x}_k}{\nabla f(\mathbf{x}_k)^\top S \nabla f(\mathbf{x}_k)}. \quad (2.46)$$

2.4 Accelerating gradient descent using momentum

Remember the basic gradient descent algorithm with a fixed learning rate:

$$\mathbf{x}_{x+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k). \quad (2.47)$$

In general, if we choose the step size small enough, gradient descent always converges (we saw this in the last section) - albeit to a local minimum. However, this rate of convergence can be very small.

2.4.1 The issue with gradient descent

Consider the following function.:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x}, \quad (2.48)$$

where $S \in \mathbb{R}^{2 \times 2}$ (we will later specify its values). Suppose now we set

$$S = \begin{bmatrix} L & 0 \\ 0 & \mu \end{bmatrix}. \quad (2.49)$$

Suppose $L \geq \mu > 0$. Observe that the condition number of this matrix is given by $\lambda_{\max}/\lambda_{\min} = L/\mu$ (where $\lambda_{\max}$ and $\lambda_{\min}$ are the largest and smallest eigenvalues). Where is the minimum of this function? It is at $\mathbf{x} = [0, 0]^\top$.

If we think about what gradient descent, with constant learning rate, is doing for this problem, we can derive the following update rules:

$$\mathbf{x}_{k+1}^1 = \mathbf{x}_k^1 - \alpha L \mathbf{x}_k^1, \quad (2.50)$$

$$\mathbf{x}_{k+1}^2 = \mathbf{x}_k^2 - \alpha \mu \mathbf{x}_k^2. \quad (2.51)$$

This means that,

$$(\mathbf{x}_T^1)^2 = (1 - \alpha L)^{2T} (\mathbf{x}_0^1)^2, \quad (2.52)$$

$$(\mathbf{x}_T^2)^2 = (1 - \alpha \mu)^{2T} (\mathbf{x}_0^2)^2. \quad (2.53)$$

and so

$$f(\mathbf{x}_T) = \frac{L}{2} (\mathbf{x}_T^1)^2 + \frac{\mu}{2} (\mathbf{x}_T^2)^2 \quad (2.54)$$

$$= \frac{L}{2} (1 - \alpha L)^{2T} (\mathbf{x}_0^1)^2 + \frac{\mu}{2} (1 - \alpha \mu)^{2T} (\mathbf{x}_0^2)^2. \quad (2.55)$$

How should we choose α ? As $T \rightarrow \infty$ the above expression is dominated by whichever of the two terms decreases slower. One can thus say that:

$$f(\mathbf{x}_T) = \mathcal{O}(\max\{|1 - \alpha L|, |1 - \alpha \mu|\}^{2T}). \quad (2.56)$$

In principle, we would want to set α small for dimensions with more curvature (in this case the L direction), and large for directions with less curvature (the one

with μ). However, we only have one learning rate parameter so we cannot do this directly. Fix some choice of α . Suppose now that L is very large and μ is small; what does this mean? We will only take very small steps in the direction of the $\mathbf{x}^2$ and thus approach zero (the optimum) very slowly (because $1 - \alpha\mu$ will be very close to one. The function value is also decreasing extremely slowly (because of the second term). Increasing the learning rate will lead to us ‘overshooting’ the minimum as then in $\mathbf{x}^1$ we may use a too large step size.

The optimal learning rate would be:

$$\alpha^* = \arg \min_{\alpha} (\max\{|1 - \alpha L|, |1 - \alpha \mu|\}). \quad (2.57)$$

Note that $|1 - \alpha L|$ (as a function of α) is a piecewise linear function that reaches zero at $\alpha = 1/L$ and the y -axis at 1; similarly for $|1 - \alpha \mu|$. Then, the maximum of the two piecewise linear functions is achieved by $1 - \mu L$ before the point where $\alpha L - 1 = 1 - \mu \alpha$ (which is $\alpha = 2/(\mu + L)$) and $\alpha L - 1$ after that point. The minimum of the maximum is then achieved at $\alpha^* = 2/(\mu + L)$. The easiest way to see all of the above is to just plot the lines on a graph - see Fig. 2.1.

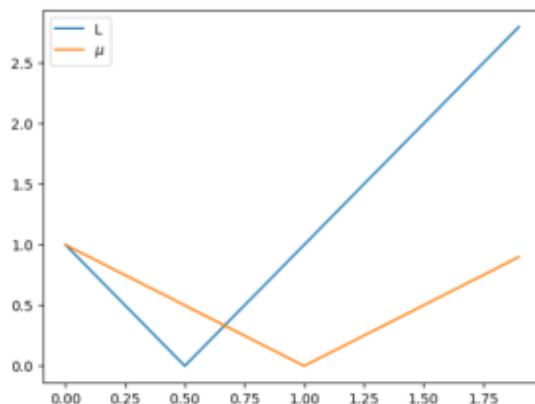


Figure 2.1: The $|1 - \alpha L|$ and $|1 - \alpha \mu|$ as a function of α for $L = 2$ and $\mu = 1$. Suppose we just have a one-dimensional quadratic $x^T L x$: we would choose the step size α at the minimum of $|1 - \alpha L|$. Generally, it would be smaller if L is larger. For a two-dimension quadratic: we'd like to set the step size larger for dimension with less curvature, and smaller for the dimension with more curvature. However, we only have one parameter. So the best we can do is choose it as $\arg \min_{\alpha} (\max\{|1 - \alpha L|, |1 - \alpha \mu|\})$: optimal convergence rate is substantially worse than what we'd get in each scenario individually.

Pathological curvature. What is the problem? Pathological curvature. The issue is that regions of f are not scaled properly, i.e. we have some kind of valleys. Progress along certain directions grinds to a halt because the gradient is very small there. See Figs. 2.2 and 2.3. Imagine that the condition number in Fig. 2.2 would

be even higher: we would have even more zig-zagging and would be approaching the optimum very slowly.

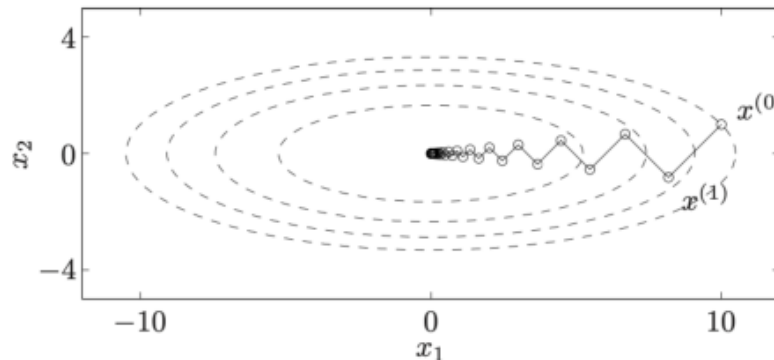


Figure 2.2: Some contour lines of the function $f(x) = (1/2)(x_1^2 + 10x_2^2)$. The condition number of the sublevel sets, which are ellipsoids, is exactly 10. The figure shows the iterates of the gradient method with exact line search, started at $x^{(0)} = (10, 1)$. The function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T S \mathbf{x}$ with $S = \begin{bmatrix} 1 & 0 \\ 0 & 10 \end{bmatrix}$ with $b = 10$. The picture is [Boyd and Vandenberghe, 2004, Fig. 9.2].

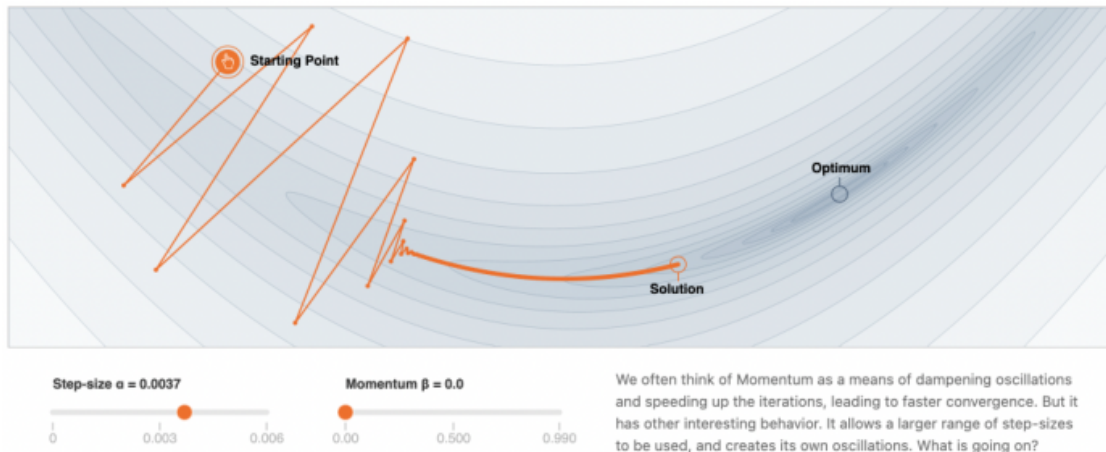


Figure 2.3: An objective function where-in we perform gradient descent (no momentum!). The zig-zag pattern is standard for a gradient-based descent method. In each iteration we take a step in the direction perpendicular to the level set. Picture is taken from <https://distill.pub/2017/momentum>. The zig-zags are too large and we do not converge to the optimum at the chosen learning rate, while a higher learning rate will make things zig-zag all over the place.

2.4.2 Improving convergence with momentum

So on a high level, the idea of momentum is to tell the difference between directions with more or less curvature using information that is already available in gradient descent. We would then use:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{z}_{k+1}, \quad (2.58)$$

$$\mathbf{z}_{k+1} = \beta \mathbf{z}_k + \nabla f(\mathbf{x}_k), \quad (2.59)$$

where $\beta \in (0, 1)$ is some hyperparameter. The idea of this method is that we have a kind of history or memory term - this memory term is what will help us converge better. The idea is as follows: if the gradients are reversing sign, we are going too fast and are overshooting the minimum, hence we'd like to slow down; if the gradients are staying in the same direction, maybe we'd like to speed up a little bit.

2.4.2.1 The analysis through eigenvectors

Proposition 2.19 (Gradient descent rate). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be the quadratic function $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x} - b^\top \mathbf{x}$, where S is a symmetric positive definite matrix. Denote by $\mathbf{x}^* = S^{-1}b$ the optimum, then the optimal learning rate of gradient descent is $\alpha^* = 2/(\lambda_1 + \lambda_d)$, where λ_1 and λ_d are the smallest and largest eigenvalues of S , with convergence:

$$\|\mathbf{x}_k - \mathbf{x}^*\|_2 = \left(\frac{\mathcal{K}(S) - 1}{\mathcal{K}(S) + 1} \right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_2, \quad (2.60)$$

where $\mathcal{K}(S) = \lambda_d/\lambda_1$ is the condition number of S .

Proof. The gradient descent is then given by,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha(S\mathbf{x}_k - b). \quad (2.61)$$

Every real, symmetric matrix S has an eigenvalue decomposition:

$$S = Q\Lambda Q^\top, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_d),$$

where Q is an orthogonal matrix and we assume that the eigenvalues λ_i are sorted as $0 < \lambda_1 \leq \dots \leq \lambda_d$. Then, Eq. (2.61) becomes

If we perform a change of basis $\mathbf{w}_k = Q^\top(\mathbf{x}_k - \mathbf{x}^*)$, we obtain, (where I now use w^i to denote each element of $\mathbf{w}$ and similarly for $\mathbf{x}$ and x^i),

$$w_{k+1}^i = w_k^i - \alpha \lambda_i w_k^i \quad (2.62)$$

$$= (1 - \alpha \lambda_i) w_k^i = (1 - \alpha \lambda_i)^{k+1} w_0^i. \quad (2.63)$$

Therefore,

$$\mathbf{w}_k = \begin{bmatrix} (1 - \alpha \lambda_1) & 0 & \dots & 0 \\ 0 & (1 - \alpha \lambda_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & (1 - \alpha \lambda_d) \end{bmatrix}^k \mathbf{w}_0. \quad (2.64)$$

We denote by D the diagonal matrix with the $(1 - \alpha\lambda_i)$ on the diagonal. Then, we have that

$$\|\mathbf{w}_k\|_2 \leq \max_i |1 - \alpha\lambda_i|^k \|\mathbf{w}_0\|_2. \quad (2.65)$$

Moreover, the ℓ_2 -norm is invariance under orthogonal transformations as $\|Qx\|_2 = \sqrt{(Qx)^\top(Qx)} = \sqrt{x^\top(Q^\top Q)x} = \|x\|_2$ since $Q^\top Q = I$. Therefore, since $\mathbf{x}_k - \mathbf{x}^* = Q\mathbf{w}_k$, we have

$$\|\mathbf{x}_k - \mathbf{x}^*\|_2 = \max_i |1 - \alpha\lambda_i|^k \|\mathbf{x}_0 - \mathbf{x}^*\|_2. \quad (2.66)$$

In each of the directions $\mathbf{q}_i$ we thus track the error through the initial value x_0^i and the compounding rate $1 - \alpha\lambda_i$. The closer this compounding rate is to 1, the slower the convergence. The eigenvectors with the largest eigenvalues converge the fastest. This results in an explosion of progress in the first iterations, but then things slow down. In order to converge, $|1 - \alpha\lambda_i|$ should be strictly less than 1. The overall convergence rate is determined by the slowest error component which will be either the λ_1 (in previous notation $\lambda_{\min}$) or λ_d (in previous notation $\lambda_{\max}$) $\max\{|1 - \alpha\lambda_1|, |1 - \alpha\lambda_d|\}$. The optimal learning rate minimises the above rate,

$$\alpha^* = \arg \min_{\alpha} \max\{|1 - \alpha\lambda_1|, |1 - \alpha\lambda_d|\}.$$

The solution is given by $\alpha^* = 2/(\lambda_1 + \lambda_d)$ (same logic as we saw before in gradient descent). The rate is then,

$$\max\{|1 - \alpha^*\lambda_1|, |1 - \alpha^*\lambda_d|\} = \frac{\lambda_d/\lambda_1 - 1}{\lambda_d/\lambda_1 + 1}. \quad (2.67)$$

We already saw that the number λ_d/λ_1 is known as the condition number. $\square$

Proposition 2.20 (Momentum gradient descent rate). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be the quadratic function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top S\mathbf{x} - b^\top \mathbf{x}$, where S is a symmetric positive definite matrix. Denote by $\mathbf{x}^* = S^{-1}b$ the optimum, then the optimal learning rate of momentum gradient descent is $\alpha^* = (2 + 2\beta)/(\lambda_1 + \lambda_d)$, with

$$\beta = \left(\frac{\sqrt{\lambda_d} - \sqrt{\lambda_1}}{\sqrt{\lambda_d} + \sqrt{\lambda_1}} \right)^2,$$

where λ_1 and λ_d are the smallest and largest eigenvalues of S , with convergence:

$$\|\mathbf{x}_k - \mathbf{x}^*\|_2 = \left(\frac{\mathcal{K}(S)^{1/2} - 1}{\mathcal{K}(S)^{1/2} + 1} \right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_2, \quad (2.68)$$

where $\mathcal{K}(S) = \lambda_d/\lambda_1$ is the condition number of S .

For momentum gradient descent, the updates become,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{z}_{k+1}, \quad (2.69)$$

$$\mathbf{z}_{k+1} = \beta \mathbf{z}_k + (S\mathbf{x}_k - b). \quad (2.70)$$

Using now $\mathbf{w}_k = Q^\top(\mathbf{x}_k - \mathbf{x}^*)$ and $\mathbf{y}_k = Q^\top \mathbf{z}_k$, we obtain

$$w_{k+1}^i = w_k^i - \alpha y_{k+1}^i, \quad (2.71)$$

$$y_{k+1}^i = \beta y_k^i + \lambda_i w_k^i. \quad (2.72)$$

We can rewrite this as,

$$\begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix} \begin{bmatrix} w_{k+1}^i \\ y_{k+1}^i \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \lambda_i & \beta \end{bmatrix} \begin{bmatrix} w_k^i \\ y_k^i \end{bmatrix}. \quad (2.73)$$

Note that

$$\begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & -\alpha \\ 0 & 1 \end{bmatrix}. \quad (2.74)$$

Hence,

$$\begin{bmatrix} w_{k+1}^i \\ y_{k+1}^i \end{bmatrix} = \begin{bmatrix} 1 - \alpha\lambda_i & -\alpha\beta \\ \lambda_i & \beta \end{bmatrix} \begin{bmatrix} w_k^i \\ y_k^i \end{bmatrix}. \quad (2.75)$$

Consequently,

$$\begin{bmatrix} w_{k+1}^i \\ y_{k+1}^i \end{bmatrix} = R^{k+1} \begin{bmatrix} w_0^i \\ y_0^i \end{bmatrix}, \quad (2.76)$$

where,

$$R = \begin{bmatrix} 1 - \alpha\lambda_i & -\alpha\beta \\ \lambda_i & \beta \end{bmatrix}. \quad (2.77)$$

When will the above converge fast? If R^k goes to zero fast. One note: in a way (informally) R has thus taken on the role of the $1 - \alpha L$ and $1 - \alpha\mu$ terms; if R has ‘nicer’ properties than those two terms we will achieve better convergence. Let us analyse R^{k+1} . Assuming that R is diagonalizable (i.e. it has a full complement of eigenvectors), we have $R = \tilde{Q} \text{diag}(e_1, e_2) \tilde{Q}^{-1}$. Then,

$$\left\| \begin{bmatrix} w_{k+1}^i \\ y_{k+1}^i \end{bmatrix} \right\| = \left\| R^{k+1} \begin{bmatrix} w_0^i \\ y_0^i \end{bmatrix} \right\| \quad (2.78)$$

$$= \left\| \tilde{Q} (\text{diag}(e_1, e_2))^{k+1} \tilde{Q}^{-1} \begin{bmatrix} w_0^i \\ y_0^i \end{bmatrix} \right\| \quad (2.79)$$

$$\leq \|\tilde{Q}\| \|(\text{diag}(e_1, e_2))^{k+1}\| \|\tilde{Q}^{-1}\| \left\| \begin{bmatrix} w_0^i \\ y_0^i \end{bmatrix} \right\| \quad (2.80)$$

$$\leq \max\{|e_1|, |e_2|\}^{k+1} \|\tilde{Q}\| \|\tilde{Q}^{-1}\| \left\| \begin{bmatrix} w_0^i \\ y_0^i \end{bmatrix} \right\|. \quad (2.81)$$

So this algorithm is going to converge at a linear rate determined by the maximum absolute value of an eigenvalue of R .

Let's compute the eigenvalues of R . The characteristic polynomial is given by

$$\det(eI - R) = 0, \quad (2.82)$$

$$(e - 1 + \alpha\lambda_i)(e - \beta) + \alpha\beta\lambda_i = 0 \quad (2.83)$$

$$e^2 - (\beta + 1 - \alpha\lambda_i)e + \beta = 0. \quad (2.84)$$

Solving the above using the quadratic formula:

$$e = \frac{(\beta + 1 - \alpha\lambda_i) \pm \sqrt{(\beta + 1 - \alpha\lambda_i)^2 - 4\beta}}{2}. \quad (2.85)$$

When

$$(\beta + 1 - \alpha\lambda_i)^2 - 4\beta < 0, \quad (2.86)$$

the roots are complex and hence the eigenvalues are complex, and in particular they must be complex conjugates of each other: $e_1 = e_2^*$, where e_2^* denotes the complex conjugate of e_2 (remember: a complex conjugate is formed by changing the sign between two terms in a complex number). It then holds.

$$|e_1| = |e_2| = \sqrt{e_2^* \cdot e_2} = \sqrt{e_1 \cdot e_2}.$$

Furthermore, $e_1 e_2$ is equal to the determinant of R which is β . Thus, $|e_1| = |e_2| = \sqrt{\beta}$. Thus, momentum gradient descent will be converging at a rate $\sqrt{\beta}$ if the above condition is satisfied. This rate is, surprisingly, independent of the step-size or the eigenvalue $\alpha\lambda$. In the general case of real roots, the rate will be given by $\max\{|e_1|, |e_2|\}$ with e_1, e_2 as in Eq. (2.85).

How we choose α and β and the properties of the original matrix S , specifically its eigenvalues λ_i , thus determine whether convergence is achieved and what kind of convergence. As a special case comes the case when the convergence is at rate $\sqrt{\beta}$ as we saw above. More generally, to obtain convergence we would want both $|e_1| < 1$ and $|e_2| < 1$. If we want to set α and β so that an optimal convergence rate is achieved, we would optimise over α and β for each possible λ_i :

$$\min_{\alpha, \beta} \max\{|e_1(\lambda_1)|, |e_2(\lambda_1)|, \dots, |e_1(\lambda_d)|, |e_2(\lambda_d)|\}. \quad (2.87)$$

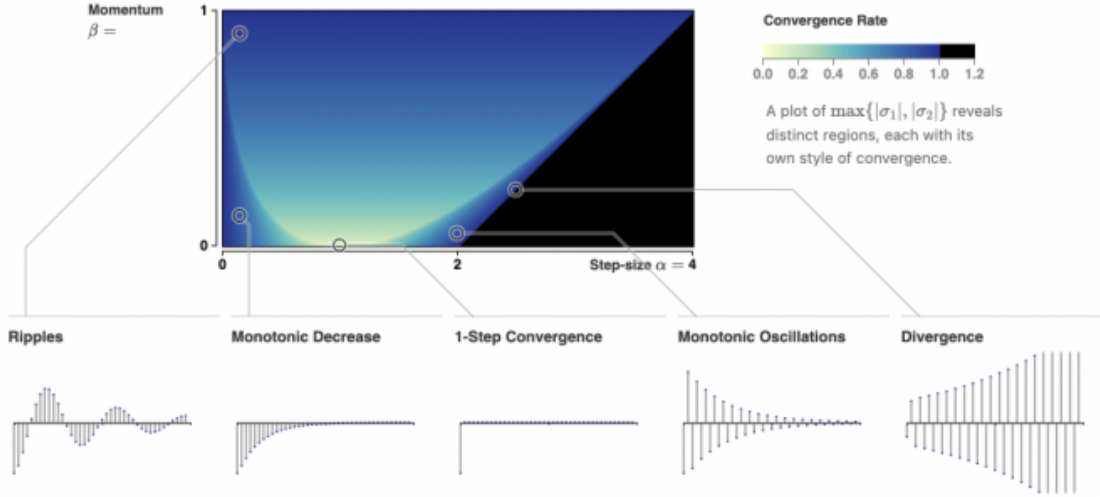


Figure 2.4: Different convergence rates. Picture is taken from <https://distill.pub/2017/momentum>

Fig. 2.4 shows the convergence rates for different choices of parameters.

If in each of the directions corresponding to λ_i the condition given by Eq. (2.86) is satisfied, all directions will converge at the rate $\sqrt{\beta}$. This will happen if (rewriting Eq. (2.86)),

$$-1 \leq \frac{1 + \beta - \alpha\lambda_i}{2\sqrt{\beta}} \leq 1, \quad (2.88)$$

is satisfied for all λ_i . Remember in our notation λ_1 was the smallest eigenvalue and λ_d the largest. Then the above will hold if,

$$-1 \leq \frac{1 + \beta - \alpha\lambda_d}{2\sqrt{\beta}}, \quad \frac{1 + \beta - \alpha\lambda_1}{2\sqrt{\beta}} \leq 1. \quad (2.89)$$

Let's solve for when the above holds with equality (the extreme cases):

$$-1 = \frac{1 + \beta - \alpha\lambda_d}{2\sqrt{\beta}}, \quad \frac{1 + \beta - \alpha\lambda_1}{2\sqrt{\beta}} = 1. \quad (2.90)$$

Adding both equations we get:

$$0 = \frac{2 + 2\beta - \alpha\lambda_d - \alpha\lambda_1}{2\sqrt{\beta}}, \quad (2.91)$$

$$0 = 2 + 2\beta - \alpha\lambda_d - \alpha\lambda_1, \quad (2.92)$$

$$\alpha = \frac{2 + 2\beta}{\lambda_1 + \lambda_d}. \quad (2.93)$$

Subtracting the right one from the left we get:

$$2 = \frac{-\alpha\lambda_1 + \alpha\lambda_d}{2\sqrt{\beta}}, \quad (2.94)$$

$$2 = \frac{\alpha(\lambda_d - \lambda_1)}{2\sqrt{\beta}}, \quad (2.95)$$

$$4\sqrt{\beta} = \alpha(\lambda_d - \lambda_1). \quad (2.96)$$

Plugging the expression for α into the one for $4\sqrt{\beta}$ we obtain

$$4\sqrt{\beta} = (\lambda_d - \lambda_1) \frac{2 + 2\beta}{\lambda_1 + \lambda_d}, \quad (2.97)$$

$$2 + 2\beta = 4\sqrt{\beta} \frac{\lambda_d + \lambda_1}{\lambda_d - \lambda_1}, \quad (2.98)$$

$$\beta - 2\sqrt{\beta} \frac{\lambda_d + \lambda_1}{\lambda_d - \lambda_1} + 1 = 0. \quad (2.99)$$

Solving the quadratic equation and considering only the negative solution since we want $\sqrt{\beta}$ to be small to achieve faster convergence:

$$\sqrt{\beta} = \frac{\lambda_d + \lambda_1}{\lambda_d - \lambda_1} - \sqrt{\left(\frac{\lambda_d + \lambda_1}{\lambda_d - \lambda_1}\right)^2 - 1} \quad (2.100)$$

$$= \frac{\lambda_d + \lambda_1}{\lambda_d - \lambda_1} - \sqrt{\frac{4\lambda_1\lambda_d}{(\lambda_d - \lambda_1)^2}} \quad (2.101)$$

$$= \frac{\lambda_d + \lambda_1}{\lambda_d - \lambda_1} - \frac{2\sqrt{\lambda_1\lambda_d}}{\lambda_d - \lambda_1} = \frac{(\sqrt{\lambda_d} - \sqrt{\lambda_1})^2}{\lambda_d - \lambda_1} = \frac{\sqrt{\lambda_d} - \sqrt{\lambda_1}}{\sqrt{\lambda_d} + \sqrt{\lambda_1}}. \quad (2.102)$$

Hence,

$$\alpha = \frac{2 + 2\beta}{\lambda_1 + \lambda_d}, \quad (2.103)$$

$$\beta = \left(\frac{\sqrt{\lambda_d} - \sqrt{\lambda_1}}{\sqrt{\lambda_1} + \sqrt{\lambda_d}} \right)^2. \quad (2.104)$$

The convergence rate is thus:

$$\sqrt{\beta} = \frac{\sqrt{\frac{\lambda_d}{\lambda_1}} - 1}{\sqrt{\frac{\lambda_d}{\lambda_1}} + 1}. \quad (2.105)$$

Compare this to what we got with gradient descent:

$$\frac{\frac{\lambda_d}{\lambda_1} - 1}{\frac{\lambda_d}{\lambda_1} + 1}. \quad (2.106)$$

What have we achieved? We have square rooted the condition number. If we have a very large condition number, we now suffer much less from the slow convergence that gives because we take the additional square root.

Fig. 2.5 is the same plot as we saw before, but now with momentum. We actually converge and with less zig-zagging.

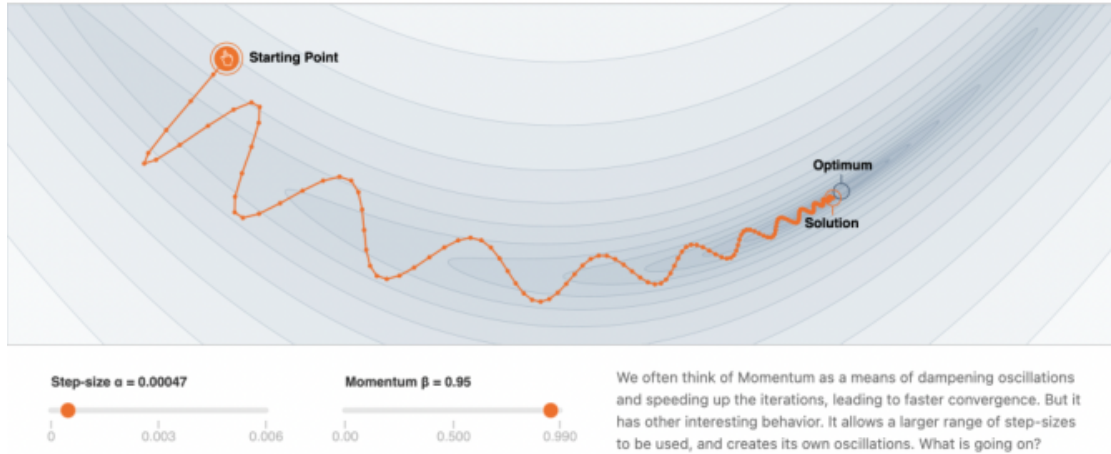


Figure 2.5: An objective function where-in we perform gradient descent this time with momentum! Picture is taken from <https://distill.pub/2017/momentum>. A much smoother convergence is observed.

2.4.2.2 Variations on momentum

There are tons of variations of the above algorithm. See this for a nice overview: <https://ruder.io/optimizing-gradient-descent/>. We will discuss Adam later on as it's a popular method used to train neural networks -it also relies on momentum-like dynamics.

2.4.3 Second-order methods - preconditioning

Newton's method. The idea of the gradient method was to take a local picture of the function we're trying to minimise to determine the direction to move to next. That local picture was a first-order Taylor approximation. We could consider adding somehow more higher-order information into the dynamics. Consider the second-order Taylor expansion:

$$f(\mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_k)^\top \nabla^2 f(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k). \quad (2.107)$$

The intuition of Newton's method is to assume that we are optimising a locally quadratic approximation of our function. Our goal is to find the minimum, i.e. have the gradient of the above approximation be zero:

$$\nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k) = 0. \quad (2.108)$$

Solving for $\mathbf{x}$:

$$\mathbf{x} = \mathbf{x}_k - (\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k), \quad (2.109)$$

where we assume that $(\nabla^2 f(\mathbf{x}_k))$ is invertible.

Perspective from preconditioning. As we saw in the last sections, the condition number of the matrix influences how ‘hard’ the optimisation is. Let’s look a bit deeper into this condition number. Consider the following function:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top S\mathbf{x}, \quad S = \begin{bmatrix} L & 0 \\ 0 & \mu \end{bmatrix}. \quad (2.110)$$

Let’s plot the level sets; see Fig. 2.6. The code is:

```

1 L = 1
2 mu = 1
3 x1 = np.arange(-100, 100, 1)
4 x2 = np.arange(-100, 100, 1)
5 z = np.zeros((x1.shape[0], x2.shape[0]))
6 for it1 in range(x1.shape[0]):
7     for it2 in range(x2.shape[0]):
8         z[it1][it2] = L/2*x1[it1]**2 + mu/2*x2[it2]**2
9 plt.contour(x1, x2, z)
10 plt.show()

```

For a higher condition number, the ellipses are more stretched out.

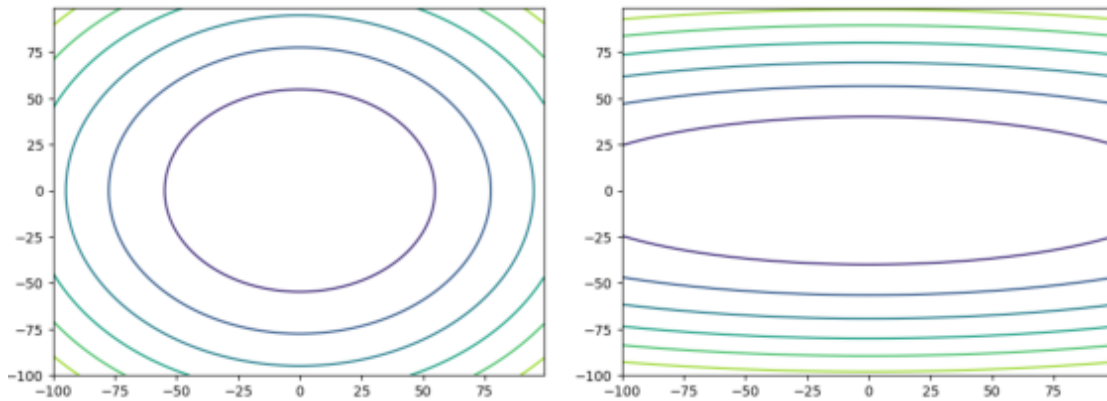


Figure 2.6: Level sets: $c = \frac{L}{2}x_1^2 + \frac{\mu}{2}x_2^2$. (Left) for $\mu = L = 1$ and (Right) for $\mu = 1, L = 10$.

The preconditioned gradient descent method can be seen as in essence ‘preconditioning’ the gradient to make it have a ‘nicer’ structure, making the ellipses above be more like circles. Specifically, consider the following objective function:

$$g(\mathbf{x}) = f(R\mathbf{x}). \quad (2.111)$$

Note that $\nabla g(\mathbf{x}) = R^\top \nabla_{R\mathbf{x}} f(R\mathbf{x})$. Gradient descent over this objective function becomes:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha R^\top \nabla_{R\mathbf{x}} f(R\mathbf{x}_k). \quad (2.112)$$

Define $\mathbf{w}_k = R\mathbf{x}_k$. Then,

$$\mathbf{w}_{k+1} = R\mathbf{x}_{k+1} = \mathbf{w}_k - \alpha RR^T \nabla_{\mathbf{w}} f(\mathbf{w}_k). \quad (2.113)$$

We are thus performing gradient descent, however with the gradient pre-multiplied by some positive definite matrix RR^T . Consider $f(R\mathbf{x}) = \frac{1}{2}\mathbf{x}^T R^T S R \mathbf{x}$ with S as above. Suppose now we take the preconditioner we used in the previous section: $RR^T = (\nabla^2 f(\mathbf{x}_k))^{-1} = S^{-1}$ with

$$R = \begin{bmatrix} 1/\sqrt{L} & 0 \\ 0 & 1/\sqrt{\mu} \end{bmatrix}. \quad (2.114)$$

Then,

$$f(R\mathbf{x}) = \frac{1}{2}\mathbf{x}^T R^T S R \mathbf{x} = \frac{1}{2}\mathbf{x}^T \mathbf{x}, \quad (2.115)$$

since $R^T S R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. We have thus made the conditioning nicer (condition number is now 1). In this case also, the iterate in Eq. (2.113) converges in a single step for $\alpha = 1$ (since $\nabla_{\mathbf{w}} f(\mathbf{w}_k) = S\mathbf{w}_k$).

Chapter 3: Introduction to deep learning

Reading list. This chapter is based on the following references:

- [Goodfellow et al., 2016], a reference textbook on deep learning.
- [Higham and Higham, 2019], a review paper on deep learning for mathematicians.

3.1 Introduction

Artificial intelligence has become a major field with applications in many research and industrial areas such as image recognition, natural language processing, speech recognition, or drug discovery. This evolution has been made possible due to the development of the computational power available in the computers such as graphic processing units (GPUs) or multi-core processors. Another factor is the increasing access to large data sets that allows one to train the artificial intelligence. Moreover, the development of accessible and high level Python libraries such as Tensorflow or Pytorch helped to democratise the field. One of the major successes of artificial intelligence goes back to the 90s with the defeat of the Chess player world champion Garry Kasparov against Deep Blue, an artificial intelligence created by IBM. A breakthrough has been done with the first victory of a Go playing system, AlphaGo developed by Google Deepmind, against a Go world champion. More recently, the development of large language models (LLMs) such as ChatGPT and Llama becomes prominent in the scientific community and the broader society. This was mainly achieved thanks to the development of a technique called deep learning, which we will explain in this chapter.

Artificial intelligence techniques are used to find structures in a data set in order to perform prediction or classification. In standard machine learning, the features used to represent the data have to be provided by the user. The aim of deep learning is to automatically construct features to describe the data using a neural network. The depth of the network determines the range of features available. Then, a deep neural network is able to identify complex patterns in a given set.

3.2 Fully connected neural networks

Let $f^* : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a nonlinear function, mapping $\mathbf{x} \in \mathbb{R}^n$ to $\mathbf{y} \in \mathbb{R}^m$, that we wish to approximate.

Definition 3.1 (Neural network). A feedforward network (or multilayer perceptron) is a succession of mathematical operations (such as additions, multiplications, composition of functions, ...) $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that aims to approximate f^* . Here, $\theta \in \mathbb{R}^s$ is a set of parameters used to parameterize the function f_θ . The number of parameters $s = |\theta|$ is referred to as the size of the network.

Example 3.2 (Image classification). One example of application could be classifying images of handwritten digits, as shown in Fig. 3.1. In this case, $n = 28 \times 28 = 784$ (the resolution of the images) and $m = 10$. There, we aim to output a vector of probability distribution over the 10 classes of digits. The function f^* is then a function that maps an image to a probability distribution over the 10 classes. The vector $\mathbf{y} = [0.9, 0.1, 0, 0, 0, 0, 0, 0, 0, 0]^\top$ would mean that the input image is most likely a 0.

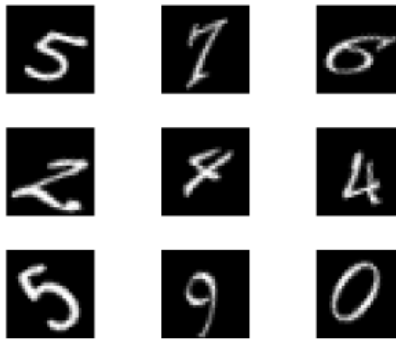


Figure 3.1: Handwritten digits from the MNIST data set were one of the first applications of deep learning to image classification tasks in the 1990s.

The aim of deep learning is to define good neural network architectures (i.e., specific structure of f_θ) and fit the model (or neural network) f_θ to f^* on a set of points $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ called the training dataset. The training process consists of finding the optimal parameters θ that minimize the difference between the output of the neural network and the true output. This is done by minimizing a loss function, which measures the difference between the output of the neural network

and the true output. The loss function can be defined as

$$\text{Loss}(\theta) = \sum_{i=1}^n \|f_{\theta}(\mathbf{x}_i) - \mathbf{y}_i\|^2 = \sum_{i=1}^n \|f_{\theta}(\mathbf{x}_i) - f^*(\mathbf{x}_i)\|^2,$$

where $\mathbf{x}_i$ is the i -th point in the training set. The optimization problem is then to find the parameters θ that minimize the loss function. This is done by using an optimization algorithm such as stochastic gradient descent (SGD) or Adam (see Section 3.3).

3.2.1 Learning the XOR operation

The XOR (exclusive or) operator is a function $f^* : \{0, 1\}^2 \rightarrow \{0, 1\}$ that takes two binary inputs x_1, x_2 , and returns 1 if exactly one of the binary values is equal to 1 and 0 otherwise. As an example, we have $f^*(0, 0) = 0$, $f^*(0, 1) = 1$, $f^*(1, 0) = 1$, and $f^*(1, 1) = 0$. Here, we do not try to generalise and train the neural network to learn the XOR operation of the four points: $\mathbf{x}_1 = [0, 0]^\top$, $\mathbf{x}_2 = [0, 1]^\top$, $\mathbf{x}_3 = [1, 0]^\top$, $\mathbf{x}_4 = [1, 1]^\top$.

We use a mean-squared error (MSE) loss function to measure the difference between the output of the neural network and the true output. This is usually not the approach recommended for binary classification. However, we use it here for simplicity. On the training dataset, the loss function is defined as

$$L(\theta) = \frac{1}{4} \sum_{i=1}^4 (f_{\theta}(\mathbf{x}_i) - f^*(\mathbf{x}_i))^2. \quad (3.1)$$

Let's choose a form for our model f_{θ} . Suppose we use a linear model of the form

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^\top \mathbf{w} + b,$$

where $\mathbf{w} \in \mathbb{R}^2$ and $b \in \mathbb{R}$. Here, $\theta = (\mathbf{w}, b)$. We then have:

$$f(\mathbf{x}, \theta) = f(\mathbf{x}; \mathbf{w}, b) = \begin{bmatrix} x_1 & x_2 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix}.$$

We define

$$X = \begin{bmatrix} \mathbf{x}_{11} & \mathbf{x}_{12} & 1 \\ \mathbf{x}_{21} & \mathbf{x}_{22} & 1 \\ \mathbf{x}_{31} & \mathbf{x}_{32} & 1 \\ \mathbf{x}_{41} & \mathbf{x}_{42} & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}, \quad \theta = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix}.$$

Then, the loss function can be written as

$$L(\theta) = \frac{1}{4} \|Y - X\theta\|^2 \quad (3.2)$$

$$= \frac{1}{4} (Y - X\theta)^\top (Y - X\theta) \quad (3.3)$$

$$= \frac{1}{4} (Y^\top Y - Y^\top X\theta - \theta^\top X^\top Y + \theta^\top X^\top X\theta). \quad (3.4)$$

Solving the gradient equation to find the minimum of the loss function as in Example 1.7 yields:

$$\nabla_\theta L(\theta) = \frac{1}{4} (-2X^\top Y + 2X^\top X\theta).$$

Setting it to 0, we obtain

$$\nabla_\theta L(\theta) = 0 \iff \theta^* = (X^\top X)^{-1} X^\top Y.$$

```

1 import numpy as np
2
3 X = np.array([[0, 0, 1], [0, 1, 1], [1, 0, 1], [1, 1, 1]])
4 Y = np.array([[0], [1], [1], [0]])
5 theta = np.linalg.lstsq(X, Y)[0]
6
7 >>> theta
8 array([[ -2.70264086e-16],
9         [ -2.22044605e-16],
10        [ 5.00000000e-01]])

```

We find that $\theta^* = [0, 0, 0.5]^\top$, hence the linear model always outputs 0.5 for any input, and is unable to fit the training dataset. The residual (training error) is equal to $\frac{1}{4} (Y - X\theta)^\top (Y - X\theta) = 1/4$ as we are always 0.5 “off” the true value and $0.5^2 = 0.25$. This is a simple example of a problem that cannot be solved by a linear model.

We are going to make our model class more complex and introduce a feedforward neural network with one hidden layer and two hidden units (see Fig. 3.2). The first layer of the neural network is a function $\mathbf{h} = f^{(1)}(\mathbf{x}; \mathbf{W}, \mathbf{c})$, where $\mathbf{h} = [h_1, h_2]^\top$, and the second layer takes as input $\mathbf{h}$ and outputs the prediction $\hat{y} = f^{(2)}(\mathbf{h}; \mathbf{w}, b)$. The model is then defined by the composition of the two functions:

$$\hat{y} = f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = f^{(2)}(f^{(1)}(\mathbf{x}; \mathbf{W}, \mathbf{c}); \mathbf{w}, b).$$

Let’s ignore the terms $\mathbf{c}$ and b for now. How do we choose $f^{(1)}$? If it is a linear function of the form

$$f^{(1)}(\mathbf{x}, \mathbf{W}) = \mathbf{W}^\top \mathbf{x},$$

where $\mathbf{W} \in \mathbb{R}^{2 \times 2}$, and

$$f^{(2)}(\mathbf{h}, \mathbf{w}) = \mathbf{h}^\top \mathbf{w},$$

then the model is still a linear model as it can be rewritten as

$$f(\mathbf{x}; \mathbf{W}, \mathbf{w}) = \mathbf{x}^\top \mathbf{W} \mathbf{w}.$$

We need to introduce some nonlinearity in the first layer to make the model more complex.

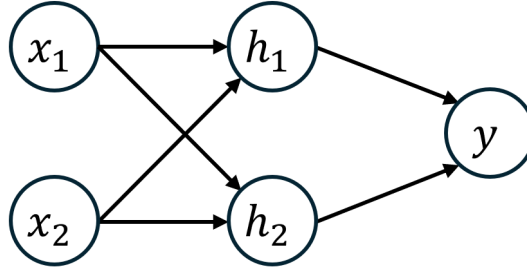


Figure 3.2: A feedforward neural network with one hidden layer and two hidden units.

We use a nonlinear function (also called activation function in deep learning) $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, which is applied element-wise to the output of the first layer:

$$\mathbf{h} = f^{(1)}(\mathbf{x}, \mathbf{W}, \mathbf{c}) = \sigma(\mathbf{W}^\top \mathbf{x} + \mathbf{c}) = \begin{bmatrix} \sigma(\mathbf{w}_1^\top \mathbf{x} + c_1) \\ \sigma(\mathbf{w}_2^\top \mathbf{x} + c_2) \end{bmatrix},$$

where $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2] \in \mathbb{R}^{2 \times 2}$ and $\mathbf{c} = [c_1, c_2]^\top$. The most commonly used activation function is the rectified linear unit (ReLU) defined by $\sigma(x) = \max(x, 0)$ (see Fig. 3.3). One reason is that it is computationally fast to evaluate and to differentiate. Many other choices are possible (even activation functions that depend themselves on trainable parameters!). The complete network is then defined as

$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\} + b.$$

We can “guess” a solution to the XOR problem by choosing the following parameters:

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b = 0. \quad (3.5)$$

Let’s see how the network processes the input data. We define $\mathbf{X}$ to be the input matrix containing the four concatenated training datapoints as

$$\mathbf{X} = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}.$$

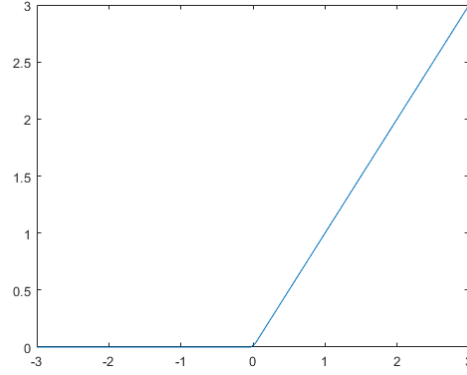


Figure 3.3: The ReLU activation function defined as $\sigma(x) = \max\{x, 0\}$.

The network operations are applied component-wise across the columns of $\mathbf{X}$, also referred to as the “batch” dimension. We begin by multiplying the input matrix by the weight matrix $\mathbf{W}$:

$$\mathbf{W}^\top \mathbf{X} = \begin{bmatrix} 0 & 1 & 1 & 2 \\ 0 & 1 & 1 & 2 \end{bmatrix}.$$

We now add the bias vector $\mathbf{c}$ to each column:

$$\mathbf{W}^\top \mathbf{X} + \mathbf{c} = \begin{bmatrix} 0 & 1 & 1 & 2 \\ -1 & 0 & 0 & 1 \end{bmatrix}.$$

To compute the values of $\mathbf{h}$, we apply the ReLU activation function to each element of the matrix:

$$\max\{0, \mathbf{W}^\top \mathbf{X} + \mathbf{c}\} = \begin{bmatrix} 0 & 1 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

We multiply the output of the ReLU activation function by the weight vector $\mathbf{w}$ and add the bias b :

$$\mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{X} + \mathbf{c}\} + b = \begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}.$$

The output of the network is then $[0, 1, 1, 0]$, which is the correct output for the XOR operation. The network has learned the XOR operation on the training dataset. In a real task, one cannot guess the parameters of the network (which may have millions or even billions of parameters for large language models) and the training process is done by minimizing the loss function using an optimization (gradient descent-based) algorithm.

3.2.2 A classification example

We generalise the XOR example on a classification task to explain how deep learning works and follow the approach of [Higham and Higham, 2019]. Assume that we have access to n points (x_i, y_i) in $\mathbb{R}^2$ with label 0 or 1 (as in Fig. 3.4 (a)) and would like to fit a function $f : \mathbb{R}^2 \rightarrow \{0, 1\}$ to this data set in order to classify further points.

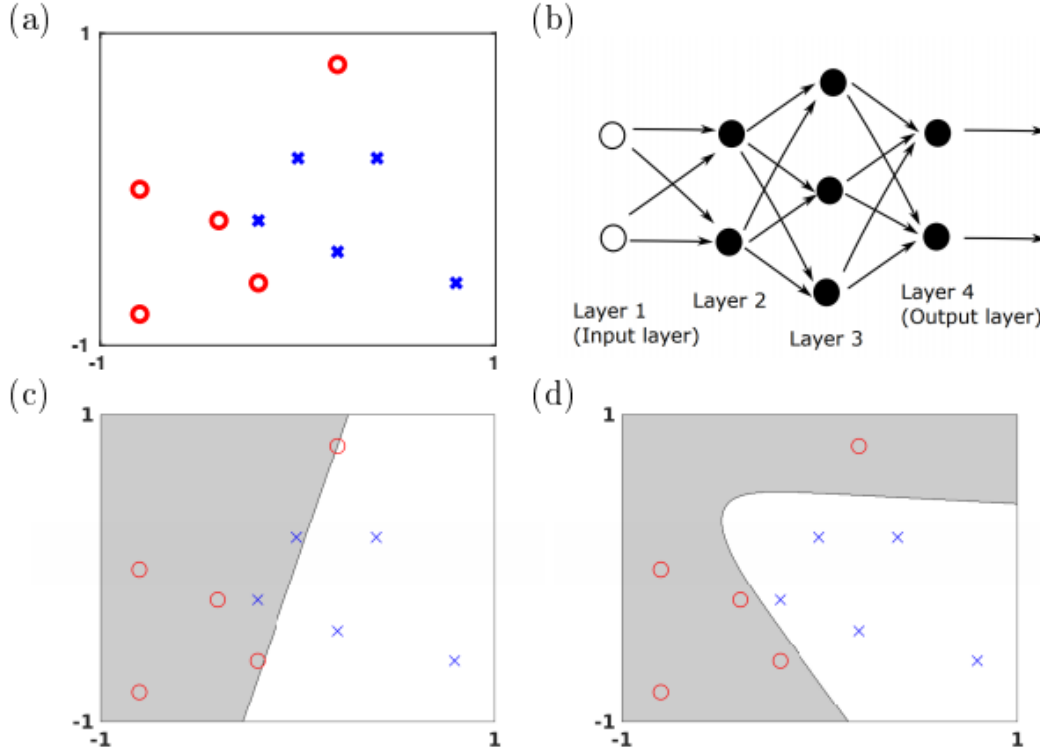


Figure 3.4: (a) Points in $\mathbb{R}^2$, circles belong in class 0 while crosses lie in class 1. (b) Structure of the neural network, reproduced from [Higham and Higham, 2019]. (c) Classification using regression. (d) Classification using the neural network represented in (b).

To achieve this we use a network, which is defined by a collection of layers and nodes within each layer. Each layer consists of a linear and a nonlinear operation. As an example, we consider a network with four layers to describe our data set (see Fig. 3.4 (b)). The dimension of the input layer, i.e. the number of nodes, is two since the points are two dimensional. Let $x \in \mathbb{R}^2$, then the output of the second layer is

$$f^1(x) = \sigma(W^1x + b^1).$$

Here, W^1 denotes a two-by-two matrix called weight matrix and b^1 is a two by one vector called bias. Neural networks are also based on a nonlinear activation functions σ , whose roles are to add nonlinearity in order to capture complex features

in the original data set.

Fig. 3.4 (c) shows the output of the neural network after training without activation functions. We observe that the data is described by a linear function, which is equivalent to a linear regression. Linear separation is clearly not sufficient to classify complicated situations. Instead, we use the sigmoid function $\sigma(x) = 1/(1 + e^{-x})$, which is applied on each component of the vector $W^1x + b^1$. Another example of activation function is the linear rectifier unit (ReLU) defined by $\sigma(x) = \max(x, 0)$, commonly used to avoid vanishing gradient issues caused by the sigmoid function. The operation is also repeated for the next layers with corresponding weight matrices $W^2 \in \mathbb{R}^{2 \times 3}$, $W^3 \in \mathbb{R}^{3 \times 2}$ and bias vectors $b^2 \in \mathbb{R}^3$, $b^3 \in \mathbb{R}^2$. At the end, the function representing the network is the composition of the different operations performed at each layer:

$$f(x) = f^3 \circ f^2 \circ f^1(x) = \sigma(W^3 \sigma(W^2 \sigma(W^1 x + b^1) + b^2) + b^3).$$

Then, $f(x)$ is a vector in $\mathbb{R}^2$, which we use to classify x in the category 0 or 1. We classify x as 0 if the first component of $f(x)$ is greater than the second, i.e. $f_1(x) > f_2(x)$, and 1 otherwise. The weights and biases are determined by an optimisation algorithm called steepest gradient descent.

We then introduce the objective function Cost, also known as the cost function, which denotes the Euclidean error between the output of the network $f(x_i)$ and the expected output y_i , corresponding to the correct classification of the point. The cost function is defined by

$$\text{Cost}(W^1, W^2, W^3, b^1, b^2, b^3) = \frac{1}{n} \sum_{i=1}^n \|y_i - f(x_i)\|_2^2.$$

Assume now that Cost is a function of a vector $p \in \mathbb{R}^s$, where p encodes the information of the weight matrices and bias vectors and $s = 4 + 6 + 6 + 2 + 3 + 2 = 23$ is the total number of parameters of the network. The weight and bias parameters are updated in the training process by minimizing the Cost function using gradient descent:

$$p = p - \eta \nabla \text{Cost}(p),$$

where η denotes the step size is called a *learning rate*. In practice however, it is common to use stochastic gradient descent instead, which consists of minimizing the Cost function over a random subset of the training set to decrease the computational time of training. Fig. 3.4 (d) shows the output of the neural network after the training period.

3.3 Optimizing neural networks

In this section, we detail some key steps towards selecting the cost function and the optimization algorithm to train a neural network.

3.3.1 Cost functions

As we saw in Section 1.6, one can rationalize the choice of the cost function by considering the maximum likelihood estimation. Here, our model defines a distribution $p(y|x, \theta)$, where y is the output of the neural network, x is the input, and θ are the parameters of the network. We can then use the *cross-entropy* between the training data and the model's prediction as a cost function, i.e., the negative log-likelihood of the training data:

$$J(\theta) = -\mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(y|x) = -\frac{1}{m} \sum_{i=1}^m \log p_{\text{model}}(y_i|x_i).$$

Definition 3.3 (Cross-entropy). The cross entropy of the distribution q relative to a distribution p over a given dataset is defined as

$$H(p, q) = -\mathbb{E}_{x \sim p}[\log q]. \quad (3.6)$$

For discrete distributions with the same support $\mathcal{X}$:

$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log q(x).$$

For continuous distributions:

$$H(p, q) = -\int_{\mathcal{X}} p(x) \log q(x) dx.$$

If we make the assumption that the output model is Gaussian with mean $f(x; \theta)$ and variable σ^2 , i.e., $p_{\text{model}}(y|x) = \frac{1}{\sqrt{2\pi}} \exp(-\|y - f(x; \theta)\|^2 / (2\sigma^2))$, then the cost function that we want to *maximize* becomes

$$J(\theta) = \frac{1}{2} \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \|y - f(x; \theta)\|^2 + \text{const.} \quad (3.7)$$

$$= -\frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_i - f(\mathbf{x}_i; \theta)\|^2 + \text{const.} \quad (3.8)$$

Note that this formula works *independently* of the model used to predict the output, i.e., not just a linear model but also a very complex neural network. The cost

function is then the mean squared error (MSE) between the output of the neural network and the true output. This is the most common cost function used in regression tasks.

One key advantage of this approach is that specifying a model $p(y|x)$ automatically determines a cost function $\log p(y|x)$.

3.3.1.1 Conditional statistics and mean absolute error (MAE)

Instead of learning the full probability distribution $p(y|x; \theta)$, we might be interested in learning one conditional statistic of y given x . As an example, one might have a predictor $f(x; \theta)$ that we wish to use to predict the mean of y .

Since neural networks can be used to approximate any function, instead of parameterizing the cost function J by a set of parameters θ , we can also view it as a functional $J(f)$ acting on functions $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and outputting real numbers. These functions are the neural networks that we are trying to optimize.

Then, we can design our cost function to have a minimum at the desired conditional statistic. For example, if we want to learn the conditional mean $\mathbb{E}[y|x]$. Then, one can show (see [Goodfellow et al., 2016, Sec. 19.4.2]) that solving the optimization problem

$$f^* = \operatorname{argmin}_f \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}} \|y - f(x)\|^2$$

yields

$$f^* = \mathbb{E}_{\mathbf{y} \sim p_{\text{data}}(y|x)}[y].$$

Using a different cost function would give rise to a different statistic that one wishes to learn. As an example,

$$f^* = \operatorname{argmin}_f \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}} \|y - f(x)\|_1$$

yields a function that predicts the median of y given x . This cost function is called the mean absolute error (MAE) and is defined as

$$\text{MAE} = \sum_{i=1}^m \|\mathbf{y}_i - f(\mathbf{x}_i)\|_1.$$

The MSE and MAE are good cost functions for regression problems but are not ideal for classification tasks, where the outputs are discrete. In general, the choice of the cost function is highly related to the choice of the output layer of the neural network.

3.3.1.2 Output layers

Linear units. One common output layer is simply an affine transformation with no nonlinearity. Given a feature vector $\mathbf{h}$, the linear output layer computes

$$\hat{\mathbf{y}} = \mathbf{W}^\top \mathbf{h} + \mathbf{b},$$

where $\mathbf{W}$ is a matrix of weights and $\mathbf{b}$ is a bias vector. These output layers are used to produce the mean of a conditional distribution:

$$p(y|x) = \mathcal{N}(y, \hat{\mathbf{y}}, \mathbf{I}),$$

where $y \mapsto \mathcal{N}(y, \hat{\mathbf{y}}, I)$ denotes the probability distribution function of the multivariate Gaussian with mean $\hat{\mathbf{y}}$ and identity covariance matrix.

Sigmoid units. A common task involves predicting the value of a binary variable $y \in \{0, 1\}$. This involves classification problems with two classes. The maximum likelihood approach consists of defining a Bernoulli distribution over y conditioned on $\mathbf{x}$. That is,

$$P(y = 1|\mathbf{x}) = p, \quad P(y = 0|\mathbf{x}) = 1 - p.$$

This means that the neural network needs to predict $P(y = 1|\mathbf{x})$. However, this number must be constrained to the interval $[0, 1]$ to be a valid probability. One could for instance threshold the value of a linear unit as

$$P(y = 1|\mathbf{x}) = \max \{0, \min \{1, \mathbf{w}^\top \mathbf{h} + b\}\}.$$

This is however not a good choice for gradient-based optimization as the gradient of the model with respect to the parameters is 0 outside $[0, 1]$, which means that the model could become stuck there. Instead, one considers a sigmoid output unit as

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{h} + b), \tag{3.9}$$

where σ is the sigmoid function defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}.$$

We see in Fig. 3.5 that this choice constraints the output of the neural network to the interval $[0, 1]$. Moreover, the gradient of this function is non-zero everywhere, which makes it a good choice for gradient-based optimization. One can think of this unit as first applying a linear unit $z = \mathbf{w}^\top \mathbf{h} + b$, then converting the output to a probability using the sigmoid function.

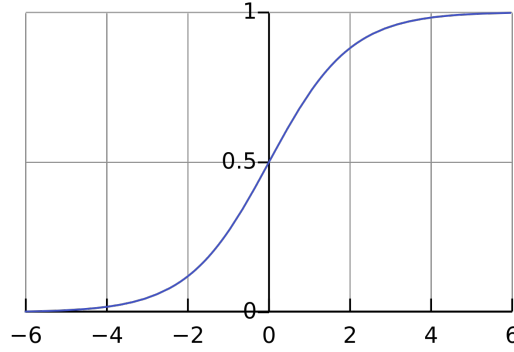


Figure 3.5: The sigmoid function $\sigma(x) = 1/(1 + e^{-x})$.

Then, the cost function is the cross-entropy between the true distribution and the predicted distribution:

$$J(\theta) = -\mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(y|x).$$

However, we assumed that the output of the neural network is a Bernoulli distribution, i.e.,

$$p_{\text{model}}(y|\mathbf{x}_i) = \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{1-y_i}.$$

Hence, the network predicts the true class y_i (either 0 or 1) with probability $\hat{y}_i$ and the other class with probability $1 - \hat{y}_i$. The cost function is then

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i). \quad (3.10)$$

Softmax units. The softmax function is a generalization of the sigmoid function to multiple classes. We then generalize the previous layer to a discrete variable with n values. We are now going to predict a vector $\hat{\mathbf{y}} \in \mathbb{R}^n$ such that

$$\hat{y}_j = P(y = j|\mathbf{x}).$$

We require that the output of the network is a probability distribution, i.e., that the sum of the components of $\hat{\mathbf{y}}$ is equal to 1:

$$\sum_{j=0}^{n-1} P(y = j|\mathbf{x}) = 1, \quad P(y = j|\mathbf{x}) \in [0, 1].$$

This defines a *categorical distribution* over the classes (also called generalized Bernoulli or multinoulli distribution). We assume that our supervised output $\mathbf{y}_i$ is given in the form of a categorical variable, i.e., $y_{i,j} = 1$ if the input $\mathbf{x}_i$ belongs to class j and 0 otherwise.

We first have a linear layer to output un-normalized log probabilities:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b}.$$

Then, we apply the softmax function to normalize the output:

$$\text{softmax}_j = \frac{\exp(z_j)}{\sum_{j'=1}^n \exp(z_{j'})}, \quad 1 \leq j \leq n.$$

We have

$$p_{\text{model}}(\mathbf{y}|\mathbf{x}_i) = \prod_{j=0}^{n-1} \hat{y}_{i,j}^{y_{i,j}},$$

where the true label vector $\mathbf{y}_i$ takes value 1 at exactly one index in $\{0, \dots, n-1\}$. This function selects the output probability corresponding to the true class of the input $\mathbf{x}_i$. The cost function is then

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=0}^{n-1} y_{i,j} \log \hat{y}_{i,j}. \quad (3.11)$$

3.3.2 Backpropagation algorithm

We will use a gradient descent algorithm to train the neural network, i.e., to minimize the cost function $J(\theta)$ with respect to the trainable parameters θ of the network:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} J(\theta) = \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} [l(\theta, \mathbf{x}, \mathbf{y})] \approx \frac{1}{N} \sum_{i=1}^N l(\theta, \mathbf{x}_i, \mathbf{y}_i).$$

As a reminder, the gradient descent algorithm updates the parameters θ in the direction of the negative gradient of the cost function:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta),$$

where η is the learning rate. The gradient of the cost function with respect to the parameters of the network is computed using the *backpropagation algorithm*.

Backpropagation is a method for computing the gradient of the cost function with respect to the parameters of the network. It is based on the chain rule of calculus and is used to compute the gradient of the cost function with respect to the parameters of the network. The algorithm is divided into two steps: the forward pass and the backward pass:

- During the forward pass, one builds a computational graph of the operations performed by the network to compute the output of the network (see Fig. 3.6). This is the same as computing the output of the network for a given input.

- During the backward pass, one computes the gradient of the cost function with respect to the parameters of the network using the chain rule of calculus. This is done by propagating the gradient of the cost function backwards through the computational graph.

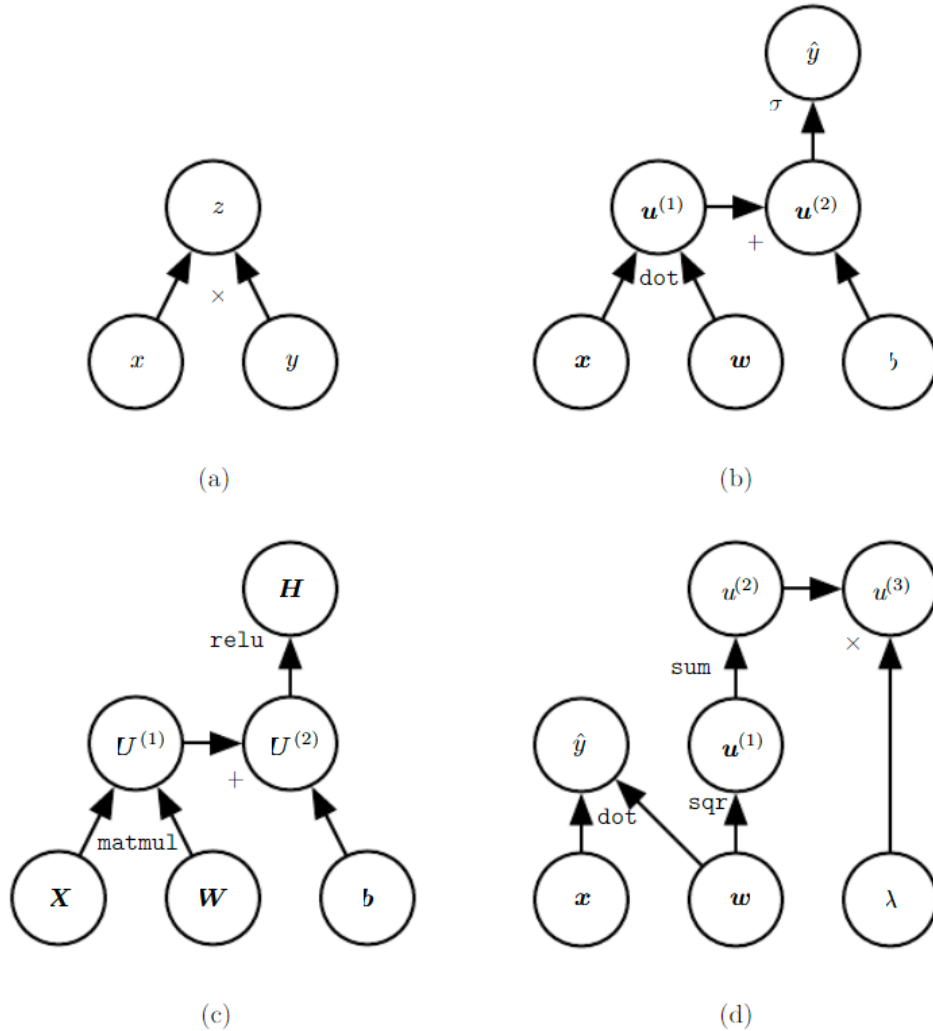


Figure 3.6: Examples of computational graphs. Figure reproduced from [Goodfellow et al., 2016, Fig. 6.8].

Remark 3.4 (Chain rules). Let $f, g : \mathbb{R} \rightarrow \mathbb{R}$ be two differentiable functions. Suppose that $y = g(x)$ and $z = f(g(x)) = f(y)$. Then, the chain rules states that

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}.$$

If we generalize this to higher dimensions with $g : \mathbb{R}^m \rightarrow \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$, we

have

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i},$$

which in vector notations is equivalent to

$$\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^\top \nabla_{\mathbf{x}} \mathbf{y},$$

where $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ is the Jacobian matrix of g .

Algorithm 3.1 Forward propagation through a deep neural network

Input: Network depth l

Input: Weight matrices $\mathbf{W}^{(i)}$ of the model for $1 \leq i \leq l$

Input: Bias vectors $\mathbf{b}^{(i)}$ of the model for $1 \leq i \leq l$

Input: Activation function σ

Input: $\mathbf{x}$, input vector of the network

Input: $\mathbf{y}$, target output of the network

```

1:  $\mathbf{h}^{(0)} = \mathbf{x}$ 
2: for  $i = 1$  to  $l$  do
3:    $\mathbf{a}^{(i)} = \mathbf{W}^{(i)} \mathbf{h}^{(i-1)} + \mathbf{b}^{(i)}$ 
4:    $\mathbf{h}^{(i)} = \sigma(\mathbf{a}^{(i)})$ 
5: end for
6:  $\hat{\mathbf{y}} = \mathbf{h}^{(l)}$ 
7: Compute the cost function  $J = L(\mathbf{y}, \hat{\mathbf{y}})$ 
```

3.3.3 Stochastic gradient descent

Most deep learning models are trained using a variant of the stochastic gradient descent algorithm (SGD) described in Algorithm 3.3. SGD is an extension of the gradient descent algorithm (seen in Section 2.3) designed to reduce its computational cost when the training dataset size is large. Hence, the cost function often decomposes with respect to the training examples as

$$J(\theta) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(f(\mathbf{x}, \theta), y) = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}_i; \theta), y_i),$$

The gradient of the cost function is then

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i),$$

Algorithm 3.2 Backward computation for the neural network in Algorithm 3.1.

- 1: After the forward computation, compute the gradient of the cost function with respect to the output of the network: $\mathbf{g} \leftarrow \nabla_{\hat{\mathbf{y}}} J = \nabla_{\hat{\mathbf{y}}} L(\hat{\mathbf{y}}, \mathbf{y})$
- 2: **for** $i = l, l - 1$ to 1 **do**
- 3: Convert the gradient of the layer's output into a gradient of the pre-activation function: $\mathbf{g} \leftarrow \nabla_{\mathbf{a}^{(i)}} J = \mathbf{g} \odot \sigma'(\mathbf{a}^{(k)})$
- 4: Compute gradients on weights and biases:

$$\nabla_{\mathbf{W}^{(i)}} J = \mathbf{g} \mathbf{h}^{(i-1)\top}, \quad \nabla_{\mathbf{b}^{(i)}} J = \mathbf{g}.$$

- 5: Propagate the gradients to the next level:

$$\mathbf{g} \leftarrow \nabla_{\mathbf{h}^{(i-1)}} J = \mathbf{W}^{(i)\top} \mathbf{g}.$$

- 6: **end for**

and requires $\mathcal{O}(m)$ gradient computations, which can be very expensive for large datasets, or difficult to compute if the dataset is too large to fit in GPU memory (one wants to avoid communication costs between GPU memory and RAM in general).

Since taking the gradient is equivalent to computing an *expectation*, one can approximate the gradient by sampling a *minibatch* of examples $\mathbb{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_{m'}\}$ uniformly from the dataset. The minibatch size m' is typically chosen to be a power of 2 and is a hyperparameter of the model (usually from 1 to a few hundreds), optimized so that the gradient computation fits into GPU memory. We then approximate the gradient of the cost function by

$$\nabla_{\theta} J(\theta) \approx \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i).$$

Usually, mini-batches are sampled uniformly from the training dataset *without replacement*. This means that the same example is not sampled twice in the same minibatch. An iteration across the entire training dataset is called an *epoch*. The number of iterations is a hyperparameter of the model and is usually chosen to be large enough to ensure convergence of the optimization algorithm.

A key parameter in SGD is the choice of the learning rate. If the learning rate is too large, the optimization algorithm might diverge, and if it is too small, the optimization algorithm might take too long to converge. In practice, it is necessary to gradually decrease the learning rate over the number of epochs to ensure convergence. This is called *learning rate scheduling*. One reason is that

Algorithm 3.3 Stochastic gradient descent.

Input: Learning rate ϵ_k , minibatch size m' , number of epochs N_{epoch} **Input:** Initial parameters θ

- 1: **for** epoch $k = 1$ to N_{epoch} **do**
- 2: **for** $j = 1$ to $\lceil m/m' \rceil$ **do**
- 3: Sample a minibatch of m' examples without replacement $\mathbb{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_{m'}\}$ from the training dataset with corresponding targets y_i .
- 4: Compute the gradient of the cost function on the minibatch:

$$\hat{\mathbf{g}} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i).$$

- 5: Update the parameters of the network:

$$\theta = \theta - \epsilon_k \hat{\mathbf{g}}.$$

- 6: **end for**
 - 7: **end for**
-

the SGD gradient estimator introduces a source of noise in the approximation of the gradient and does not vanish when we arrive at a minimum of the true cost function. One condition to guarantee convergence of the SGD algorithm is

$$\sum_{k=1}^{\infty} \epsilon_k = +\infty, \quad \text{and} \quad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty.$$

Note that this guarantees convergence of the SGD algorithm but not necessarily to the global (or even local) minimum of the cost function.

In practice, one can decrease the learning rate linearly until some iteration τ , such as

$$\epsilon_k = (1 - \alpha)\epsilon_0 + \alpha\epsilon_{\tau},$$

where $\alpha = \frac{k}{\tau}$ and ϵ_0 and ϵ_{τ} are the initial and final learning rates, respectively. After iteration τ , one can leave the learning rate constant.

3.3.4 Adam

Adam [Kingma and Ba, 2014] is one of the most popular optimization algorithms used to train deep learning models. It is an adaptive learning rate algorithm that combines the advantages of two other optimization algorithms: AdaGrad and RMSProp. The algorithm is based on the first and second moments of the gradient,

which are used to adapt the learning rate of the optimization algorithm. We begin by introducing the method of momentum, and adaptive learning rates algorithms, before detailing the Adam algorithm.

3.3.4.1 Momentum

The momentum algorithm (see Algorithm 3.4) consists of accumulating an exponentially decaying average of past gradients to speed up the convergence of the optimization algorithm. The update rule of the momentum algorithm is

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\theta} \left(\frac{1}{m'} \sum_{i=1}^{m'} L(f(\mathbf{x}_i; \theta), y_i) \right), \quad (3.12)$$

$$\theta \leftarrow \theta + \mathbf{v}, \quad (3.13)$$

where $\mathbf{v}_k$ is the velocity of the optimization algorithm, α is the momentum parameter, and ϵ_k is the learning rate. The hyperparameter $\alpha \in [0, 1)$ determines the exponential decay rate of the past gradient accumulation.

Before, the size of the step was determined by the norm of the gradient $\mathbf{g}$ multiplied by the learning rate: $\epsilon \|\mathbf{g}\|$. Now, it depends on how large and how aligned the previous gradients are. If the momentum algorithm always observes the same gradient $\mathbf{g}$, it will accelerate in the direction of $-\mathbf{g}$ until reaching a maximum velocity with step size

$$\frac{\epsilon \|\mathbf{g}\|}{1 - \alpha}.$$

Hence, if we denote by v_k the velocity at iteration k , we have

$$v_{k+1} = \alpha v_k - \epsilon \mathbf{g}, \quad v_0 = 0.$$

We recognize an arithmetico-geometric sequence, whose fixed point is

$$v_{\infty} = -\frac{\epsilon \mathbf{g}}{1 - \alpha}.$$

Hence,

$$v_{k+1} - v_{\infty} = \alpha(v_k - v_{\infty}).$$

Therefore,

$$v_k - v_{\infty} = \alpha^k(v_0 - v_{\infty}).$$

Finally, we have

$$v_k = -\frac{\epsilon \mathbf{g}}{1 - \alpha} + \alpha^k \frac{\epsilon \mathbf{g}}{1 - \alpha}.$$

In the limit, the step size converges to

$$\|v_{\infty}\| = \frac{\epsilon \|\mathbf{g}\|}{1 - \alpha}.$$

Algorithm 3.4 SGD with momentum.

Input: Learning rate ϵ_k , minibatch size m' , number of epochs N_{epoch} , momentum parameter α

Input: Initial parameters θ

1: **for** epoch $k = 1$ to N_{epoch} **do**

2: **for** $j = 1$ to $\lceil m/m' \rceil$ **do**

3: Sample a minibatch of m' examples without replacement $\mathbb{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_{m'}\}$ from the training dataset with corresponding targets y_i .

4: Compute the gradient of the cost function on the minibatch:

$$\hat{\mathbf{g}} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i).$$

5: Update the velocity of the network:

$$\mathbf{v} = \alpha \mathbf{v} - \epsilon_k \hat{\mathbf{g}}.$$

6: Update the parameters of the network:

$$\theta = \theta + \mathbf{v}.$$

7: **end for**

8: **end for**

3.3.4.2 Nesterov momentum

[Sutskever et al., 2013] introduced the Nesterov momentum algorithm, which is a modification of the momentum algorithm. The update rule of the Nesterov momentum algorithm is

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\theta} \left(\frac{1}{m'} \sum_{i=1}^{m'} L(f(\mathbf{x}_i; \theta + \alpha \mathbf{v}), y_i) \right), \quad (3.14)$$

$$\theta \leftarrow \theta + \mathbf{v}. \quad (3.15)$$

The difference lies at the point θ where the gradients are evaluated. With Nesterov momentum, the gradients are evaluated at the point $\theta + \alpha \mathbf{v}$, after the current velocity is applied.

3.3.4.3 Adaptive learning rates algorithms

The AdaGrad algorithm adapts the learning rate of the model parameters individually by scaling them inversely proportional to the square root of the sum of the squares of the gradients. The update rule of the AdaGrad algorithm is provided in Algorithm 3.5.

The RMSProp algorithm performs better than AdaGrad for nonconvex loss function by changing the gradient accumulation into an exponentially decaying average. The update rule of the RMSProp algorithm is provided in Algorithm 3.6.

3.3.4.4 Adam algorithm

Adam (“Adaptive Moments”) is one of the most popular algorithms in deep learning for training deep neural networks. It can be seen as a variant on the RMSProp algorithm with momentum. The update rule of the Adam algorithm is provided in Algorithm 3.7. Here, momentum is incorporated as an estimate of the first-order moment of the gradient.

3.4 Overview of architectures and operations

In this section, we provide a brief tour of the main neural network architectures and common layers used when designing deep learning models. We will cover the following topics: feedforward neural networks, activation functions, convolutional neural networks (CNNs).

Algorithm 3.5 AdaGrad algorithm.

Input: Global learning rate ϵ , minibatch size m' , number of epochs N_{epoch} , small constant $\delta \approx 10^{-7}$ for numerical stability

Input: Initial parameters θ

- 1: Initialize gradient accumulation variable $\mathbf{r} = 0$
- 2: **for** epoch $k = 1$ to N_{epoch} **do**
- 3: **for** $j = 1$ to $\lceil m/m' \rceil$ **do**
- 4: Sample a minibatch of m' examples without replacement $\mathbb{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_{m'}\}$ from the training dataset with corresponding targets y_i .
- 5: Compute the gradient of the cost function on the minibatch:

$$\hat{\mathbf{g}} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i).$$

- 6: Accumulate the squared gradient:

$$\mathbf{r} = \mathbf{r} + \hat{\mathbf{g}} \odot \hat{\mathbf{g}}.$$

- 7: Compute element-wise update of the parameters of the network:

$$\Delta\theta = -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}.$$

- 8: Update the parameters of the network:

$$\theta = \theta + \Delta\theta.$$

- 9: **end for**
 - 10: **end for**
-

Algorithm 3.6 RMSProp.

Input: Global learning rate ϵ , minibatch size m' , number of epochs N_{epoch} , decay rate ρ , small constant $\delta \approx 10^{-6}$ for numerical stability

Input: Initial parameters θ

- 1: **for** epoch $k = 1$ to N_{epoch} **do**
- 2: **for** $j = 1$ to $\lceil m/m' \rceil$ **do**
- 3: Sample a minibatch of m' examples without replacement $\mathbb{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_{m'}\}$ from the training dataset with corresponding targets y_i .
- 4: Compute the gradient of the cost function on the minibatch:

$$\hat{\mathbf{g}} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i).$$

- 5: Accumulate the squared gradient:

$$\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}.$$

- 6: Compute element-wise parameter update

$$\Delta \theta = -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}.$$

- 7: Update the parameters of the network:

$$\theta = \theta + \Delta \theta.$$

- 8: **end for**
 - 9: **end for**
-

Algorithm 3.7 Adam algorithm.

Input: learning rate $\epsilon = 10^{-3}$, exponential decay rates $\beta_1 = 0.9$, $\beta_2 = 0.999$ for moments, small constant $\delta \approx 10^{-8}$ for numerical stability

Input: Initial parameters θ

1: Initialize first and second moment variables $\mathbf{s} = 0$, $\mathbf{r} = 0$

2: Initialize time step $t = 0$

3: **for** epoch $k = 1$ to N_{epoch} **do**

4: **for** $j = 1$ to $\lceil m/m' \rceil$ **do**

5: Sample a minibatch of m' examples without replacement $\mathbb{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_{m'}\}$ from the training dataset with corresponding targets y_i .

6: Compute the gradient of the cost function on the minibatch:

$$\hat{\mathbf{g}} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i).$$

7: Update the time step: $t \leftarrow t + 1$.

8: Update biased first moment estimate: $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1) \hat{\mathbf{g}}$.

9: Update biased second moment estimate: $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$.

10: Correct the bias in the first moment: $\hat{\mathbf{s}} = \frac{\mathbf{s}}{1 - \beta_1^t}$.

11: Correct the bias in the second moment: $\hat{\mathbf{r}} = \frac{\mathbf{r}}{1 - \beta_2^t}$.

12: Compute element-wise parameter update: $\Delta\theta = -\epsilon \frac{\hat{\mathbf{s}}}{\delta + \sqrt{\hat{\mathbf{r}}}}$.

13: Update the parameters of the network:

$$\theta = \theta + \Delta\theta.$$

14: **end for**

15: **end for**

3.4.1 Activation functions

In Section 3.2, we saw one of the most basic deep learning architectures, which is called a fully connected neural networks or multi-layer perceptrons (MLPs). The output of each layer is computed as a succession of linear transformation, and pointwise nonlinearities called activation functions. The choice of the activation function is crucial in the design of the neural network, as it determines the expressiveness of the model and can lead to faster convergence of the optimization algorithm (better behaviour of the gradient of the activation function).

While researchers used to employ the sigmoid function, defined as $\sigma(x) = 1/(1 + e^{-x})$, or tanh function as activation functions to guarantee smoothness of the outputs of the network and boundedness of the layers' outputs, they suffer from the vanishing gradient problem. This problem arises when the gradient of the activation function is close to zero for large inputs, which makes the optimization algorithm slow to converge.

Then, the most popular and recommended activation function in deep learning is the *Rectified Linear Unit (ReLU)*, plotted in Fig. 3.3 and defined as follows.

Definition 3.5 (ReLU function). The ReLU function is defined as

$$\text{ReLU}(x) = \max\{0, x\}, \quad x \in \mathbb{R}.$$

This function has several advantages: it is computationally efficient to apply to any input as it behaves as the identity function on the positive real plane, and easy to differentiate. Hence, the gradient of the ReLU function is

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 0, & \text{if } x < 0, \\ 1, & \text{if } x > 0. \end{cases}$$

Note that this function is not differentiable at 0. However, to avoid optimization issues, one can extend the gradient of the ReLU function at 0 to be 0, as it is done in Pytorch.

```
1 import torch
2
3 x = torch.tensor([0.0], requires_grad=True)
4 y = torch.relu(x)
5 y.backward()
6
7 >>> x.grad
```

8 `tensor([0.])`

Many other activation functions have been proposed in the literature that even include some trainable parameters, such as the leaky ReLU, defined as

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x > 0, \\ \alpha x, & \text{if } x \leq 0, \end{cases}$$

where α is a hyperparameter of the model. The leaky ReLU function is used to avoid the dying ReLU problem, which occurs when the ReLU function outputs zero for all inputs. The leaky ReLU function allows a small gradient for negative inputs, which can help the optimization algorithm to converge faster. See also [Ramachandran et al., 2017] that performed an automatic search of activation functions, which led to the introduction of the swish unit: $\text{swish}(x) = x\sigma(x)$, where σ is the sigmoid function.

3.4.2 Weights initialization

For this section, the lecture material is [Goodfellow et al., 2016, Section 8.4].

Example 3.6 (Kaiming/He initialization). Consider a fully connected neural network with L layers, where the k th layer has n_{k+1} neurons (input of dimension n_k and output of dimension n_{k+1}). We denote by $\mathbf{W}^{(k)} \in \mathbb{R}^{n_{k+1} \times n_k}$ the weight matrix of layer k , and by $\mathbf{b}^{(k)} \in \mathbb{R}^{n_{k+1}}$ the bias vector of layer k . We assume that the activation function is the ReLU function. At a given layer, one has

$$\begin{aligned} y^{(k)} &= W^{(k)}x^{(k)} + b^{(k)} \\ x^{(k)} &= \sigma(y^{(k-1)}). \end{aligned}$$

We initialize the weights and biases as i.i.d. as follows:

$$b^{(k)} = 0, \quad W_{ij}^{(k)} \sim \mathcal{U}[-\alpha_k, \alpha_k],$$

and would like to choose α_k such that the variance of the outputs of each layer is the same as the variance of the inputs:

$$\text{Var}(y^{(k)}) = \text{Var}(x^{(k)}).$$

At initialization, we have

$$y_i^{(k)} = \sum_{j=1}^{n_k} W_{ij}^{(k)} x_j^{(k)}.$$

Hence, since the weights are independent, we have

$$\mathbb{E}[y_i^{(k)}] = \sum_{j=1}^{n_k} \mathbb{E}[x_j^{(k)}] \mathbb{E}[W_{ij}^{(k)}] = 0.$$

Moreover, since the $W_{i,j}x_j$ are i.i.d., we have

$$\begin{aligned} \text{Var}(y_i^{(k)}) &= n_k \text{Var}(W_{ij}^{(k)} x_j^{(k)}) \\ &= n_k \left(\mathbb{E}[W_{i,j}^{(k)2}] \mathbb{E}[x_j^{(k)2}] - \underbrace{\mathbb{E}[W_{i,j}^{(k)}]^2}_{=0} \mathbb{E}[x_j^{(k)}]^2 \right). \end{aligned}$$

Since $W_{i,j}^{(k)} \sim \mathcal{U}[-\alpha_k, \alpha_k]$, we have $\mathbb{E}[W_{i,j}^{(k)2}] = \alpha_k^2/3$.

$$\mathbb{E}[x_j^{(k)2}] = \mathbb{E}[\sigma(y_j^{(k-1)})^2] = \frac{1}{2} \mathbb{E}[y_j^{(k-1)2}] = \frac{1}{2} \text{Var}(y_j^{(k-1)}),$$

where we used the fact that $y_j^{(k-1)}$ is symmetric around 0 at initialization ($y_j^{(k-1)}$ and $-y_j^{(k-1)}$ have the same distribution because the coefficients of $W^{(k)}$ are i.i.d. centered at 0). Finally, we have

$$\text{Var}(y_i^{(k)}) = n_k \frac{\alpha_k^2}{3} \frac{1}{2} \text{Var}(y_j^{(k-1)}) = \frac{n_k \alpha_k^2}{6} \text{Var}(y_j^{(k-1)}).$$

To ensure that the variance is preserved across layers, we choose

$$\alpha_k = \sqrt{\frac{6}{n_k}}.$$

This initialization is called the Kaiming or He initialization [He et al., 2015].

3.4.3 Dropout

Dropout [Srivastava et al., 2014] is a powerful method for regularizing deep neural networks and reducing overfitting by avoiding depending on a small subset of nodes. The dropout layer works by randomly setting a fraction of the input units to zero during training. It can informally be seen as training an ensemble of smaller neural networks in a similar way as bagging (see this blogpost on bagging vs boosting: <https://www.kaggle.com/code/prashant111/bagging-vs-boosting>).

To learn with bagging: we define k different models and datasets by sampling the training datasets with replacement and train model i on dataset i for $1 \leq i \leq k$. Then, the final prediction is obtained by combining the different models through a global average. This method is called *bagging* (bootstrap aggregating).

Here, we use a mini-batch learning algorithm (such as stochastic gradient descent) and at every batch, we randomly sample a binary mask to the input and hidden units of the network. The dropout probability of including a unit is a hyperparameter of the model and is usually set to 0.5. The dropout layer is only applied during training and is removed during testing.

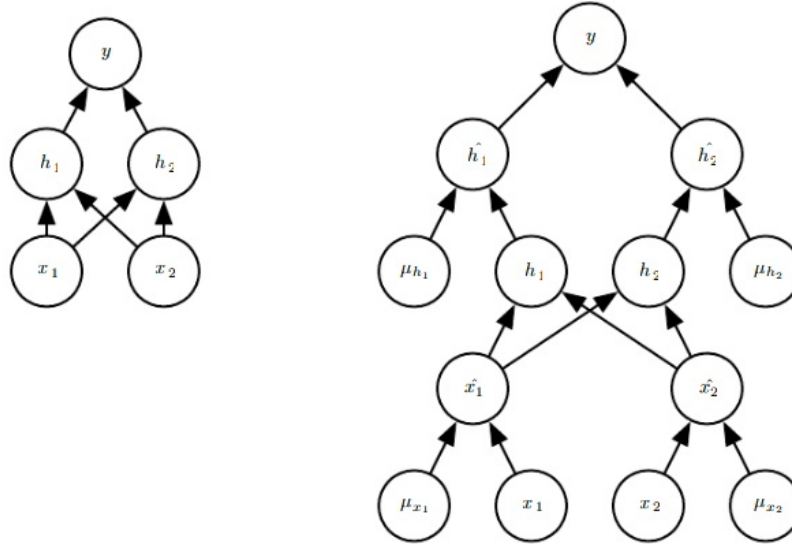


Figure 3.7: Left: feedforward neural network with 2 inputs, 2 hidden unit and one output. Right: Forward propagation with dropout. We randomly sample a binary vector μ of size $4 = 2 + 2$ (number of inputs + number of hidden units), and evaluate the corresponding network with forward propagation. This is equivalent to randomly sampling a random subnetwork and running forward propagation with it. Figure adapted from [Goodfellow et al., 2016, Fig. 7.7].

3.4.4 Convolution neural networks (CNNs)

Convolutional neural networks (CNNs) are a type of neural network architectures that processes inputs defined on a grid, such as time series or images. They have been widely used in many computer vision tasks, such as image classification, and rely on a mathematical operation called the convolution operator.

3.4.4.1 Convolution

Convolution is a specific type of linear operation that is used in CNNs to extract features from the input data. It is used in place of the weight matrix multiplication

in fully connected neural networks to exploit the spatial structure of the input data and reduce the number of parameters in the model, i.e., share some parameters across different units.

Definition 3.7 (Convolution of functions). Let f and g be two functions defined on $\mathbb{R}$, the convolution of f and g is defined as

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t - \tau) d\tau, \quad t \in \mathbb{R}.$$

One condition for existence of the convolution is that $f, g \in L^1(\mathbb{R})$, in this case, the convolution is well-defined and belongs to $L^1(\mathbb{R})$ by Fubini–Tonelli’s theorem.

In machine learning terminology, the function f is referred to as the *input* while g is denoted as the *kernel*. One commonly use an image I (i.e., a two dimensional array) as the input and a kernel matrix K as the convolutional layer’s parameters. The kernel is a small matrix that is convolved with the input to produce a feature map. The kernel is applied to the input by sliding it across the input data and computing the dot product between the kernel and the input at each position. The output of the convolution is called a feature map.

As the convolutional operation leads to an infinite summation, one assumes that the input and kernel are zero-padded (i.e., extended with zero) to ensure that the output has the same size as the input. The convolution operation is defined as

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n)K(i - m, j - n).$$

Since the convolution operation is commutative, it is usually more convenient to flip the kernel and write

$$S(i, j) = (K * I)(i, j) = \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} I(i - m, j - n)K(m, n),$$

where N is the kernel width and height. Usually, one chooses the kernel to be square with odd parameters to ensure that the kernel has a center (e.g., $M = N = 3$ or $M = N = 5$).

Most deep learning software actually implement the *cross-correlation operation* and call it convolution. It is similar to the convolution operation but without flipping the kernel.

Definition 3.8 (Cross-correlation). Let I be a $d_1 \times d_2$ input matrix and K be a $M \times N$ kernel matrix, the cross-correlation operation is defined as

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} I(i+m, j+n) K(m, n),$$

for $0 \leq i \leq d_1 - 1$, and $0 \leq j \leq d_2 - 1$. Note that we extended I by zero whenever we evaluate it outside of its domain.

The amount of *padding* applied to the input determines the shape of the output images. It is usually chosen to ensure that the output has the same size as the input. The padding is applied to the input by adding zeros around the input data.

An important hyperparameter of the convolutional layer is the *stride*, which determines the step size of the kernel when sliding it across the input data. The stride is usually set to 1 to ensure that the kernel covers the entire input data. However, one can increase the stride to reduce the size of the output feature map.

Another important parameter is the *number of channels*, which determines the number of kernels applied to the input data. As an example, if an input image of dimension $d_1 \times d_2$ goes through a convolutional layer with 2 channels, the output of the layer will be a tensor of shape $(2, d_1, d_2)$. Each convolutional kernel will contain trainable parameters that are learned during the training of the neural network.

All of these parameters affect the output size of the convolutional layer (see <https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html> for more details concerning the Pytorch implementation).

Motivation and intuition. One of the motivations for introducing convolutional layers is computational efficiency. The convolution operation reduces the number of parameters in the model by sharing the weights across different units (see Fig. 3.9). This is particularly useful when processing images, as the input data has a spatial structure that can be exploited by the convolutional operation. Here, one would want pixels that are close to each other to be connected to the output nodes, as they are likely to contain similar information. This is achieved by the convolution operation, which applies the same kernel to different parts of the input data.

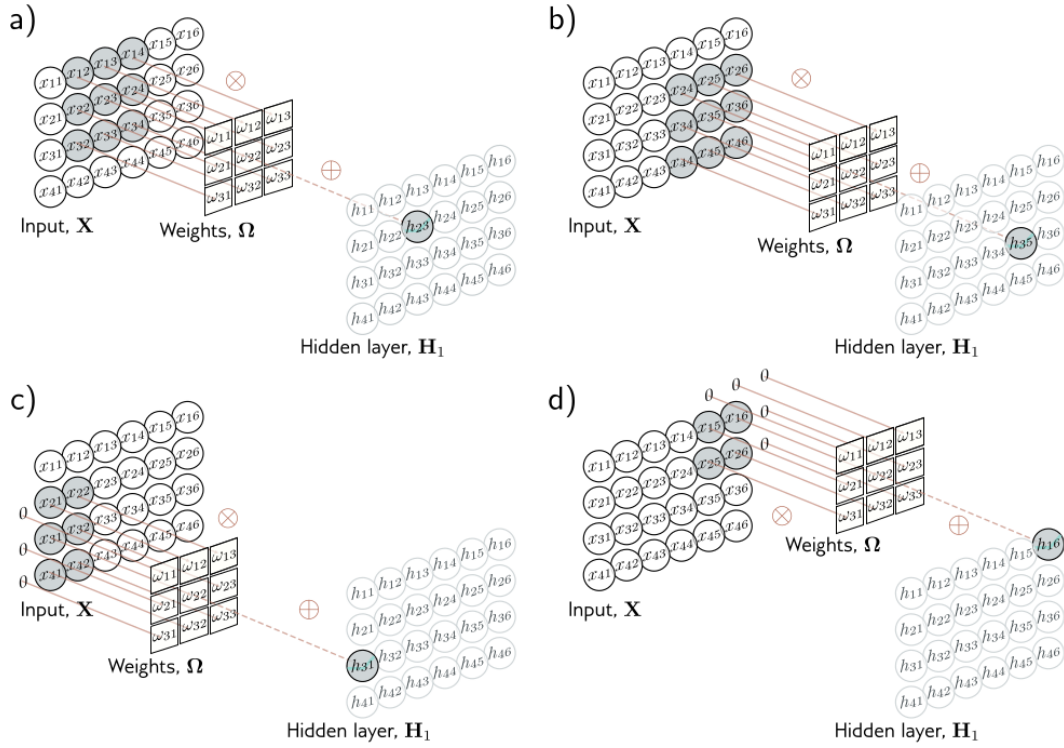


Figure 3.8: Application of a two-dimensional convolutional layer. Figure taken from [Prince, 2023, Fig. 10.9].

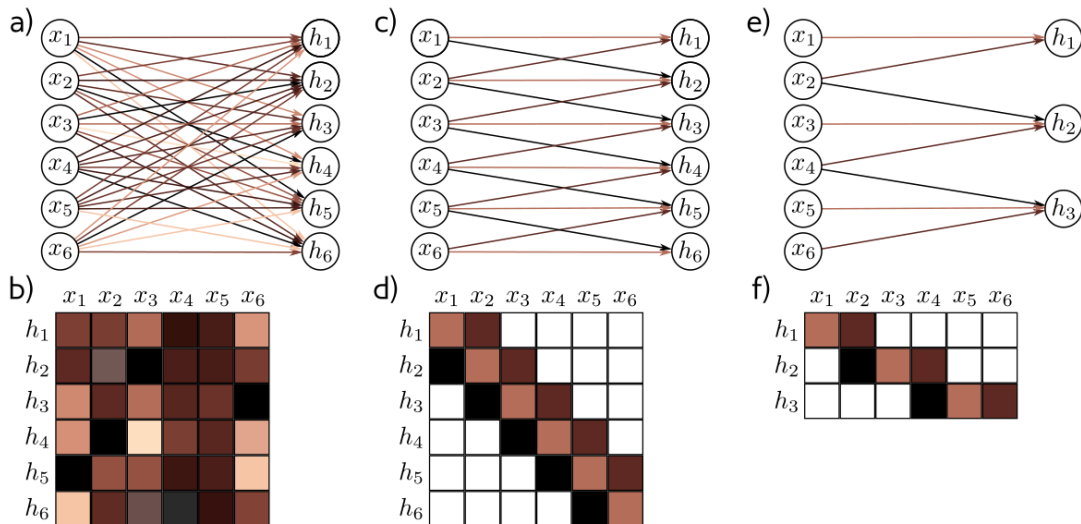


Figure 3.9: Comparison between fully connected and 1D convolutional layer. A convolutional layer is equivalent to multiplication by a structured matrix, called Toeplitz matrix that share weights across different units. Figure taken from [Prince, 2023, Fig. 10.4].

3.4.4.2 Pooling

The pooling layer is an important operator in convolutional neural networks that usually take place after the convolution and nonlinear activation function to make the neural network approximately invariant with respect to small translation perturbations. If $I \in \mathbb{R}^{d \times d}$ is an input image, the *maximum pooling* operation is defined as

$$\text{pool}(I)_{i,j} = \max_{m,n \in \mathcal{N}(i,j)} I_{m,n},$$

where $\mathcal{N}(i,j)$ is a rectangular neighborhood of the pixel (i,j) in the input image. The pooling operation is usually applied to each channel of the input image independently. The most common pooling operation is the max pooling, which takes the maximum value of the input data in the neighborhood of the pixel. Other pooling operations include the average pooling, which takes the average value of the input data in the neighborhood of the pixel. One can also combine pooling with downsampling to reduce the size of the input data by using a sliding window with a stride parameter greater or equal than 2.

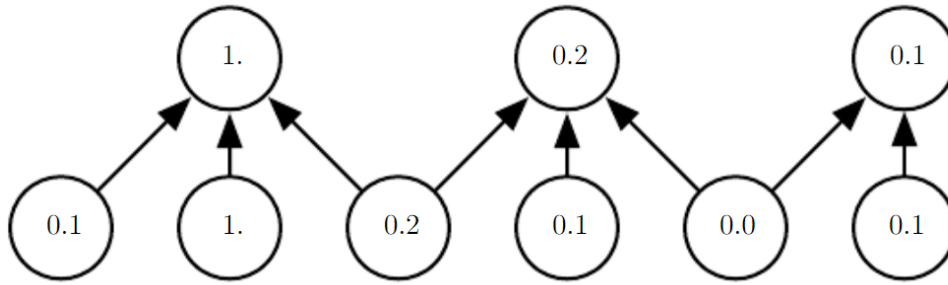


Figure 3.10: Example of max pooling operation with downsampling. Here, we use a pooling with a width of 3 and stride 2. This decreases the number of nodes by a factor of two at the next layer. Figure taken from [Goodfellow et al., 2016, Fig. 9.10].

Chapter 4: Approximation theory of neural networks

In this chapter, we aim to understand why neural networks are powerful function approximators and how they can approximate smooth functions. We will begin with a standard result regarding the universal approximation ability of neural networks and then discuss the benefit of depth, i.e., using many layers, in deep learning.¹

4.1 Universal approximation theorem

4.1.1 Univariate case

We begin by studying the univariate case and consider simple piecewise constant neural network.

Proposition 4.1. Let $g : \mathbb{R} \rightarrow \mathbb{R}$ be a ρ -Lipschitz function. For any $\epsilon > 0$, there exists a 2-layer network f with $\lceil \frac{\rho}{\epsilon} \rceil$ threshold nodes $z \mapsto \mathbf{1}[z \geq 0]$ such that $\sup_{x \in [0,1]} |f(x) - g(x)| \leq \epsilon$.

Proof. Let $m := \lceil \frac{\rho}{\epsilon} \rceil$, we construct the network f as

$$f(x) = \sum_{i=0}^{m-1} a_i \mathbf{1}[x \geq b_i], \quad (4.1)$$

where $b_i := i\epsilon/\rho$ for $i \in \{0, \dots, m-1\}$, and

$$a_0 = g(0), \quad a_i = g(b_i) - g(b_{i-1}).$$

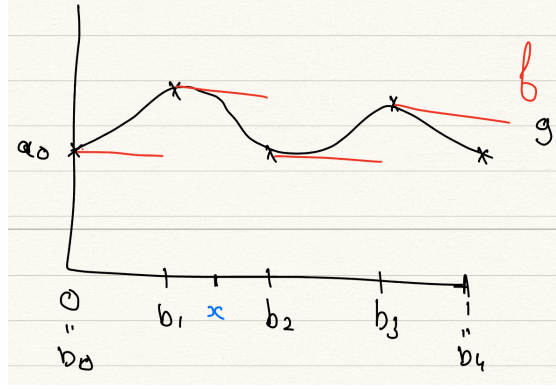
Then, for any $x \in [0, 1]$, we denote k to be the largest index so that $b_k \leq x$:

$$k = \operatorname{argmax}_{1 \leq i \leq m-1} \{i \mid b_i \leq x\}.$$

Then, for any $\tilde{x} \in [b_k, x]$, we have

$$f(\tilde{x}) = \sum_{i=0}^k a_i \mathbf{1}[\tilde{x} \geq b_i] + \sum_{i=k+1}^{m-1} a_i \mathbf{1}[\tilde{x} \geq b_i] = \sum_{i=1}^k a_i,$$

¹The chapter is based on the lecture notes on deep learning theory written by Matus Telgarsky.

Figure 4.1: Piecewise constant approximation of g by f .

and f is constant along $[b_k, x]$. Therefore,

$$\begin{aligned}
 |g(x) - f(x)| &\leq |g(x) - g(b_k)| + |g(b_k) - f(b_k)| + |f(b_k) - f(x)| \\
 &\leq \rho|x - b_k| + \left| g(b_k) - \sum_{i=0}^k a_i \right| + 0 \\
 &\leq \rho(\epsilon/\rho) + \left| g(b_k) - g(b_0) - \sum_{i=1}^k (g(b_i) - g(b_{i-1})) \right| \\
 &= \epsilon.
 \end{aligned}$$

Since for all $x \in [0, 1]$, we have $|g(x) - f(x)| \leq \epsilon$, we conclude that $\sup_{x \in [0, 1]} |g(x) - f(x)| \leq \epsilon$. $\square$

4.1.2 Multivariate case

Now let's handle the multivariate case. We will replicate the univariate approach: we will increment function values when the target function changes. In the univariate case, we could “localize” function modifications, but in the multivariate case by default we will modify an entire halfspace at once. To get around this, we use an additional layer.

Theorem 4.2. Let $g : \mathbb{R}^d \rightarrow \mathbb{R}$ be a ρ -Lipschitz function and $\epsilon > 0$. Then, there exists a 3-layer network f with $\Omega(\lceil \frac{\rho}{\epsilon} \rceil^d)$ ReLU nodes such that $\int_{[0, 1]^d} |f(x) - g(x)| dx \leq 2\epsilon$.

Remark 4.3. We note that the curse of dimensionality (exponential dependence on d) also appears in the lower bounds [von Luxburg and Bousquet, 2004]. Note CIFAR has $d = 3072$. This issue is inherent in approximating arbitrary continuous

functions, and makes this irrelevant in practice. Construction also has large weights and Lipschitz constant. Later in Theorem 4.10 (Hornik, Stinchcombe, and White 1989) we will give another approach that controls $\sup_x |f(x) - g(x)|$ and uses only one activation layer, but it will not be a constructive proof, and trying to obtain estimates from it has all the preceding weaknesses as well.

The proof of Theorem 4.2 uses the following lemma, which approximates continuous functions by piecewise constant functions.

Lemma 4.4 (Piecewise constant approximation). Let g, ρ, ϵ be given as in Theorem 4.2. Let $k = \lceil \rho/(\epsilon\sqrt{d}) \rceil$, $m = k^d$, and $\mathcal{P} = (R_1, \dots, R_m)$ be a partition of $[0, 1]^d$ into half-open cubes of side length $1/k$. Then, there exist scalars $(\alpha_1, \dots, \alpha_m)$ so that

$$\sup_{x \in U} |g(x) - h(x)| \leq \epsilon, \quad \text{where} \quad h = \sum_{i=1}^m \alpha_i \mathbf{1}_{R_i}.$$

Proof. Let $k := \lceil \frac{\rho}{\epsilon\sqrt{d}} \rceil$, $m = k^d$, and $\mathcal{P} = (R_1, \dots, R_m)$ be a partition of $[0, 1]^d$ into half-open cubes of side length $1/k$. If each cube R_j has corners $u_j \in \mathbb{R}^d$ and $v_j \in \mathbb{R}^d$, then we can write $R_j = \times_{i=1}^d [u_{j,i}, v_{j,i})$. Define $a_j := g(u_j)$ and consider the piecewise-constant function $h : \mathbb{R}^d \rightarrow \mathbb{R}$ defined as

$$h(x) = \sum_{j=1}^m a_j \mathbf{1}_{R_j}(x).$$

For any $x \in \mathbb{R}^d$, we denote by R_s the cube containing x , and we have

$$|h(x) - g(x)| = |a_s - g(x)| = |g(u_s) - g(x)| \leq \rho \|u_s - x\|_2 \leq \epsilon.$$

Here, we use the standard inequality $\|\cdot\|_2 \leq \sqrt{d} \|\cdot\|_\infty$ and use the fact that the cube has side length $1/k$ (so the infinity norm of the difference of two points in the cube is smaller than $1/k$). $\square$

We are now ready to prove Theorem 4.2.

Proof of Theorem 4.2. The proof is complete if for each R_j we can construct a 3-layer ReLU network f_j with

$$\int_{[0,1]^d} |f_j(x) - \mathbf{1}_{R_j}| dx \leq \tau := \frac{\epsilon}{\sum_{j=1}^m |a_j|}.$$

Hence, $f(x) := \sum_{j=1}^m a_j f_j(x)$ is also a 3-layer ReLU network and satisfies

$$\begin{aligned}
 \int_{[0,1]^d} |f(x) - g(x)| \, dx &= \int_{[0,1]^d} |f(x) - h(x)| + |h(x) - g(x)| \, dx \\
 &\leq \int_{[0,1]^d} \left| \sum_{j=1}^m a_j (f_j(x) - \mathbf{1}_{R_j}(x)) \right| \, dx + \epsilon \\
 &\leq \sum_{j=1}^m |a_j| \int_{[0,1]^d} |f_j(x) - \mathbf{1}_{R_j}(x)| \, dx + \epsilon \\
 &\leq \sum_{j=1}^m |a_j| \tau + \epsilon \leq 2\epsilon.
 \end{aligned}$$

We will now see how we can construct f_j for $1 \leq j \leq m$. Let $1 \leq j \leq m$ and $R_j = [u_j, v_j]$. We are going to drop the subscript j , which is fixed, for convenience. Let $\gamma > 0$ be arbitrary and for each $i \in \{1, \dots, d\}$ define

$$\begin{aligned}
 f_{\gamma,i}(z) &:= \sigma\left(\frac{z - (u_i - \gamma)}{\gamma}\right) - \sigma\left(\frac{z - u_i}{\gamma}\right) - \sigma\left(\frac{z - v_i}{\gamma}\right) + \sigma\left(\frac{z - (v_i + \gamma)}{\gamma}\right) \\
 &\in \begin{cases} \{1\}, & z \in [u_i, v_i], \\ \{0\}, & z \notin [u_i - \gamma, v_i + \gamma], \\ [0, 1], & \text{otherwise.} \end{cases}
 \end{aligned}$$

and additionally

$$f_\gamma(x) := \sigma\left(\sum_{i=1}^d f_{\gamma,i}(x_i) - (d-1)\right).$$

Note that a second hidden layer is crucial in this construction, it is not clear how to proceed without it, certainly with only $\mathcal{O}(d)$ nodes. Later proofs can use only a single hidden layer, but they are not constructive, and need $\mathcal{O}(d)$ nodes.

Note that $f_\gamma \approx \mathbf{1}_{R_j}$ as desired, specifically

$$f_\gamma(x) = \begin{cases} 1 & x \in R_j, \\ 0 & x \notin \times_i [u_i - \gamma, v_i + \gamma], \\ [0, 1] & \text{otherwise,} \end{cases}$$

From which it follows that

$$\begin{aligned}
 \int_{[0,1]^d} |f_\gamma(x) - \mathbf{1}_{R_j}(x)| \, dx &= \int_{R_j} |f_\gamma - \mathbf{1}_{R_j}| \, dx + \int_{\times_{i=1}^d [u_i - \gamma, v_i + \gamma] \setminus R_j} |f_\gamma - \mathbf{1}_{R_j}| \, dx \\
 &\quad + \int_{[0,1]^d \setminus \times_{i=1}^d [u_i - \gamma, v_i + \gamma]} |f_\gamma - \mathbf{1}_{R_j}| \, dx
 \end{aligned}$$

$$\leq 0 + \prod_{i=1}^d (v_i - u_i + 2\gamma) - \prod_{i=1}^d (v_i - u_i) + 0.$$

We know that $v_i - u_i = 1/k$. Moreover, with $x = 1/k + 2\gamma$ and $y = 1/k$, we have

$$|x - y|^d = |x - y|(x^{d-1} + x^{d-2}y + \dots + xy^{d-2}y^{d-1}) \leq d(1/k + 2\gamma)^{d-1}(2\gamma) = \mathcal{O}(\gamma),$$

as $d(1/k + 2\gamma)^{d-1}(2\gamma)/\gamma$ is bounded as $\gamma \rightarrow 0$. Therefore,

$$\int_{[0,1]^d} |f_\gamma(x) - \mathbf{1}_{R_j}(x)| dx = \mathcal{O}(\gamma),$$

which means we can ensure $\|\mathbf{1}_{R_j} - f_\gamma\|_1 \leq \frac{\epsilon}{\sum_i |\alpha_i|}$ by choosing sufficiently small γ , which completes the proof. $\square$

4.1.3 Single-layer neural networks are universal approximators

The proof of Theorem 4.2 uses two layers to construct y_γ such that $g_\gamma(x) \approx \mathbf{1}[x \in \times_i [a_i, b_i]]$. If instead we had a way to approximate multiplication we could approximate the function

$$x \mapsto \prod_i \mathbf{1}[x_i \in [a_i, b_i]] = \mathbf{1}[x \in \times_i [a_i, b_i]]. \quad (4.2)$$

Can we do this and then form a linear combination, all with just one hidden layer?

The answer will be yes, and we will use this to resolve the classical universal approximation question with a single hidden layer.

Definition 4.5 (Universal approximator). A class of functions $\mathcal{F}$ is a universal approximator over a compact set S if, for every continuous function g and target accuracy $\epsilon > 0$, there exists $f \in \mathcal{F}$ such that

$$\sup_{x \in S} |f(x) - g(x)| \leq \epsilon. \quad (4.3)$$

Remark 4.6. Typically, we will take $S = [0, 1]^d$; we can then reduce arbitrary compact sets to this case by defining a new function which re-scales the input.

Consider unbounded width networks with one hidden layer.

$$\mathcal{F}_{\sigma,d,m} := \mathcal{F}_{d,m} := \{x \mapsto a^\top \sigma(Wx + b) : a \in \mathbb{R}^m, W \in \mathbb{R}^{m \times d}, b \in \mathbb{R}^m\}.$$

$$\mathcal{F}_{\sigma,d} := \mathcal{F}_d := \bigcup_{m \geq 0} \mathcal{F}_{\sigma,d,m}.$$

Note that $\mathcal{F}_{\sigma,m,1}$ denotes networks with a single node, and $\mathcal{F}_{\sigma,d}$ is the linear span (in function space) of single-node networks.

The classical Weierstrass theorem establishes that polynomials are universal approximators [Weierstrass, 1885], and its generalization, the Stone–Weierstrass theorem, says that any family of functions satisfying some of the same properties as polynomials will also be a universal approximator. Thus we will show $\mathcal{F}_{\sigma,d}$ is a universal approximator via Stone–Weierstrass, a key step being closure under multiplication; this proof scheme was first suggested in [Hornik et al., 1989], but is now a fairly standard way to prove universal approximation.

First, here is the statement of the Stone–Weierstrass Theorem [Folland, 1999, Thm. 4.45].

Theorem 4.7 (Stone–Weierstrass). Let $\mathcal{F}$ be a class of functions from $\mathbb{R}^d \rightarrow \mathbb{R}$ with the following properties:

1. Each $f \in \mathcal{F}$ is continuous.
2. For every x there exists $f \in \mathcal{F}$ with $f(x) \neq 0$.
3. For every $x \neq x'$ there exists $f \in \mathcal{F}$ with $f(x) \neq f(x')$ ($\mathcal{F}$ separates points).
4. $\mathcal{F}$ is closed under multiplication and vector space operations ($\mathcal{F}$ is an algebra)

Then $\mathcal{F}$ is a universal approximator: for every continuous $g : \mathbb{R}^d \rightarrow \mathbb{R}$ and $\epsilon > 0$, there exists $f \in \mathcal{F}$ with $\sup_{x \in [0,1]^d} |f(x) - g(x)| \leq \epsilon$.

Remark 4.8.

- This is a heavyweight tool, but a convenient way to quickly check universal approximation.
- Proofs are not constructive, but the size lower bound $\Omega(\frac{1}{\epsilon^d})$ seems to naturally appear in various places; e.g., to show closure under products as above, we double (or more) the number of terms for each dimension.
- Weierstrass theorem itself has interesting proofs

- The modern standard one is due to Bernstein; it picks a fine grid and then a convenient set of interpolating polynomials which behave stably off the grid.
- Weierstrass's original proof convolved the target with a Gaussian, which makes it analytic, and also leads to good polynomial approximation.
- The second and third conditions in Stone-Weierstrass are necessary; if there exists x such that $f(x) = 0 \forall f \in \mathcal{F}$, then we can't approximate g with $g(x) \neq 0$; if we can't separate points $x \neq x'$, then we can't approximate functions with $g(x) \neq g(x')$.

Lemma 4.9. $\mathcal{F}_{\exp,d}$ is universal.

Proof. Let's check the Stone-Weierstrass conditions:

1. Each $f \in \mathcal{F}_{\exp,d}$ is continuous.
2. For each x , $\exp(0^\top x) = 1 \neq 0$.
3. For each $x \neq x'$, $f(z) := \exp((z - x')^\top (x - x') / \|x - x'\|^2) \in \mathcal{F}_d$ satisfies

$$f(x) = \exp(1) \neq \exp(0) = f(x').$$

4. $\mathcal{F}_{\exp,d}$ is closed under vector space operations by construction; for products

$$\left(\sum_{i=1}^n r_i \exp(a_i^\top x) \right) \left(\sum_{j=1}^m s_j \exp(b_j^\top x) \right) = \sum_{i=1}^n \sum_{j=1}^m r_i s_j \exp((a_i + b_j)^\top x).$$

□

Now let's handle arbitrary activations.

Theorem 4.10 (Hornik, Stinchcombe, and White 1989). Suppose $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is sigmoidal: it is continuous, and

$$\lim_{z \rightarrow -\infty} \sigma(z) = 0, \quad \lim_{z \rightarrow +\infty} \sigma(z) = 1.$$

Then $\mathcal{F}_{\sigma,d}$ is universal.

Remark 4.11. • ReLU is fine: use $z \mapsto \sigma(z) - \sigma(z - 1)$ and split nodes.

- $\exp$ didn't need bias in the proof, but this seems natural due to $\exp(a^\top x + b) = e^b \cdot \exp(a^\top x)$. On the other hand, approximating $\exp$ with ReLU uses bias terms, so we don't obtain a trick via $\exp$ to remove biases in general.
- Weakest conditions on σ [Leshno et al., 1993]: universal approximation if and only if σ is not a polynomial.
- Carefully accounting within the proof seems to indicate curse of dimension again (size $\Omega(\frac{1}{\epsilon^d})$), due for instance to expanding all terms in a product of d terms.

4.2 Approximation power of neural networks

For this section, I refer to the paper [Yarotsky, 2017] and in particular Theorem 4. We consider functions in the Sobolev spaces $W^{n,\infty}([0, 1]^d)$, for $n \geq 1$, which are the spaces of functions with bounded n th order weak derivatives. The norm in $W^{n,\infty}([0, 1]^d)$ is given by

$$\|f\|_{W^{n,\infty}} = \max_{\mathbf{n}: |\mathbf{n}| \leq n} \text{ess sup}_{\mathbf{x} \in [0,1]^d} |D^{\mathbf{n}} f(\mathbf{x})|,$$

where $D^{\mathbf{n}} f$ denotes the n th weak derivative of f , and $|\mathbf{n}| = \sum_{i=1}^d n_i$, with $n_i \in \{0, \dots, n\}$. This space is equivalent to the space of functions in $C^{n-1}([0, 1]^d)$ such that their derivative of order $n-1$ are Lipschitz continuous. We denote by $F_{d,n}$ the unit ball in $W^{n,\infty}([0, 1]^d)$, i.e.,

$$F_{d,n} = \{f : [0, 1]^d \rightarrow \mathbb{R} : \|f\|_{W^{n,\infty}} \leq 1\},$$

and aim to prove the following theorem:

Theorem 4.12 (Yarotsky, 2017). Let $d, n, \epsilon \in (0, 1)$, and $f \in F_{d,n}$. There exists a ReLU network R_ϵ with at most $c(\ln(1/\epsilon) + 1)$ layers and at most $c\epsilon^{-d/n}(\ln(1/\epsilon) + 1)$ weights such that

$$\|f - R_\epsilon\|_{L^\infty([0,1]^d)} \leq \epsilon,$$

where $c = c(d, n)$ is a constant that depends only on d and n .

Remark 4.13. While the network architecture is fixed for any function in $F_{d,n}$ (i.e. the number of layers, structure, ...), the neural network R_ϵ depends on the function f that we wish to approximate.

The proof of Theorem 4.12 relies on a local Taylor approximation of the function (see Lemma 4.14), and the approximation of each of the terms (piecewise linear functions and polynomials) in the Taylor approximation by ReLU networks. The proof is quite technical and we will only provide a sketch of the main ideas.

Lemma 4.14. Let $f \in F_{d,n}$, and $\epsilon > 0$. There exists a piecewise polynomial function f_N with $(N+1)^d$ pieces, where $N = \mathcal{O}(\epsilon^{-1/n})$, and polynomials of degree less than n such that

$$\max_{\mathbf{x} \in [0,1]^d} |f(\mathbf{x}) - f_N(\mathbf{x})| \leq \epsilon/2.$$

Proof. Let $N \geq 1$ be an integer and consider a partition of unity of $(N+1)^d$ functions $\phi_{\mathbf{m}}$ on the domain $[0,1]^d$, i.e.,

$$\sum_{\mathbf{m} \in \{0,\dots,N\}^d} \phi_{\mathbf{m}}(\mathbf{x}) = 1, \quad \phi_{\mathbf{m}}(\mathbf{x}) = \prod_{k=1}^d \psi_{m_k}(x_k), \quad \mathbf{x} = (x_1, \dots, x_d),$$

where $\mathbf{m} = (m_1, \dots, m_d)$, and ψ_{m_k} is given by

$$\psi_{m_k}(x) = \begin{cases} 1, & \text{if } |x_k - \frac{m_k}{N}| < \frac{1}{3N}, \\ 0, & \text{if } |x_k - \frac{m_k}{N}| > \frac{2}{3N}, \\ 2 - 3N |x_k - \frac{m_k}{N}|, & \text{otherwise.} \end{cases}$$

We now define a local Taylor approximation of f by

$$f_N(\mathbf{x}) = \sum_{\mathbf{m} \in \{0,\dots,N\}^d} \phi_{\mathbf{m}}(\mathbf{x}) P_{\mathbf{m}}(\mathbf{x}),$$

where $P_{\mathbf{m}}$ denotes the degree $n-1$ Taylor polynomial of f at $\mathbf{x} = \mathbf{m}/N$. That is,

$$P_{\mathbf{m}}(\mathbf{x}) = \sum_{|\mathbf{n}| < n} \frac{D^{\mathbf{n}} f(\frac{\mathbf{m}}{N})}{\mathbf{n}!} \left(\mathbf{x} - \frac{\mathbf{m}}{N} \right)^{\mathbf{n}}, \quad (4.4)$$

where $|\mathbf{n}| = \sum_{k=1}^d n_k$, $\mathbf{n}! = \prod_{k=1}^d n_k!$, and $(\mathbf{x} - \mathbf{m}/N)^{\mathbf{n}} = \prod_{k=1}^d (x_k - m_k/N)^{n_k}$. Let $\mathbf{x} \in [0,1]^d$ and note that

$$\text{support}(\phi_{\mathbf{m}}) \subset \left\{ \mathbf{x} = (x_1, \dots, x_d) : \left| x_k - \frac{m_k}{N} \right| < \frac{1}{N} \right\}, \quad \mathbf{m} \in \{0, \dots, N\}^d.$$

Hence, the approximation error between f and its local Taylor approximation satisfies

$$|f(\mathbf{x}) - f_N(\mathbf{x})| = \left| \sum_{\mathbf{m} \in \{0,\dots,N\}^d} \phi_{\mathbf{m}}(f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})) \right|$$

$$\begin{aligned}
&\leq \sum_{\mathbf{m}: |x_k - \frac{m_k}{N}| < \frac{1}{N}} |f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})| \\
&\leq \frac{2^d d^n}{n!} \left(\frac{1}{N}\right)^n \max_{|\mathbf{n}|=n} \operatorname{esssup}_{\mathbf{x} \in [0,1]^d} |D^{\mathbf{n}} f(\mathbf{x})| \\
&\leq \frac{2^d d^n}{n!} \left(\frac{1}{N}\right)^n.
\end{aligned}$$

We now select

$$N = \left\lceil \left(\frac{n!}{2^d d^n} \frac{\epsilon}{2} \right)^{-1/n} \right\rceil,$$

so that

$$\max_{\mathbf{x} \in [0,1]^d} |f(\mathbf{x}) - f_N(\mathbf{x})| \leq \epsilon/2. \quad (4.5)$$

□

The approximation of polynomials by a ReLU network relies on the following key proposition ([Yarotsky, 2017, Prop. 2]).

Proposition 4.15 (Yarotsky, 2017). The function $f : x \mapsto x^2$ can be approximated on $[0, 1]$ within error ϵ by a ReLU network with $\mathcal{O}(\ln(1/\epsilon))$ number of layers and weights.

Proof. We employ a construction originally proposed by [Telgarsky, 2015] and consider the “tooth” function $g : [0, 1] \rightarrow [0, 1]$ defined by

$$g(x) = \begin{cases} 2x, & x < 1/2, \\ 2(1-x), & x \geq 1/2, \end{cases}$$

and the iterated functions

$$g_s(x) = \underbrace{g \circ g \circ \dots \circ g}_{s \text{ times}}(x), \quad x \in [0, 1]. \quad (4.6)$$

Telgarsky showed ([Telgarsky, 2015, Lem. 2.4]) that g_s is a “sawtooth” function with 2^{s-1} uniformly distributed “teeth” (each application of g doubles the number of teeths):

$$g_s(x) = \begin{cases} 2^s \left(x - \frac{2k}{2^s}\right), & x \in \left[\frac{2k}{2^s}, \frac{2k+1}{2^s}\right], \quad k = 0, 1, \dots, 2^{s-1} - 1, \\ 2^s \left(\frac{2k}{2^s} - x\right), & x \in \left[\frac{2k-1}{2^s}, \frac{2k}{2^s}\right], \quad k = 1, 2, \dots, 2^{s-1} - 1. \end{cases} \quad (4.7)$$

The function $f(x) = x^2$ can be approximated by linear combinations of the functions g_s . Let f_m be the piecewise linear interpolation of f with $2^m + 1$ uniformly distributed breakpoints located at $k/2^m$ for $k = 0, \dots, 2^m$:

$$f_m\left(\frac{k}{2^m}\right) = \left(\frac{k}{2^m}\right)^2, \quad k = 0, \dots, 2^m.$$

Hence,

$$f_m(x) = \frac{2k+1}{2^m}x - \frac{k^2+k}{2^{2m}}, \quad x \in \left[\frac{k}{2^m}, \frac{k+1}{2^m}\right].$$

On the interval $[k/2^m, (k+1)/2^m]$, the maximum error between f and f_m is attained at

$$x_k = \frac{2k+1}{2^{m+1}},$$

and

$$|f(x) - f_m(x)| \leq |f(x_k) - f_m(x_k)| = \frac{(2k+1)^2}{2^{2m+2}} - \frac{k^2+k}{2^{2m}} = \frac{1}{2^{2m+2}}.$$

Therefore, $\|f - f_m\|_{L^\infty([0,1])} \leq 2^{-2m-2}$. We now notice that

$$f_{m-1} - f_m = g_m(x)/2^{2m},$$

and deduce that

$$f_m(x) = x - \sum_{s=1}^m \frac{g_s(x)}{2^{2s}}.$$

We note that the function g (which we iterate to obtain the sawtooth function) can be implemented by a finite ReLU network as

$$g(x) = 2\sigma(x) - 4\sigma\left(x - \frac{1}{2}\right) + 2\sigma(x - 1).$$

Moreover, the construction of f_m involves $\mathcal{O}(m)$ linear operations and compositions of g so we can implement f_m with a ReLU network of $\mathcal{O}(m)$ layers and weights. The result follows by choosing $m = \mathcal{O}(\ln(1/\epsilon))$ such that $\epsilon_m \leq \epsilon$. $\square$

References

- [Balestrieri et al., 2021] Balestrieri, R., Pesenti, J., and LeCun, Y. (2021). Learning in high dimension always amounts to extrapolation. *arXiv preprint arXiv:2110.09485*.
- [Bárány and Füredi, 1988] Bárány, I. and Füredi, Z. (1988). On the shape of the convex hull of random points. *Probability theory and related fields*, 77:231–240.

- [Boyd and Vandenberghe, 2004] Boyd, S. P. and Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press.
- [Folland, 1999] Folland, G. B. (1999). *Real analysis: modern techniques and their applications*, volume 40. John Wiley & Sons.
- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep learning*. MIT Press.
- [He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034.
- [Higham and Higham, 2019] Higham, C. F. and Higham, D. J. (2019). Deep learning: An introduction for applied mathematicians. *Siam Review*, 61(4):860–891.
- [Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366.
- [James et al., 2013] James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An introduction to statistical learning*. Springer.
- [Kingma and Ba, 2014] Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [Leshno et al., 1993] Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural networks*, 6(6):861–867.
- [Prince, 2023] Prince, S. J. (2023). *Understanding Deep Learning*. The MIT Press.
- [Ramachandran et al., 2017] Ramachandran, P., Zoph, B., and Le, Q. V. (2017). Searching for activation functions. *arXiv preprint arXiv:1710.05941*.
- [Srivastava et al., 2014] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958.
- [Sutskever et al., 2013] Sutskever, I., Martens, J., Dahl, G., and Hinton, G. (2013). On the importance of initialization and momentum in deep learning. In *International conference on machine learning*, pages 1139–1147. PMLR.

- [Telgarsky, 2015] Telgarsky, M. (2015). Representation benefits of deep feedforward networks. *arXiv preprint arXiv:1509.08101*.
- [von Luxburg and Bousquet, 2004] von Luxburg, U. and Bousquet, O. (2004). Distance-Based Classification with Lipschitz Functions. *J. Mach. Learn. Res.*, 5(Jun):669–695.
- [Weierstrass, 1885] Weierstrass, K. (1885). Über die analytische Darstellbarkeit sogenannter willkürlicher Functionen einer reellen Veränderlichen. *Sitzungsberichte der Königlich Preussischen Akademie der Wissenschaften zu Berlin*, 2(633-639):364.
- [Yarotsky, 2017] Yarotsky, D. (2017). Error bounds for approximations with deep ReLU networks. *Neural networks*, 94:103–114.